Universidad de Cádiz
Escuela Superior de Ingeniería

TRABAJO FIN DE MÁSTER

MÁSTER EN SEGURIDAD INFORMÁTICA (CIBERSEGURIDAD)

SISTEMA DE DETECCIÓN Y CLASIFICACIÓN DE MALWARE UTILIZANDO ALGORITMOS DE MACHINE LEARNING Y FUENTES DE DATOS DIVERSAS

AUTOR/A: ALBERTO DOMÍNGUEZ MELLADO

Puerto Real, septiembre de 2025

Universidad de Cádiz
Escuela Superior de Ingeniería

TRABAJO FIN DE MÁSTER

MÁSTER EN SEGURIDAD INFORMÁTICA (CIBERSEGURIDAD)

SISTEMA DE DETECCIÓN Y CLASIFICACIÓN DE MALWARE UTILIZANDO ALGORITMOS DE MACHINE LEARNING Y FUENTES DE DATOS DIVERSAS

DIRECTOR: GABRIEL JOSE GUERRERO CONTRERAS
CO-DIRECTORA: SARA BALDERAS DÍAZ
AUTOR: ALBERTO DOMÍNGUEZ MELLADO

Puerto Real, septiembre de 2025

Resumen

Este proyecto habla sobre el desarrollo y la evaluación de un sistema para la detección de *malware*, utilizando técnicas de aprendizaje automático. El estudio se estructura en dos grandes partes: una primera, enfocada en la teoría de los modelos y la situación actual, y una segunda, centrada en el desarrollo de distintos algoritmos. El trabajo se inicia con una revisión del estado actual, analizando desde la seguridad en el Internet de las Cosas (IoT) hasta enfoques específicos como FEdroid y otras propuestas basadas en *Machine Learning*. A continuación, se especifican los fundamentos teóricos necesarios. En el área de *Machine Learning*, se explican los diferentes aprendizajes, se describen los algoritmos clave —como las máquinas de soporte vectorial (SVM), los árboles de decisión y las redes neuronales— y se detallan aspectos prácticos como los métodos de balanceo de datos, las métricas de evaluación y la validación cruzada. También, se define qué es el *malware*, su clasificación y las técnicas empleadas para su análisis. Finalmente, esta primera parte concluye con una descripción de las herramientas tecnológicas usadas, principalmente *Python* y las librerías utilizadas como *Scikit-learn*, *Pandas*, *Keras* e *Imbalanced-learn*. La fase principal del proyecto se centra en la metodología aplicada, que se inicia con la obtención y el preprocesamiento de los datos. Se realiza la ingeniería de características para enriquecer el *dataset*, la limpieza de valores nulos y la preparación de la variable objetivo para la clasificación. Una vez los datos están listos, se describe el *pipeline* de entrenamiento que se aplicará de forma estandarizada a todos los modelos, con la normalización de datos, el balanceo de clases y el uso de validación cruzada para garantizar la robustez de los resultados. Posteriormente, se lleva a cabo el entrenamiento de los diferentes algoritmos, entre los que se incluyen *CatBoost*, *LightGBM*, *XGBoost*, bosques aleatorios, KNN, redes neuronales y SVM. El documento finaliza con la presentación de los resultados obtenidos para cada uno de estos modelos, permitiendo realizar un análisis comparativo exhaustivo de su rendimiento y eficacia en la tarea de detección de *malware*. Entre los resultados más destacados, se desarrolla la creación de un *dataset* propio y que permitió una evaluación realista del problema, así como la identificación de que los modelos de *Gradient Boosting* —en particular LightGBM— ofrecieron el mejor rendimiento global, alcanzando métricas sobresalientes en términos de MCC y *F1-score* para la clase de "Riesgo bajo".

Palabras clave: Detección de Malware, Machine Learning, Ciberseguridad, Clasificación, Análisis de Características

Abstract

This project focuses on the development and evaluation of a system for detecting malware, using machine learning techniques. The study is structured into two main parts: the first one, focused on the theory of the models and the current state of the field, and the second one, centered on the development of different models. The work begins with a review of the current state, analyzing aspects ranging from security in the Internet of Things (IoT) to specific approaches such as FEdroid and other proposals based on Machine Learning. Next, the necessary theoretical foundations are specified. In the area of Machine Learning, the different learning paradigms are explained, key algorithms are described —such as support vector machines (SVM), decision trees, and neural networks— and practical aspects are detailed, including data balancing methods, evaluation metrics, and cross-validation. Additionally, the concept of malware, its classification, and the techniques used for its analysis are defined. Finally, this first part concludes with a description of the technological tools employed, mainly Python and libraries such as Scikit-learn, Pandas, Keras, and Imbalanced-learn. The main phase of the project focuses on the applied methodology, which begins with the collection and preprocessing of the data. Feature engineering is carried out to enrich the dataset, missing values are cleaned, and the target variable is prepared for classification. Once the data are ready, the training pipeline to be applied in a standardized way to all models is described, including data normalization, class balancing, and the use of cross-validation to ensure the robustness of the results. Subsequently, the training of different algorithms is carried out, including CatBoost, LightGBM, XGBoost, random forests, KNN, neural networks, and SVM. The document concludes with the presentation of the results obtained for each of these models, allowing for a comprehensive comparative analysis of their performance and effectiveness in the task of malware detection. Among the most outstanding results, the creation of a dedicated dataset that enabled a realistic evaluation of the problem is highlighted, as well as the identification that Gradient Boosting models —particularly LightGBM— offered the best overall performance, achieving outstanding metrics in terms of MCC and F1-score for the “Low risk” class.

Keywords: Malware Detection, Machine Learning, Cybersecurity, Classification, Feature Analysis

Índice general

Índice de figuras **11**

Índice de tablas **14**

I Prolegómeno **1**

1. Introducción **2**
1.1. Motivación . . . 2
1.2. Alcance y objetivos . . . 3
1.2.1. Alcance del proyecto . . . 3
1.2.2. Objetivos . . . 4
1.3. Planificación del proyecto . . . 5
1.4. Estimación de costes . . . 7
1.5. Glosario de términos . . . 8
1.6. Organización del documento . . . 9

2. Antecedentes **13**
2.1. Estado del arte . . . 13

3. Fundamentos teóricos **19**
3.1. Machine learning y deep learning . . . 19
3.1.1. Aprendizaje no supervisado . . . 21
3.1.2. Aprendizaje supervisado . . . 22
3.1.3. Métodos de balanceo . . . 33
3.1.4. Métricas de evaluación en clasificación . . . 35
3.1.5. Matriz de confusión . . . 38
3.1.6. Cross-Validation . . . 39
3.1.7. Sobreajuste y subajuste . . . 39
3.2. Malware . . . 41
3.2.1. ¿Qué es el *malware*? . . . 41
3.2.2. Clasificación del *malware* . . . 42
3.2.3. Características del *malware* . . . 44
3.2.4. Detección y análisis de *malware* . . . 45

4. Herramientas y tecnologías utilizadas **47**
4.1. Python . . . 47
4.2. Librerías de python . . . 47
4.2.1. Numpy . . . 47
4.2.2. Pandas . . . 48
4.2.3. Matplotlib . . . 48
4.2.4. Scikit-learn . . . 49

4.2.5. Seaborn . . . 49
4.2.6. Imbalanced-learn . . . 50
4.2.7. Keras . . . 50

II Desarrollo **52**

5. Metodología **54**
5.1. Obtención de datos . . . 54
5.2. Preprocesamiento de los datos . . . 61
5.2.1. Inclusión de datos complementarios . . . 61
5.2.2. Importación del dataset . . . 62
5.2.3. Eliminación de valores nulos . . . 62
5.2.4. Eliminación de columnas . . . 62
5.2.5. Ingeniería de características y preprocesamiento avanzado . . . 63
5.2.6. Resumen de columnas añadidas y eliminadas . . . 65
5.2.7. Clasificación variable objetivo . . . 66
5.2.8. Estudio exploratorio de los datos . . . 67
5.2.9. Exportación del dataset . . . 75
5.3. Pasos generales de entrenamiento de los algoritmos . . . 76
5.3.1. Imputación de valores nulos . . . 76
5.3.2. Creación dataset predictor y objetivo . . . 77
5.3.3. Tratamiento variables categóricas . . . 77
5.3.4. División de datos en entrenamiento y test . . . 78
5.3.5. Normalización . . . 78
5.3.6. Balanceo de clases . . . 79
5.3.7. Cross-Validation . . . 82
5.3.8. Entrenamiento del modelo . . . 84
5.4. Entrenamiento específico de los algoritmos . . . 86
5.4.1. CatBoost . . . 86
5.4.2. Árbol de decisión . . . 86
5.4.3. KNN . . . 90
5.4.4. LigthGBM . . . 92
5.4.5. MLP . . . 94
5.4.6. `RandomForest` . . . 96
5.4.7. SVM . . . 98
5.4.8. XGBoost . . . 100
5.4.9. Redes neuronales . . . 102

6. Resultados **105**
6.1. *CatBoost* . . . 105
6.2. *DecisionTree* . . . 107
6.2.1. Sin balanceo . . . 107
6.2.2. Oversampling . . . 109
6.2.3. *Undersampling* . . . 111
6.3. KNN . . . 116
6.3.1. *Oversampling* . . . 116
6.3.2. *Undersampling* . . . 118

6.4. LightGBM 123
6.5. MLP 125
6.5.1. *Oversampling* 125
6.5.2. *Undersampling* 128
6.6. *RandomForest* 134
6.6.1. Sin balanceo 134
6.6.2. *Oversampling* 136
6.6.3. *Undersampling* 138
6.7. SVM 141
6.7.1. *Oversampling* 141
6.7.2. *Undersampling* 143
6.8. *XGBoost* 147
6.8.1. Sin balanceo 147
6.8.2. *Oversampling* 149
6.8.3. *Undersampling* 151
6.9. Red Neuronal 154
6.9.1. *Oversampling* 154
6.9.2. *Undersampling* 157
6.10. Resultados y Discusión 166

III Epílogo **169**

7. Conclusiones **170**

Bibliografía **173**

Índice de figuras

1.1. Fases de realización del TFM 5
1.2. Diagrama de Gantt 6
1.3. Distribución de costes del proyecto. 8

3.1. Paradigma de Mitchell. (Mitchell, 2018) 19
3.2. Enfoque tradicional *Machine Learning* (Géron, 2019) 20
3.3. (a) La altura y el peso de algunas personas. (b) Un posible clustering utilizando K = 2 clusters. 21
3.4. Ejemplo de aprendizaje supervisado (Géron, 2019) 22
3.5. Ejemplo de *K-NN* (Soni, 2019) 24
3.6. Ejemplo de un *Decision Tree* (Fernández, 2020) 25
3.7. Ejemplo de *Random Forest* (DIDA.do, 2023) 26
3.8. Ejemplo de *CatBoost* (Kushwaha, 2021) 27
3.9. Ejemplo de *XGBoost* (Omarzai, 2021) 28
3.10. Ejemplo de LightGBM (Mandot, 2021) 29
3.11. Ejemplo de un *MLP* (DataCamp, 2023) 30
3.12. Ejemplo de hiperplano de SVM (Gupta, 2023) 31
3.13. Ejemplo de una red neuronal (Amazon Web Services, 2024) 32
3.14. Funciones de activación (ReLU, Sigmoide, Softmax) 33
3.15. Ejemplo de *dropout* que elimina 3 neuronas (Géron, 2019) 33
3.16. Ejemplo de muestras con SMOTE 34
3.17. Ejemplo de balanceo con *Random UnderSampling* 34
3.18. Ejemplo de balanceo con *Edited Nearest Neighbors* 35
3.19. Ejemplo de balanceo con Cluster Centroids 35
3.20. Precision y *Recall* de manera gráfica (Encord, 2023) 36
3.21. Ejemplo de curva ROC (Géron, 2019) 37
3.22. Ejemplo de matriz de confusión (Géron, 2019) 38
3.23. *Cross-Validation* con 5-*fold* (learn Developers, 2024e) 39
3.24. Comportamiento de *Overfitting* y *Underfitting* (Patel, 2022) 41

5.1. Página inicial de virustotal 56
5.2. Flujo a la hora de crear el dataset 61
5.3. Categoríasobtenidas a partir de VirusTotal 67
5.4. Extensiones de archivos más comunes 68
5.5. Comparativa extensiones existentes con la clasificación de *malware* . . . 69
5.6. Disposición malicious *score* con categoría de *malware* 70
5.7. Relación entre diversas características binarias y la Categoría VT. Cada gráfico ilustra la distribución del número de muestras según la presencia o ausencia de la característica elegida, clasificada por categoría de riesgo. 72
5.8. Mapa correlación variables 74
5.9. Diagrama de flujo compacto del preprocesamiento y configuración para `CatBoost`. 87

5.10. Diagrama de flujo del preprocesamiento para `DecisionTree` (corregido origen de flechas manuales). 89
5.11. Diagrama de flujo para `KNN` con balanceo manual (corregido y simplificado). 91
5.12. Diagrama de flujo del proceso de preprocesamiento y configuración para el modelo `LightGBM`. 93
5.13. Diagrama de flujo para `MLPClassifier` (corregido). 95
5.14. Diagrama de flujo del preprocesamiento y configuración para `RandomForestClassifier` con opciones de balanceo. 97
5.15. Diagrama de flujo del preprocesamiento y configuración para `SVM` con balanceo manual (corregido). 99
5.16. Diagrama de flujo del preprocesamiento y configuración para `XGBoost` con opciones de balanceo. 101
5.17. Diagrama de flujo para el entrenamiento de Redes Neuronales. 103

6.1. Matriz de confusión - *CatBoost* 107
6.2. Matriz de confusión - *DecisionTree* (Sin balanceo) 109
6.3. Matriz de confusión - *DecisionTree* (SMOTE) 111
6.4. Matriz de confusión - *DecisionTree* (*Undersampling*) 112
6.5. Matriz de confusión - *DecisionTree* (ENN) 114
6.6. Matriz de confusión - *DecisionTree* (CC) 115
6.7. Matriz de confusión - KNN (SMOTE) 117
6.8. F1*Macro* en CV - KNN 117
6.9. F1*Macro* en CV - KNN (*Undersampling*) 121
6.10. Matriz de confusión - KNN (*Undersampling*) 121
6.11. F1*Macro* en CV - KNN (CC) 121
6.12. Matriz de confusión - KNN (CC) 122
6.13. F1*Macro* en CV - KNN (ENN) 122
6.14. Matriz de confusión - KNN (ENN) 123
6.15. Matriz de confusión - *LightGBM* 125
6.16. Capas ocultas MLP - SMOTE 127
6.17. Matriz de confusión - MLP (SMOTE) 127
6.18. Evolución neuronas MLP (*Undersampling*) 132
6.19. Matriz de confusión - MLP (*Undersampling*) 132
6.20. Evolución neuronas MLP (ENN) 133
6.21. Matriz de confusión - MLP (ENN) 133
6.22. Evolución neuronas MLP (CC) 133
6.23. Matriz de confusión - MLP (CC) 134
6.24. Matriz de confusión - *RandomForest* 136
6.25. Matriz de confusión - *RandomForest* (SMOTE) 138
6.26. Matriz de confusión - *RandomForest* (*Undersampling*) 140
6.27. Matriz de confusión - *RandomForest* (ENN) 140
6.28. Matriz de confusión - *RandomForest* (CC) 140
6.29. Matriz de confusión - SVM (SMOTE) 142
6.30. Matriz de confusión - SVM (*Undersampling*) 146
6.31. Matriz de confusión - SVM (ENN) 146
6.32. Matriz de confusión - SVM (CC) 147

6.33. Matriz de confusión - *XGBoost* 149
6.34. Matriz de confusión - *XGBoost* (SMOTE) 151
6.35. Matriz de confusión - *XGBoost* (*Undersampling*) 153
6.36. Matriz de confusión - *XGBoost* (ENN) 153
6.37. Matriz de confusión - *XGBoost* (CC) 154
6.38. Matriz de confusión (SMOTE 32-64) 156
6.39. Matriz de confusión (SMOTE 64-128) 156
6.40. Gráficos LossAccuracy - DNN SMOTE(32-64) 157
6.41. Gráficos LossAccuracy - DNN SMOTE (64-128) 157
6.42. Matriz de confusión - DNN (*Undersampling* 32-64) 161
6.43. Matriz de confusión - DNN (*Undersampling* 64-128) 161
6.44. Matriz de confusión - DNN (ENN 32-64 162
6.45. Matriz de confusión - DNN (ENN 64-128) 162
6.46. Matriz de confusión - DNN (CC 32-64) 162
6.47. Matriz de confusión - DNN (CC 64-128) 163
6.48. Gráficos LossAccuracy - DNN *Undersampling* (32-64) 163
6.49. Gráficos LossAccuracy - DNN *Undersampling* (64-128) 164
6.50. Gráficos LossAccuracy - DNN ENN (32-64) 164
6.51. Gráficos LossAccuracy - DNN ENN (64-128) 164
6.52. Gráficos LossAccuracy - DNN CC (32-64) 165
6.53. Gráficos LossAccuracy - DNN CC (64-128) 165

Índice de tablas

1.1. Coste de personal por fases (25 €/h). 7
1.2. Costes indirectos. 7

5.1. Ejemplos de muestras que se encuentran en el fichero descargado. . . . 55
5.2. Información general de archivos analizados con VirusTotal 60
5.3. Estadísticas de análisis y comportamiento en sandbox 60
5.4. Información de red asociada al archivo (IPs y puertos) 60
5.5. Resumen de columnas añadidas y eliminadas 66
5.6. Hiperparámetros y valores en `CatBoostClassifier` 88
5.7. Hiperparámetros y valores en `DecisionTreeClassifier` sin balanceo . 88
5.8. Hiperparámetros y valores en `DecisionTreeClassifier` con balanceo . 88
5.9. Hiperparámetros y valores en `KNeighborsClassifier` 90
5.10. Hiperparámetros y valores en `LGBMClassifier` 92
5.11. Hiperparámetros y valores en `MLPClassifier` 94
5.12. Hiperparámetros y valores en `RandomForestClassifier` balanceo automático 96
5.13. Hiperparámetros y valores en `RandomForestClassifier` 96
5.14. Hiperparámetros y valores en `SVC` 98
5.15. Hiperparámetros y valores en `XGBClassifier` 100
5.16. Hiperparámetros y configuración para el modelo de Red Neuronal. . . . 102

6.1. Resultados promedio de *cross-validation* (5 *folds*) 105
6.2. Reporte de clasificación en el conjunto de entrenamiento (CatBoost) . . 105
6.3. Reporte de clasificación en el conjunto de prueba (*CatBoost*) 106
6.4. Métricas adicionales del conjunto de prueba y tiempo de ejecución (CatBoost) 106
6.5. Resultados promedio de *cross-validation* 108
6.6. Reporte de clasificación en el conjunto de entrenamiento - *DecisionTree* sin *SMOTE* sin escalado. 108
6.7. Reporte de clasificación en el conjunto de prueba - *DecisionTree* sin *SMOTE* sin escalado. 108
6.8. Métricas adicionales del conjunto de prueba y tiempo de ejecución - *DecisionTree* sin *SMOTE* sin escalado. 108
6.9. Resultados promedio de *cross-validation* (5 *folds*) - *DecisionTree* con SMOTE. 110
6.10. Reporte de clasificación en el conjunto de entrenamiento - *DecisionTree* con SMOTE y sin escalado. 110
6.11. Reporte de clasificación en el conjunto de prueba - *DecisionTree* con SMOTE y sin escalado. 110
6.12. Métricas adicionales del conjunto de prueba y tiempo de ejecución - *DecisionTree* con SMOTE y sin escalado. 110
6.13. Resultados promedio de *cross-validation* (5 *Folds*) - *DecisionTree* con *Undersampling*. 111

6.14. Reporte de clasificación en el conjunto de entrenamiento - *DecisionTree* con *Undersampling*. 112
6.15. Reporte de clasificación en el conjunto de prueba - *DecisionTree* con *Undersampling*. 112
6.16. Métricas adicionales (prueba) y tiempo - *DecisionTree* con *Undersampling*. 112
6.17. Resultados promedio de *cross-validation* (5 *Folds*) - *DecisionTree* con ENN. 113
6.18. Reporte de clasificación en el conjunto de entrenamiento - *DecisionTree* con ENN. 113
6.19. Reporte de clasificación en el conjunto de prueba - *DecisionTree* con ENN. 113
6.20. Métricas adicionales (prueba) y tiempo - *DecisionTree* con ENN. 113
6.21. Resultados promedio de *cross-validation* (5 *Folds*) - *DecisionTree* con Cluster Centroids. 114
6.22. Reporte de clasificación en el conjunto de entrenamiento - *DecisionTree* con Cluster Centroids. 114
6.23. Reporte de clasificación en el conjunto de prueba - *DecisionTree* con Cluster Centroids. 115
6.24. Métricas adicionales (prueba) y tiempo - *DecisionTree* con Cluster Centroids. 115
6.25. Optimización del Hiperparámetro k para KNN mediante *cross-validation*. 116
6.26. Reporte de clasificación en el conjunto de entrenamiento - KNN con SMOTE (k=4). 116
6.27. Reporte de clasificación en el conjunto de prueba - KNN con SMOTE (k=4). 116
6.28. Métricas adicionales del conjunto de prueba y tiempo de ejecución - KNN con SMOTE (k=4). 117
6.29. Optimización de k para KNN con *Undersampling*. 118
6.30. Reporte de clasificación (entrenamiento) - KNN con *Undersampling* (k=6). 118
6.31. Reporte de clasificación (prueba) - KNN con *Undersampling* (k=6). . . 119
6.32. Métricas adicionales (prueba) y tiempo - KNN con *Undersampling* (k=6). 119
6.33. Optimización de k para KNN con Cluster Centroids. 119
6.34. Reporte de clasificación (entrenamiento) - KNN con Cluster Centroids (k=8). 119
6.35. Reporte de clasificación (prueba) - KNN con Cluster Centroids (k=8). . 119
6.36. Métricas adicionales (prueba) y tiempo - KNN con Cluster Centroids (k=8). 120
6.37. Optimización de k para KNN con ENN. 120
6.38. Reporte de clasificación (entrenamiento) - KNN con ENN (k=1). . . . 120
6.39. Reporte de clasificación (prueba) - KNN con ENN (k=1). 120
6.40. Métricas adicionales (prueba) y tiempo - KNN con ENN (k=1). 120
6.41. Resultados promedio de *cross-validation* (5 *Folds*) - *LightGBM* sin balanceo. 124
6.42. Reporte de clasificación en el conjunto de entrenamiento - *LightGBM* sin balanceo. 124
6.43. Reporte de clasificación en el conjunto de prueba - *LightGBM* sin balanceo. 124
6.44. Métricas adicionales del conjunto de prueba y tiempo de ejecución - *LightGBM* sin balanceo. 124

6.45. Configuración del modelo red neuronal (MLP) y preprocesamiento . . . 125
6.46. Reporte de clasificación en el conjunto de entrenamiento (MLP con SMOTE) . . . 126
6.47. Reporte de clasificación en el conjunto de prueba (MLP con SMOTE) . 126
6.48. Métricas adicionales del conjunto de prueba y tiempo de ejecución (MLP con SMOTE) . . . 126
6.49. Configuración del modelo red neuronal (MLP) y preprocesamiento (*Undersampling*) . . . 128
6.50. Reporte de clasificación en el conjunto de entrenamiento (MLP con *Undersampling*) . . . 128
6.51. Reporte de clasificación en el conjunto de prueba (MLP con *Undersampling*) . . . 129
6.52. Métricas adicionales del conjunto de prueba y tiempo de ejecución (MLP con *Undersampling*) . . . 129
6.53. Configuración del modelo red neuronal (MLP) y preprocesamiento (ENN) 129
6.54. Reporte de clasificación en el conjunto de entrenamiento (MLP con ENN) 129
6.55. Reporte de clasificación en el conjunto de prueba (MLP con ENN) . . . 130
6.56. Métricas adicionales del conjunto de prueba y tiempo de ejecución (MLP con ENN) . . . 130
6.57. Configuración del modelo red neuronal (MLP) y preprocesamiento (Cluster Centroids) . . . 130
6.58. Reporte de clasificación en el conjunto de entrenamiento (MLP con Cluster Centroids) . . . 130
6.59. Reporte de clasificación en el conjunto de prueba (MLP con Cluster Centroids) . . . 131
6.60. Métricas adicionales del conjunto de prueba y tiempo de ejecución (MLP con Cluster Centroids) . . . 131
6.61. Resultados promedio de *cross-validation* (*RandomForest*) . . . 134
6.62. Reporte de clasificación en el conjunto de entrenamiento (*RandomForest*) 135
6.63. Reporte de clasificación en el conjunto de prueba (*RandomForest*) . . . 135
6.64. Métricas adicionales del conjunto de prueba y tiempo de ejecución (*RandomForest*) . . . 135
6.65. Resultados promedio de *cross-validation* (*RandomForest* con SMOTE) 136
6.66. Reporte de clasificación en el conjunto de entrenamiento (*RandomForest* con SMOTE) . . . 137
6.67. Reporte de clasificación en el conjunto de prueba (*RandomForest* con SMOTE) . . . 137
6.68. Métricas adicionales del conjunto de prueba y tiempo de ejecución (*RandomForest* con SMOTE) . . . 137
6.69. Resultados promedio de *Cross-Validation* (*RandomForest* con Submuestreo) . . . 138
6.70. Resumen de *RandomForest* con *Undersampling* General . . . 138
6.71. Resumen de *RandomForest* con ENN . . . 139
6.72. Resumen de *RandomForest* con Cluster Centroids . . . 139
6.73. Configuración del modelo SVM,resultados de *Cross-Validation* y preprocesamiento . . . 141

6.74. Reporte de clasificación en el conjunto de entrenamiento (SVM con SMOTE, C=10) 141
6.75. Reporte de clasificación en el conjunto de prueba (SVM con SMOTE, C=10) 142
6.76. Métricas adicionales del conjunto de prueba y tiempo de ejecución (SVM con SMOTE, C=10) 142
6.77. Configuración del modelo SVM y resultados de CV (*Undersampling*) . 143
6.78. Reporte de clasificación en entrenamiento (SVM con *Undersampling*, C=10) 143
6.79. Reporte de clasificación en prueba (SVM con *Undersampling*, C=10) . 144
6.80. Métricas adicionales y tiempo (SVM con *Undersampling*, C=10) 144
6.81. Configuración del modelo SVM y resultados de CV (ENN) 144
6.82. Reporte de clasificación en entrenamiento (SVM con ENN, C=10) . . . 144
6.83. Reporte de clasificación en prueba (SVM con ENN, C=10) 144
6.84. Métricas adicionales y tiempo (SVM con ENN, C=10) 145
6.85. Configuración del modelo SVM y resultados de CV (Cluster Centroids) 145
6.86. Reporte de clasificación en entrenamiento (SVM con Cluster Centroids, C=10) 145
6.87. Reporte de clasificación en prueba (SVM con Cluster Centroids, C=10) 145
6.88. Métricas adicionales y tiempo (SVM con Cluster Centroids, C=10) . . 145
6.89. Resultados promedio de *Cross-Validation* (*XGBoost*) 147
6.90. Reporte de clasificación en el conjunto de entrenamiento (*XGBoost*) . . 148
6.91. Reporte de clasificación en el conjunto de prueba (*XGBoost*) 148
6.92. Métricas adicionales del conjunto de prueba y tiempo de ejecución (*XGBoost*) 148
6.93. Resultados promedio de *cross-validation* (*XGBoost* con SMOTE) . . . 149
6.94. Reporte de clasificación en el conjunto de entrenamiento (*XGBoost* con SMOTE) 150
6.95. Reporte de clasificación en el conjunto de prueba (*XGBoost* con SMOTE) 150
6.96. Métricas adicionales del conjunto de prueba y tiempo de ejecución (*XGBoost* con SMOTE) 150
6.97. Resultados promedio de *cross-validation* (*XGBoost* con Submuestreo) . 151
6.98. Resumen de *XGBoost* con *Undersampling* General 152
6.99. Resumen de *XGBoost* con ENN 152
6.100. Resumen de *XGBoost* con Cluster Centroids 152
6.101. Resumen de rendimiento del modelo DNN con SMOTE (32-64) 155
6.102. Reporte de clasificación en prueba (DNN con SMOTE 32-64) 155
6.103. Tiempo de ejecución (DNN con SMOTE 32-64) 155
6.104. Resumen de rendimiento del modelo DNN con SMOTE (64-128) 155
6.105. Reporte de clasificación en prueba (DNN con SMOTE 64-128) 155
6.106. Tiempo de ejecución (DNN con SMOTE 64-128) 156
6.107. Resumen del modelo DNN con *Undersampling* (32-64) 158
6.108. Reporte de clasificación en prueba (DNN con *Undersampling* 32-64) . . 158
6.109. Resumen del modelo DNN con *Undersampling* (64-128) 158
6.110. Reporte de clasificación en prueba (DNN con *Undersampling* 64-128) . 158
6.111. Resumen del modelo DNN con ENN (32-64) 158
6.112. Reporte de clasificación en prueba (DNN con ENN 32-64) 159

6.113Resumen del modelo DNN con ENN (64-128) 159
6.114Reporte de clasificación en prueba (DNN con ENN 64-128) 159
6.115Resumen del modelo DNN con Cluster Centroids (32-64) 159
6.116Reporte de clasificación en prueba (DNN con Cluster Centroids 32-64) 159
6.117Resumen del modelo DNN con Cluster Centroids (64-128) 160
6.118Reporte de clasificación en prueba (DNN con Cluster Centroids 64-128) 160

Parte I

Prolegómeno

1. Introducción

1.1. Motivación

Desde los últimos años y hasta la actualidad, se ha podido ver cómo diferentes sectores sufren distintos ataques con el objetivo de obtener información o provocar daños a dichas instituciones. El 62 % de las organizaciones a nivel mundial al menos han sufrido un ciberataque exitoso (Group, 2024). Todo esto se encuentra relacionado con la ciberseguridad, que es la encargada de proteger la información digital, los dispositivos y los activos (Microsoft, 2025). El impacto de estos ataques en las distintas organizaciones es muy profundo y dejando unas secuencias muy duras y costosas. Se pueden realizar ataques tan simples como introducir un virus en pendrive para obtener información del usuario o bloquear diferentes archivos del usuario, hasta ejecutar ataques que puedan incluso paralizar toda una gestión de un aeropuerto. Algunos de los ataques más conocidos pueden ser de ingeniería social, *ransomware* o denegación de servicio.

Para que los daños que estos ataques producen sean mínimos, es crucial su temprana observación. Para ello, se llevan a cabo diferentes operaciones para que esto no suceda. Algunos ejemplos de ellos puede ser la utilización de antivirus, sistemas de detección y prevención de intrusos (*IDS/IPS*) o diferentes sistemas que monitorizan el comportamiento de los archivos. Cuando alguna de estas herramientas detecte un ataque, se lo comunica al usuario con el objetivo de que pueda realizar acciones para mitigar estas intrusiones. Estos materiales pueden consumir muchos recursos a la hora de analizar los distintos indicadores e incluso pueden resultar ambiguos, llegando a no detectar nuevos ataques que surjan con las nuevas tecnologías.

En la actualidad, muchos dispositivos utilizan sistemas de autenticación basados en reconocimiento facial o de huellas dactilares, los cuales dependen de la Inteligencia Artificial (IA) para garantizar un acceso seguro. También nos proporciona diferentes beneficios como la autenticación sin necesidad de contraseñas, más difíciles de adivinar, o utilización de tarjetas físicas. Otra función principal de la IA en la seguridad puede ser la detección de los diferentes correos marcados como spam o *phising*.(IABAC, 2024).

El uso de *Machine Learning (ML)* (aprendizaje automático) se ha postulado como una de las mejores opciones a la hora de mejorar la detección de *malware* en los diferentes dispositivos. El aprendizaje automático dispone de la capacidad de analizar grandes cantidades de datos, obtenidos a partir del análisis de los diferentes ataques, y aprender diferentes patrones para poder identificar con éxito un comportamiento sospechoso. Se obtendrá información como el tamaño del fichero que contiene el virus, el nombre del fichero o la dirección IP de destino a donde se envía el contenido. Toda esta información nos servirá de una gran utilidad debido a que cuanta más información se disponga mejores resultados ofrece el *machine learning*.

Además del impacto técnico, los ciberataques representan una amenaza económica y social de gran envergadura, generando pérdidas millonarias cada año y afectando a

la confianza de los usuarios en servicios digitales. A nivel normativo, la existencia de marcos regulatorios como el GDPR en Europa y las leyes de protección de datos en otros países obliga a las organizaciones a reforzar su ciberseguridad. Las soluciones tradicionales basadas en firmas resultan insuficientes, ya que los atacantes generan constantemente nuevas variantes de *malware*. En este contexto, el presente trabajo se plantea como una respuesta directa a estas limitaciones, explorando el potencial de distintos algoritmos de *Machine Learning* para la detección y clasificación de *malware* de forma más precisa y robusta.

La principal novedad de este trabajo es la creación de un **dataset a partir de fuentes de datos diversas**. Para su construcción se ha utilizado el repositorio público *MalwareBazaar*, que ofrece muestras de *malware* clasificadas y compartidas con fines de investigación, y se ha completado con información procedente de *VirusTotal*. Esta última plataforma combina los resultados de múltiples motores antivirus y herramientas de análisis, además de incorporar las contribuciones de la propia comunidad de usuarios, lo que permite disponer de indicadores adicionales como *hashes*, familias de *malware* y metadatos de comportamiento.

La fusión de estas dos fuentes ha permitido generar un conjunto de datos mucho más completo y representativo que si se hubiera recurrido únicamente a una de ellas. En particular, *MalwareBazaar* aporta la base estructurada de muestras, mientras que *VirusTotal* añade un nivel de validación y detalle que resulta fundamental para entrenar modelos de *Machine Learning* con mayor fiabilidad y capacidad de generalización. De esta forma, el trabajo no solo se apoya en un único repositorio, sino que integra distintas perspectivas y criterios de clasificación, reflejando con mayor realismo la diversidad de escenarios que se presentan en la detección de *malware* en entornos reales.

1.2. Alcance y objetivos

Esta sección define los límites del proyecto, los elementos organizativos a los que afectaría en un entorno real y los objetivos clave que se persiguen.

1.2.1. Alcance del proyecto

El desarrollo de este sistema se concibe como un prototipo funcional. En un entorno organizativo, afectaría principalmente al departamento de seguridad informática (p. ej., un *SOC* - Centro de Operaciones de Seguridad), proporcionando una herramienta de análisis automatizado para los analistas de *malware* y los equipos de respuesta a incidentes.

El alcance del proyecto abarca las siguientes fases fundamentales:

- Investigación y estado del arte: una revisión de las técnicas actuales para la detección de *malware*, con especial atención en los enfoques basados en *Machine Learning*.

- Recolección y consolidación de datos: creación de un conjunto de datos (dataset) completo y diverso mediante la recopilación y fusión de muestras de *malware* y software benigno de fuentes públicas reconocidas como *VirusTotal*, *MalwareBazaar* y *The Zoo*.
- Preprocesamiento y extracción de características: implementación de un módulo para el procesamiento de los datos, extrayendo características relevantes tanto estáticas (tamaño, hashes, secciones del fichero) como dinámicas (comportamiento en un entorno controlado o sandbox).
- Implementación y entrenamiento de modelos: desarrollo y entrenamiento de una selección de algoritmos de clasificación supervisada, incluyendo *Random Forest*, *Support Vector Machines (SVM)*, Redes *Neuronales Profundas (DNN)* y modelos de *Gradient Boosting Machines (GBM)*.
- Análisis y evaluación de resultados: un análisis comparativo del rendimiento de los modelos implementados, utilizando métricas estándar para evaluar su eficacia en la detección y categorización de *malware*.

1.2.2. Objetivos

Los objetivos que se esperan alcanzar con la finalización y puesta en producción teórica de este sistema son:

Objetivo Principal

- **Diseñar y desarrollar un sistema automatizado**, capaz de procesar ficheros, extraer características relevantes y clasificar las muestras de software no solo como benignas o maliciosas, sino también categorizar el *malware* detectado en diferentes familias (p. ej., *ransomware*, troyano, *spyware*).

Objetivos Secundarios

- **Desarrollar módulos de extracción de características** que permitan obtener información relevante de los ficheros para su análisis posterior por los modelos de Machine Learning.
- **Implementar y evaluar diversos modelos de *Machine Learning*** para la clasificación de *malware* en diferentes categorías y familias.
- **Evaluar de forma rigurosa el rendimiento** de los distintos modelos de Machine Learning implementados, utilizando métricas clave como la precisión, el *recall*, el *F1-score*, la matriz de confusión y el área bajo la curva ROC (AUC-ROC).
- **Realizar un análisis comparativo profundo** entre los diferentes algoritmos y técnicas de preprocesamiento.

1.3. Planificación del proyecto

A continuación, se exponen las fases del proyecto y el tiempo invertido en cada una de ellas, presentados en forma de cronograma y diagrama de Gantt.

Fase	Tarea	Horas
Estudio Inicial	Investigación inicial de la problemática y motivación	10
	Investigación del contexto actual y antecedentes	10
Fundamentos teóricos	Estudio de *Machine Learning* y *Deep Learning* (tipos, conceptos)	20
	Investigación sobre métodos de balanceo de clases de algoritmos de Machine Learning	8
	Selección y estudio de herramientas y tecnologías (Python, librerías)	10
Preparación de los datos	Búsqueda de fuentes de datos de malware	12
	Descarga de *datasets* iniciales	8
	Inclusión de datos externos	16
	Preprocesamiento de datos	20
Desarrollo de modelos	Desarrollo de algoritmos	24
Análisis de resultados	Análisis de los resultados	30
	Redacción de resultados	14
Documentación	Realizar documentación del proyecto	26
Horas totales		208

Figura 1.1: Fases de realización del TFM

Tareas	NOV				DIC				ENE				FEB				MAR				ABR				MAY				JUN				JUL			
2024-25	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
Estudio inicial																																				
Fundamentos teóricos																																				
Preparación de los datos																																				
Desarrollo de modelos																																				
Análisis de resultados																																				
Documentación																																				

Figura 1.2: Diagrama de Gantt

1.4. Estimación de costes

Para el cálculo de costes se han tenido en cuenta los siguientes aspectos:

- **Coste de personal:** se ha considerado un valor de referencia de 25 €/hora, correspondiente a un perfil de desarrollador junior. El total de horas invertidas en el proyecto asciende a 208, distribuidas en las distintas fases de trabajo.
- **Costes indirectos:** se incluyen aquellos asociados al uso de recursos materiales y servicios, en particular electricidad, conexión a internet y el uso del portátil. Estos costes se han prorrateado según el tiempo real de dedicación al proyecto.

El coste de personal desglosado por fases se muestra en la Tabla 1.1.

Fase	**Horas**	**Coste unitario (€/h)**	**Coste total (€)**
Estudio inicial	20	25,00	500,00
Fundamentos teóricos	38	25,00	950,00
Preparación de los datos	56	25,00	1400,00
Desarrollo de modelos	24	25,00	600,00
Análisis de resultados	44	25,00	1100,00
Documentación	26	25,00	650,00
Total personal	**208**		**5.200,00**

Tabla 1.1: *Coste de personal por fases (25 €/h).*

En cuanto a los costes indirectos, se han considerado los siguientes cálculos:

- **Electricidad:** el portátil utilizado tiene un consumo medio de 65 W. Para un total de 208 horas de trabajo, esto supone un consumo aproximado de 13,52 kWh. Considerando un precio medio de 0,25 €/kWh, el coste total asciende a 3,38 €.
- **Conexión a internet:** se ha considerado una tarifa mensual estándar de 30 €/mes. Dado que el proyecto se desarrolló a lo largo de unos dos meses, el coste estimado es de 60,00 €.
- **Portátil (hardware):** con un coste de adquisición de 1.200 €, una vida útil estimada de 3 años y un uso medio de 5 horas diarias (5.475 horas en total). Prorrateado a las 208 horas del proyecto, el coste es de 45,60 €.

Estos costes indirectos se resumen en la Tabla 1.2.

Concepto	**Cantidad**	**Coste unitario**	**Coste total (€)**
Electricidad	13,52 kWh	0,25	3,38
Conexión a internet	2 meses	30,00	60,00
Portátil (prorrateado)	208 h	0,22	45,60
Total costes indirectos			**108,98**

Tabla 1.2: *Costes indirectos.*

Sumando ambas partidas, el coste total estimado del proyecto asciende a **5.308,98 €**, de los cuales aproximadamente el **98 %** corresponde al trabajo de personal. La Figura 1.3 muestra de forma gráfica la distribución de costes.

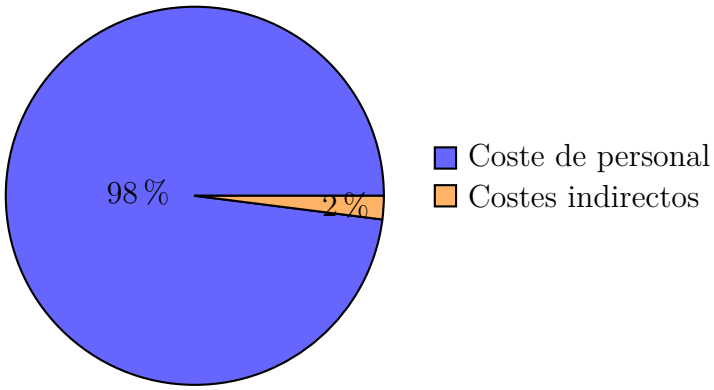


Figura 1.3: Distribución de costes del proyecto.

Como puede observarse, la partida de personal constituye el grueso del presupuesto, mientras que los costes indirectos representan un porcentaje muy reducido. Esto es consistente con la naturaleza del trabajo, ya que se trata de un proyecto de investigación cuyo principal recurso es el tiempo y conocimiento especializado del investigador.

En un escenario real con un equipo de varias personas o con necesidades de infraestructura más exigentes (por ejemplo, servidores dedicados, licencias de software o servicios en la nube), el coste total podría multiplicarse, siendo los costes de infraestructura una partida relevante. En este caso, al haberse utilizado *software* de código abierto y recursos propios, el presupuesto se ha mantenido bajo en términos de costes indirectos.

1.5. Glosario de términos

AUC-ROC (*Area Under the Receiver Operating Characteristic Curve*) Métrica que evalúa el rendimiento de un modelo de clasificación en todos los umbrales de clasificación. Un valor cercano a 1 indica un modelo con excelente capacidad de discriminación.

DL (*Deep Learning*) Subcampo del *Machine Learning* basado en redes neuronales con múltiples capas (profundas), capaz de aprender representaciones complejas de los datos.

DNN (*Deep Neural Network*) Red Neuronal Profunda. Un tipo de arquitectura de Deep Learning utilizada para tareas de clasificación y regresión.

F1-Score Métrica que combina la precisión (*precision*) y la exhaustividad (*recall*) en un solo valor, calculado como la media armónica de ambos. Es especialmente útil en casos de desbalance de clases.

GBM (*Gradient Boosting Machines*) Familia de algoritmos de ensamblado que construyen modelos de forma secuencial, donde cada nuevo modelo corrige los

errores del anterior. Incluye implementaciones como *XGBoost*, *LightGBM* y *CatBoost*.

Hash Valor alfanumérico de longitud fija que representa e identifica de forma única un conjunto de datos (p. ej., un fichero). Algoritmos comunes son *MD5*, *SHA-1* y *SHA-256*.

Malware Abreviatura de *malicious software* (software malicioso). Término genérico para cualquier software diseñado para causar daño, interrumpir el funcionamiento o robar información de un sistema informático.

MCC (*Matthews Correlation Coefficient*) Coeficiente de Correlación de Matthews. Una métrica de calidad para la clasificación binaria y multiclase, considerada muy robusta, especialmente con clases desbalanceadas.

ML (*Machine Learning*) Campo de la inteligencia artificial que se centra en el desarrollo de algoritmos que permiten a los ordenadores aprender a partir de datos sin ser programados explícitamente.

SVM (*Support Vector Machine*) Máquina de Soporte Vectorial. Un algoritmo de aprendizaje supervisado que busca encontrar un hiperplano que separe de la mejor manera posible las clases en un espacio de características.

VirusTotal Servicio en línea que analiza archivos y URLs sospechosas utilizando múltiples motores antivirus y herramientas de análisis, proporcionando una evaluación agregada sobre su seguridad.

Zero-Day Amenaza o vulnerabilidad de seguridad que es desconocida para el desarrollador del software o para el público general, y para la cual no existe todavía un parche o firma de detección.

1.6. Organización del documento

La presente memoria se estructura en varios capítulos para guiar al lector de manera lógica a través del desarrollo del proyecto.

- **Capítulo 1 - Introducción:** el presente capítulo, donde se exponen la motivación, el alcance y los objetivos del proyecto, además de un glosario de términos y la organización de este documento.
- **Capítulo 2 - Antecedentes:** incluye la revisión del *estado del arte*, donde se analizan investigaciones previas relacionadas con la detección de *malware* y las aproximaciones más utilizadas en este campo.
- **Capítulo 3 - Fundamentos teóricos:** se introducen los conceptos esenciales de *machine learning*, métodos de balanceo, métricas de evaluación, y una descripción detallada del *malware* y sus características.
- **Capítulo 4 - Herramientas y tecnologías utilizadas:** se presentan las herramientas de software y librerías empleadas en el desarrollo del trabajo (Python, NumPy, Pandas, Matplotlib, Scikit-learn, entre otras).

- **Capítulo 5 - Metodología:** se describe en detalle la metodología seguida, incluyendo la obtención y preprocesamiento de datos, la ingeniería de características, las fases de entrenamiento, y la justificación de los algoritmos seleccionados.
- **Capítulo 6 - Resultados:** se presentan y analizan los resultados obtenidos en los experimentos, comparando el rendimiento de los diferentes modelos de clasificación en términos de métricas de evaluación.
- **Capítulo 7 - Conclusiones:** se resumen las principales conclusiones extraídas del trabajo, se evalúa el cumplimiento de los objetivos y se proponen posibles líneas de investigación y desarrollo futuro.

Junto a esta memoria, se entrega en soporte informático el código fuente del proyecto, consistente en una serie de *notebooks* de Jupyter que documentan cada paso del proceso, desde la carga de datos hasta la evaluación final de los modelos.

2. Antecedentes

2.1. Estado del arte

Hoy en día, el *machine learning* tiene un valor significativo y se utiliza en muchos ámbitos que permite conseguir avance y mejoras. Se puede utilizar en transacciones financieras para detectar el fraude o para predecir si las acciones de un banco sube o baja. Asimismo, se usa en el ámbito de la sanidad para la detección precoz del cáncer o para analizar mamografías. Otro entorno en el que se aplica puede ser en el transporte. *Google Maps* utiliza *machine learning* para determinar la ruta más rápida o estimar los tiempos de llegada (IBM, 2023).

En el ámbito de la ciberseguridad, el aprendizaje automático ha cobrado un protagonismo considerable en los últimos años. Diferentes trabajos proponen enfoques variados para detectar y clasificar el *malware*, adaptándose a los entornos específicos donde aparece: desde el *internet of things* (IoT), el tráfico de red, hasta los dispositivos móviles y modelos avanzados con técnicas modernas de aprendizaje profundo. A continuación, se presenta una revisión organizada de dichos enfoques.

Machine learning en IoT para la detección de *malware*

Este estudio habla sobre los problemas y riesgos en el *internet of things* (IoT), centrándose en la detección de *malware*. Se menciona los diferentes problemas y la poca seguridad que estos dispositivos disponen. Algunos de estos ataques que sufren pueden ser *spyware*, *ransomware* y troyanos. Opta por utilizar el *machine learning* para detectar estos ataques. Algunos algoritmos que emplea son el *Support Vector Machine* (SVM), *Decision Tree* (DT), *Logistic Regression* (LR) y *Random Forest* (RF). El *dataset* utilizado se llama *CIC-MalMem*. Los resultados obtenidos en cuanto a precisión nos muestra un 98,45 % en el algoritmo *Random Forest*. Para *Logistic Regression*, *Decision Tree* y *Support Vector Machine* se ha logrado una precisión del 96,83 %, 97,67 % y 93 %, respectivamente (Sharma et al., 2024).

Estos resultados muestran que, aunque los dispositivos IoT son muy vulnerables, el *machine learning* es capaz de mejorar significativamente la detección de amenazas en este entorno. No obstante, cuando el tráfico se desplaza fuera de los dispositivos hacia la red, surgen otros retos específicos.

Análisis de tráfico de red y *malware*

En el artículo expone los diferentes problemas a lo que se enfrenta el tráfico una vez es enviado a través de la red. Los autores mencionan cómo los métodos convencionales no funcionan bien y cómo la ciberseguridad debe adaptarse a las nuevas tecnologías

y los nuevos avances que surjan. Informa cómo el *malware* puede infectar ordenadores, borrar datos sensibles y aumentar el riesgo de los ataques sofisticados. Explica la metodología llevada a cabo y las distintas etapas por la que el *machine learning* pasa para poder detectar el *malware*: desde la recogida de información, preprocesamiento de datos, algoritmos y resultados. Utiliza algoritmos comunes como son el *Random Forest* (RF), *k-nearest neighbors* (KNN) y *Adaboost*. Muestra distintas gráficas relacionando el número de falsos positivos y verdaderos positivos. También expone diferentes matrices de confusión de cada algoritmo utilizado. Los tres algoritmos utilizados superan en un 95 % la precisión a la hora de detectar si es *malware* o no es *malware*. El algoritmo *Adaboost* es el que mayor precisión tiene con un 99,95 % (Omopintemi et al., 2023).

Así, mientras que el caso de IoT se centra en la protección de dispositivos individuales, en el tráfico de red la prioridad está en reducir los falsos positivos y garantizar la detección en tiempo real. Otro ámbito clave donde se aplica el aprendizaje automático es en los dispositivos móviles, especialmente los basados en *Android*.

Detección de *malware* en *Android*: FEdroid

Los sistemas operativos *Android* son conocidos a nivel mundial por su flexibilidad y su naturalidad de fuentes abiertas en las que cualquier cosa puede implementarse y está permitida. Se pueden instalar múltiples tienda de aplicaciones y descargar cualquier archivo, pertenezca o no a estas tiendas de aplicaciones. Utilizar esto supone exponerse a ataques o acciones maliciosas por parte de los hackers. Este artículo crea un sistema de detección de *malware* en dispositivos *Android* llamado FEdroid. Extrae diferente información a partir de los archivos *Android Package Kit* (APK), que son archivos que contienen los datos de la propia aplicación. A partir de estos archivos obtienen el código fuente, las llamadas a las APIs o los permisos de las aplicaciones, información crucial a la hora de la detección de *malware*. Emplea el algoritmo *XGBoost* para validación cruzada para concentrarse en un conjunto de datos cruciales. Los algoritmos de clasificación que utiliza para detectar si es *malware* o no es *malware* son *XGBoost*, *Random Forest* (RF), *Support Vector Machine* (SVM), *Logistics Regression* (LR), *Gaussian Naive Bayes* (GNB) y *k-nearest neighbors* (KNN). La mejor precisión obtenida ha sido en el algoritmo *XGBoost* con un 98,26 % y la peor ha sido el algoritmo *GNB* con un 56,96 % (Huang et al., 2025).

Este enfoque destaca cómo la detección de *malware* en sistemas operativos móviles requiere trabajar directamente con los archivos *APK*. Sin embargo, no todos los ataques son tan estáticos: algunos tipos de *malware*, como el polimórfico, cambian continuamente sus características, lo que plantea nuevos retos.

Machine learning para *malware* polimórfico

El *malware* sigue siendo el mayor problema que enfrentan los usuarios de internet en el mundo actual. En este artículo se habla sobre un tipo de *malware* especial que es el polimorfismo, que es aquel que cambia continuamente sus características esenciales para evadir la detección mediante patrones de firmas convencionales. Se desarrolla un

modelo que sirve para detectar distintas familias de *malware* y qué software antivirus o anti-*malware* puede crear una firma para identificarlo. Los distintos tipos de *malware* de los que se habla en el estudio son virus, gusanos, troyanos, *adware*, *spyware*, *backdoor* y *ransomware*. Se explica las diferentes etapas que se van a desarrollar, como son la colección de datos, el preprocesamiento de datos y el desarrollo de los modelos predictivos a partir de los datos obtenidos. Posteriormente, se realiza un estudio de los resultados obtenidos. Los algoritmos utilizados son *Support Vector Machine* (SVM), *Random Forest* (RF) y *k-nearest neighbors* (KNN). Estos algoritmos establecen distintas métricas para los diferentes tipos de *malware* que se clasifican. El artículo concluye destacando que el uso del aprendizaje automático (ML) mejora significativamente la detección y clasificación del *malware*, lo cual resulta esencial frente al aumento constante en la cantidad y complejidad de los ataques (Manikanta Prasad et al., 2024).

Frente a estas variantes dinámicas, surge la necesidad de emplear técnicas más avanzadas, que combinen modelos clásicos de *machine learning* con métodos de *deep learning* y explicabilidad.

XAI* y *machine learning* en la detección de *malware* en *Android

El *malware* representa una grave amenaza para la seguridad y la privacidad, ya que roba datos sensibles y altera los sistemas informáticos. Los métodos tradicionales de detección basados en firmas se han vuelto ineficientes y consumen mucho tiempo. En este artículo habla como el aprendizaje automático (ML) y el aprendizaje profundo (DL) puede ayudar a la hora de detectar diferente *malware*. Utiliza Inteligencia Artifical Explicable (XAI), que es un conjunto de procesos y métodos que permiten a los usuarios humanos comprender y confiar en los resultados creados por los algoritmos de aprendizaje automático (ML) (IBM, 2024). El conjunto de datos necesarios para entrenar los diferentes modelos de ML se llama CICMalDroid. Está enfocado a la detección de *malware* en sistemas operativos *Android*. Habla de los diferentes métodos de preprocesamiento de datos como es *BERT* para extraer características importantes. Utiliza *SMOTE*, una técnica de *over-sampling*, como método de balanceo para el algoritmo de clasificación. *XGBoost* (XGB), *LightGBM* (LGBM), *Decision Tree* (DT), *Gradient Boosted Trees* (GBT), *Support Vector Machine* (SVM), *Logistics Regression* (LR), *Random Forest* (RF) son los algoritmos utilizados para entrenar al diferente conjunto de datos. Después de la evaluación de los modelos, en el que *LightGBM* obtiene la mejor precisión, realiza un análisis de explicabilidad con *SHAP* y *LIME*, en los que se explica los resultados del modelo de aprendizaje (Baghirov, 2025).

El uso de técnicas de *explainable artificial intelligence* (XAI) aporta un valor añadido, ya que no solo se logra una alta precisión, sino que también se proporciona transparencia en las decisiones de los modelos. Más allá de la explicabilidad, otra línea de investigación busca aumentar la robustez y la capacidad de generalización mediante modelos de ensamblado.

Modelos de ensamblado en la clasificación de *malware*

En el artículo habla como el *malware* puede provocar la pérdida de información importante o alterarla de manera perjudicial. Asimismo, expone como los métodos tradicionales para detectar y detener este tipo de amenazas no son siempre eficaces debido a que consumen mucho tiempo y no identifican nuevas variantes de *malware*. Propone un enfoque robusto basado en un modelo de ensamblado para la detección y clasificación de *malware*. El conjunto de datos con el que se trabaja se compone de diferentes tipos de *malware* como son *exploits*, *rootkits*, *hulk*, *goldenEye*, troyanos, *backdoor* y otros. El modelo propuesto combina tres clasificaciones base para mejorar la precisión y resistencia frente al malware en evolución. Algunos de los clasificadores utilizados son *Random Forest* (RF), *Support Vector Machine* (SVM), *Decision Tree* (DT), *Logistics Regression* (LR), *Naive Bayes* (NB), *Adaboost* y *Convolutional Neural Networks* (CNN). La mayoría de los clasificadores tienen una precisión mayor de un 85 % menos SVM que tiene un 74 %. Se realiza el preprocesamiento de datos y se realiza el balanceo de clases utilizando *SMOTE*. Por otro lado, se emplea el algoritmo *Gray Wolf Optimization* (GWO) para extraer las mejores características. Se construye el mejor modelo de ensamblado utilizando tres modelos secuenciales. El resultado final utilizando el modelo de ensamblado construido obtiene una precisión del 96,20 %. Por último, se muestran diferentes gráficas que apoyan el resultado y el número de falsos positivos, falsos negativos, verdaderos positivos y verdaderos negativos (Shelke et al., 2024). Este tipo de enfoques representan un paso más en la evolución del uso de *machine learning* en ciberseguridad: desde algoritmos individuales aplicados a entornos concretos (IoT, tráfico de red, *Android*) hasta técnicas modernas basadas en ensamblados, optimización de características y explicabilidad, que permiten abordar de manera más sólida la detección y clasificación del *malware*.

En resumen, los trabajos revisados muestran una evolución clara en la aplicación del *machine learning* para la detección de *malware*: desde entornos específicos como el *internet of things* y el tráfico de red, hasta dispositivos móviles basados en *Android*, incorporando posteriormente técnicas más avanzadas como modelos de ensamblado e inteligencia artificial explicable (*XAI*). Sin embargo, la mayoría de estos estudios se centran en problemas de clasificación binaria (*malware*/no *malware*) o en la identificación de familias de *malware*, sin abordar en profundidad escenarios multiclase con fuerte desbalance.

A diferencia de las investigaciones previas, en el presente trabajo se ha creado un *dataset* propio, actual y construido a partir de múltiples fuentes de datos. Además, se ha realizado un estudio comparativo exhaustivo de múltiples algoritmos de *machine learning* —desde modelos clásicos hasta técnicas avanzadas de *boosting*— junto con diversas estrategias de preprocesamiento y balanceo de clases. Este enfoque no solo permite evaluar de forma sistemática las capacidades de cada modelo en un problema multiclase complejo, sino también identificar qué técnicas resultan más efectivas.

3. Fundamentos teóricos

En esta sección se abordarán los elementos clave utilizados a lo largo del desarrollo del trabajo. Se brindará una base teórica que facilite la comprensión de su utilidad y su papel dentro del contexto general.

En primer lugar, el desarrollo se centrará en la inteligencia artificial y sus ámbitos relacionados que nos han servido de utilidad a la hora de realizar el trabajo. Se estudiará que área de la inteligencia artificial es más útil y por qué, así como los distintos algoritmos que se deben de aplicar para nuestro modelo de datos concreto.

Por otro lado, se explicará el significado de *malware* y los diferentes tipos y conceptos relacionados que han sido utilizados en el trabajo a la hora de realizar la clasificación. Se comprenderá porque es importante en la actualidad y porque debemos de mitigar y establecer riesgos respecto a ello.

3.1. Machine learning y deep learning

Aprendizaje automático -*Machine Learning* - es la ciencia de programar ordenadores para que puedan aprender de los datos. Según Arthur Samuel, en 1959, lo define como campo de estudio que da a los ordenadores la capacidad de aprender sin ser programados explícitamente. Tom Mitchell, en 1997, lo describe como si un programa informático aprende de la experiencia E con respecto a una tarea T y alguna medida de rendimiento P, si su rendimiento en T, medido por P, mejora con la experiencia E. (Géron, 2019)

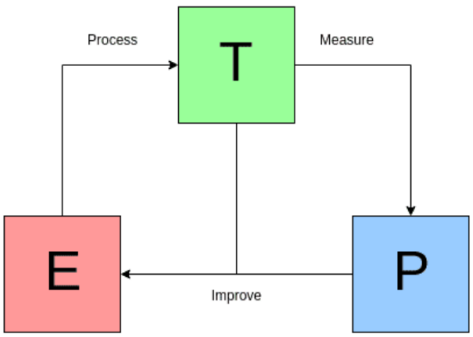


Figura 3.1: Paradigma de Mitchell. (Mitchell, 2018)

Su filtro de spam es un programa de aprendizaje automático que, dados ejemplos de mensajes de spam (por ejemplo, marcados por los usuarios) y ejemplos de correos electrónicos normales, puede aprender a marcar el spam. Los ejemplos que el sistema utiliza para aprender se denominan conjunto de entrenamiento. Cada ejemplo de entrenamiento se denomina instancia (o muestra) de entrenamiento. En este caso, la

tarea T consiste en marcar el spam de los nuevos correos electrónicos, la experiencia E son los datos de entrenamiento y es necesario definir la medida de rendimiento P; por ejemplo, se puede utilizar la proporción de correos electrónicos clasificados correctamente. Esta medida de rendimiento se denomina precisión y se utiliza a menudo en tareas de clasificación. (Géron, 2019)

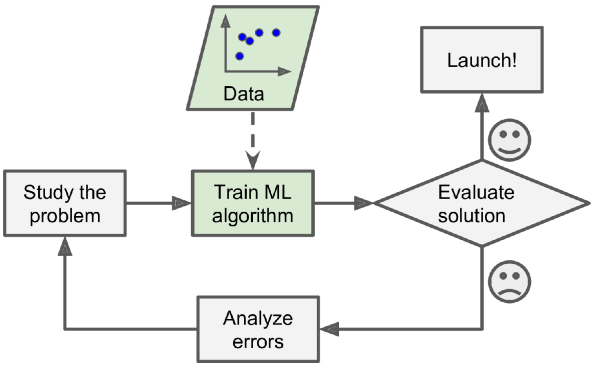


Figura 3.2: Enfoque tradicional *Machine Learning* (Géron, 2019)

La Figura 3.2 ilustra el enfoque tradicional para aplicar *Machine Learning*. Se parte del análisis del problema que se desea resolver, después se seleccionan los datos y se entrena un modelo de aprendizaje automático y si el resultado es bueno el sistema se lanza. En caso contrario, se analizan los errores. El proceso es iterativo pero no automatizado.

Algunos ejemplos dónde se aplica el aprendizaje automático pueden ser:

- *Clasificación de archivos como benignos o maliciosos a partir de sus características estáticas*
  Se trata de un problema de clasificación binaria, comúnmente abordado con algoritmos de *Gradient Boosting* (*CatBoost*, *XGBoost*, *LightGBM*) (Shabtai et al., 2012).
- *Identificación de tráfico malicioso en la red*
  Consiste en clasificar flujos de red para detectar ataques como *phishing*, *botnets* o *DDoS*, normalmente usando técnicas de detección de anomalías (Sommer y Paxson, 2010).
- *Predicción de intentos de intrusión en sistemas de información*
  Se basa en el análisis de logs de seguridad y se aborda mediante algoritmos supervisados y no supervisados (*clustering*, *autoencoders*) (Ring et al., 2019).
- *Priorización de alertas en sistemas IDS/IPS*
  El aprendizaje automático ayuda a reducir falsos positivos y a dar más peso a las alertas críticas (Symantec, 2019).

Hay diferentes tipos de sistemas de aprendizaje automático que resulta útil clasificarlos en grandes categorías, basándose en los siguientes criterios:

- Si se entrenan o no con supervisión humana (supervisados, no supervisados, semisupervisados y de aprendizaje por refuerzo).

- Si pueden o no aprender de forma incremental sobre la marcha (aprendizaje en línea o por lotes).
- Si funcionan simplemente comparando nuevos puntos de datos con puntos de datos conocidos, o detectando patrones en los datos de entrenamiento y construyendo un modelo predictivo, como hacen los científicos (aprendizaje basado en instancias frente a aprendizaje basado en modelos).

3.1.1. Aprendizaje no supervisado

El **aprendizaje no supervisado** es aquel en el que solo se nos dan datos de salida, sin ninguna información de entrada. El objetivo es descubrir estructuras interesantes en los datos, lo que a veces se denomina *descubrimiento de conocimiento*. En su lugar, formalizaremos nuestra tarea como una **estimación de la densidad**, es decir, queremos construir modelos de la forma $p(x_i|\theta)$.

Hay dos diferencias con respecto al caso supervisado:

- En primer lugar, se desarrolla $p(x_i|\theta)$ en lugar de $p(y_i|x_i, \theta)$; es decir, el aprendizaje supervisado es una *estimación de densidad condicional*, mientras que el no supervisado es una *estimación de densidad incondicional*.
- En segundo lugar, x_i es un vector de características, por lo que necesitamos crear modelos de probabilidad multivariantes. En cambio, en el aprendizaje supervisado, y_i suele ser una única variable que intentamos predecir. Esto significa que, para la mayoría de los problemas supervisados, podemos utilizar modelos de probabilidad univariantes (con parámetros dependientes de la entrada), lo que simplifica considerablemente el problema.

Podría decirse que el aprendizaje no supervisado es más típico del aprendizaje humano y animal.

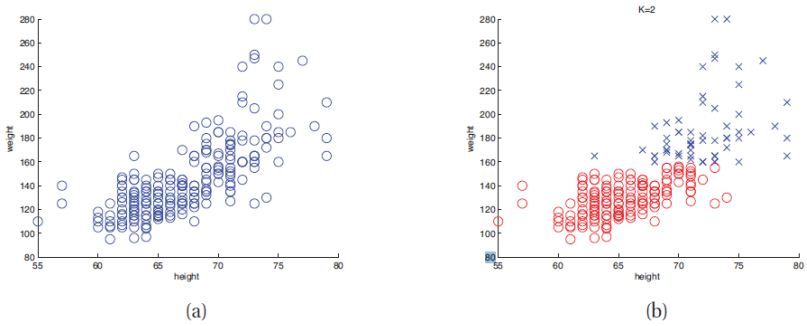


Figura 3.3: (a) La altura y el peso de algunas personas. (b) Un posible clustering utilizando K = 2 clusters.

Algunos de los algoritmos de aprendizaje no supervisado más importantes son:

- ***Clustering***

- *K-Means*
- *DBSCAN*

- **Reducción de la dimensionalidad**
  - PCA (*Principal Component Analysis*
  - t-SNE (*t-Distribuited Stochastic Neighbor Embedding*
- **Detección de anomalías**
  - *Isolation Forest*

3.1.2. Aprendizaje supervisado

En el enfoque de **aprendizaje predictivo o supervisado**, el objetivo es aprender una correspondencia entre las entradas $\mathbf{x}$ y las salidas y, dado un conjunto etiquetado de pares entrada-salida $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$. A este conjunto se le denomina *conjunto de entrenamiento* y N es el número de ejemplos de entrenamiento.

En el caso más sencillo, cada entrada de entrenamiento $\mathbf{x}_i$ es un vector numérico de dimensión D que representa, por ejemplo, características como la altura y el peso de una persona. Estos valores se denominan *características*, *atributos* o *covariables*. Sin embargo, en general, $\mathbf{x}_i$ puede ser un objeto estructurado complejo, como una imagen, una frase, un mensaje de correo electrónico, una serie temporal, una forma molecular, un grafo, etc.

Del mismo modo, la forma de la variable de salida o de respuesta puede ser, en principio, cualquier cosa. No obstante, la mayoría de los métodos asumen que y_i es una variable categórica o nominal de un conjunto finito, $y_i \in \{1, \ldots, C\}$ (por ejemplo, hombre o mujer), o bien un escalar de valor real (como el nivel de ingresos).

Cuando y_i es categórico, el problema se conoce como *clasificación* o *reconocimiento de patrones*; cuando y_i es de valor real, el problema se conoce como *regresión*. Existe también una variante conocida como *regresión ordinal*, que ocurre cuando el espacio de etiquetas $\mathcal{Y}$ tiene algún orden natural, como las calificaciones A–F.(Géron, 2019)

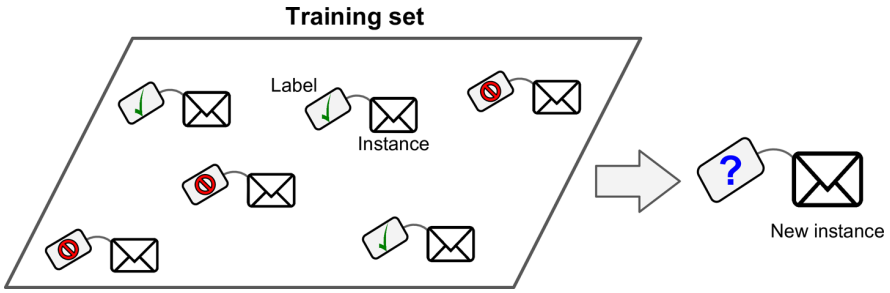


Figura 3.4: Ejemplo de aprendizaje supervisado (Géron, 2019)

Dentro del aprendizaje supervisado hay dos tipos de tareas:

- **Clasificación**: Cuando se tiene como objetivo principal predecir una clase.
- **Regresión**: Cuando el objetivo es predecir un valor continuo.

Algunos ejemplos de algoritmos de aprendizaje supervisados son:

- Regresión lineal
- Regresión logística
- Máquinas de soporte vectorial(SVM)
- Árboles de decisión
- Bosques aleatorios (*Random Forest*)
- K-vecinos más cercanos (*K-NN*)
- Redes neuronales artificiales
- Naive Bayes
- Métodos de boosting (como *CatBoost*, *Gradient Boosting*, *XGBoost*)

En este trabajo se abordarán los diferentes tipos de algoritmos en relación con un problema de clasificación. Se explicará con detalle cada uno a continuación:

K-Vecinos más cercanos (*K-NN*)

El algoritmo **K-vecinos más cercanos (*K-Nearest Neighbors*, *K-NN*)** es un método de aprendizaje supervisado no paramétrico utilizado tanto para problemas de clasificación como de regresión. Su principio se basa en que datos similares suelen encontrarse cerca en el espacio de características.(Hastie et al., 2009)

Dado un conjunto de entrenamiento $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$, para predecir la etiqueta de una nueva muestra $\mathbf{x}$, el algoritmo sigue los siguientes pasos:

1. **Cálculo de distancias:** se mide la distancia entre $\mathbf{x}$ y cada punto del conjunto de entrenamiento.
2. **Selección de vecinos:** se eligen los k puntos más cercanos a $\mathbf{x}$.
3. **Predicción:** se asigna la clase más frecuente entre los k vecinos.

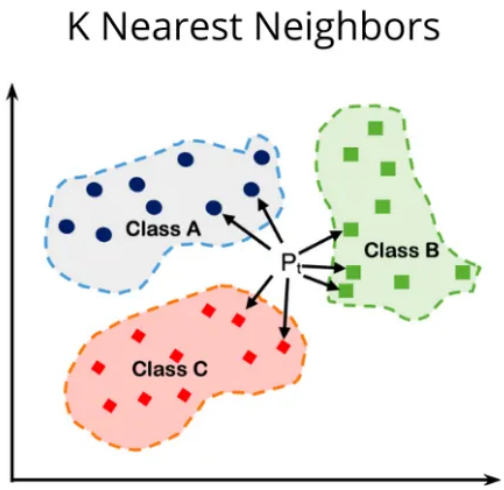


Figura 3.5: Ejemplo de *K-NN* (Soni, 2019)

Hay diferentes tipos de medidas de distancia. Las más importantes son: (Mitchell, 1997)

- **Distancia Euclídea:** es la medida de distancia más común y representa la distancia en línea recta entre dos puntos en un espacio n-dimensional.

$$d(\mathbf{x}, \mathbf{x}') = \sqrt{\sum_{i=1}^{n}(x_i - x_i')^2}$$

- **Distancia Manhattan (o de bloque):** se calcula como la suma de las diferencias absolutas entre las coordenadas correspondientes.

$$d(\mathbf{x}, \mathbf{x}') = \sum_{i=1}^{n} |x_i - x_i'|$$

- **Distancia de *Hamming*:** utilizada cuando los vectores contienen valores categóricos o binarios. Cuenta el número de posiciones en las que los elementos difieren.

$$d(\mathbf{x}, \mathbf{x}') = \sum_{i=1}^{n} \mathbf{I}(x_i \neq x_i')$$

Árbol de decisión (*Decision Tree*)

Un **árbol de decisión** es un modelo de aprendizaje supervisado utilizado tanto para tareas de *clasificación* como de *regresión*. Se representa como una estructura jerárquica en forma de árbol, donde cada nodo interno corresponde a una prueba sobre una característica, cada rama representa el resultado de esa prueba, y cada hoja proporciona una predicción (una clase o un valor numérico).(Hastie et al., 2009)

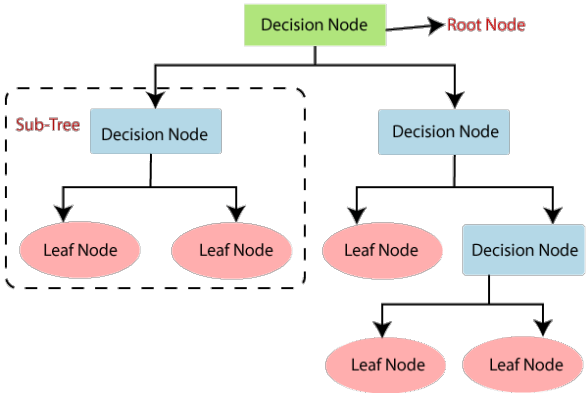


Figura 3.6: Ejemplo de un *Decision Tree* (Fernández, 2020)

El modelo se construye dividiendo de manera recursiva el conjunto de datos en subconjuntos más homogéneos, en función de alguna métrica, como son:

- La **entropía** proviene de la teoría de la información y mide la cantidad de incertidumbre asociada a una distribución de clases. Para un nodo con C clases, se define como:

$$H(p) = -\sum_{i=1}^{C} p_i \log_2(p_i)$$

  donde p_i es la proporción de elementos de la clase i en el nodo. La entropía es máxima cuando las clases están distribuidas uniformemente, e igual a cero cuando todas las muestras pertenecen a una sola clase. (Quinlan, 1993)

- El **índice de Gini** mide la probabilidad de clasificar incorrectamente una muestra seleccionada aleatoriamente si se asigna según la distribución de clases en el nodo. Se define como:

$$G(p) = 1 - \sum_{i=1}^{C} p_i^2$$

  Un valor de Gini bajo indica un nodo más puro. Es el criterio predeterminado en algoritmos como *CART*. (Breiman et al., 1984)

- La **varianza** se utiliza como criterio de impureza en árboles de *regresión*, donde la variable objetivo es continua. Se calcula como:

$$Var(S) = \frac{1}{|S|} \sum_{i=1}^{|S|} (y_i - \bar{y})^2$$

  donde $\bar{y}$ es la media de los valores de salida en el nodo S, y y_i son los valores individuales. Un valor bajo de varianza indica que los valores están más concentrados alrededor de la media. (Breiman et al., 1984)

Bosques Aleatorios (*Random Forest*)

Random Forest es un algoritmo de aprendizaje supervisado basado en árboles de decisión. Se trata de un método de *ensamblado* (ensemble) que combina múltiples árboles de decisión para mejorar la precisión y reducir el sobreajuste. (Hastie et al., 2009; Breiman, 2001)

La idea principal es entrenar muchos árboles de decisión independientes sobre diferentes subconjuntos de los datos y luego combinar sus predicciones:

- Para problemas de **clasificación**, cada árbol vota por una clase, y la clase final es la que recibe la mayoría de los votos.
- Para problemas de **regresión**, la predicción final es el promedio de las salidas de todos los árboles.

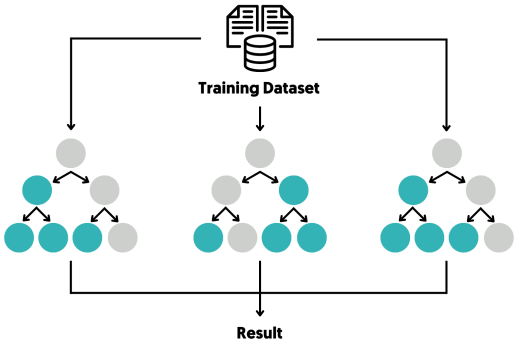


Figura 3.7: Ejemplo de *Random Forest* (DIDA.do, 2023)

CatBoost

CatBoost (*Categorical Boosting*) es un algoritmo de *gradient boosting* desarrollado por Yandex, diseñado para ofrecer alto rendimiento, eficiencia y manejo avanzado de variables categóricas. Al igual que otros métodos de boosting como *XGBoost* o *LightGBM*, *CatBoost* construye modelos de forma aditiva, entrenando árboles de decisión de manera secuencial, donde cada nuevo árbol corrige los errores de los anteriores. (Dorogush et al., 2018; Yandex, 2024)

- **Manejo automático de variables categóricas:** *CatBoost* convierte variables categóricas en valores numéricos de manera eficiente mediante estadísticas ordenadas y combinaciones de categorías, lo que evita el uso de técnicas de codificación manual como one-hot encoding.(Dorogush et al., 2018)
- **Reducción de *overfitting*:** Utiliza *target statistics* con esquemas de orden aleatorio para prevenir la fuga de información durante el entrenamiento.(Dorogush et al., 2018)
- **Compatibilidad con datos desbalanceados y ruido:** *CatBoost* es robusto frente a datos desequilibrados y variables irrelevantes. (Sharma, 2020)

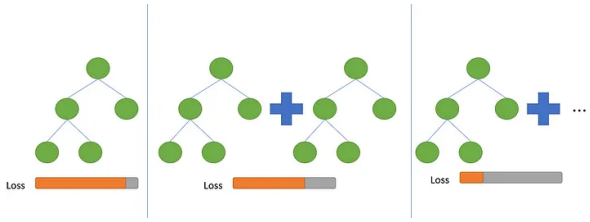


Figura 3.8: Ejemplo de *CatBoost* (Kushwaha, 2021)

XGBoost

XGBoost (*Extreme Gradient Boosting*) es un algoritmo de aprendizaje supervisado basado en *gradient boosting*, diseñado para optimizar la velocidad y el rendimiento del entrenamiento. Fue desarrollado por Tianqi Chen y ha sido ampliamente adoptado en competiciones de ciencia de datos y aplicaciones reales debido a su precisión y eficiencia (Chen y Guestrin, 2016; XGBoost Developers, 2024).

Al igual que otros métodos de boosting, *XGBoost* construye árboles de decisión de forma secuencial, donde cada nuevo árbol corrige los errores de predicción cometidos por el conjunto anterior (Nigam, 2021).

- **Regularización integrada:** Introduce términos de penalización L_1 y L_2 para reducir el sobreajuste, lo cual no está presente en muchas otras implementaciones de boosting (Chen y Guestrin, 2016).

- **Soporte para datos faltantes:** *XGBoost* maneja automáticamente los valores ausentes durante el entrenamiento y la predicción.

- **Compatibilidad con clasificación, regresión, ranking y detección de anomalías** (Shukla, 2020).

- **Función objetivo regularizada:** A diferencia de algoritmos clásicos, *XGBoost* minimiza una función objetivo compuesta por una función de pérdida y un término de regularización:

$$\mathcal{L}(\theta) = \sum_{i=1}^{n} l(y_i, \hat{y}_i) + \sum_{k=1}^{K} \Omega(f_k)$$

donde $\Omega(f) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2$ penaliza la complejidad del modelo (Chen y Guestrin, 2016).

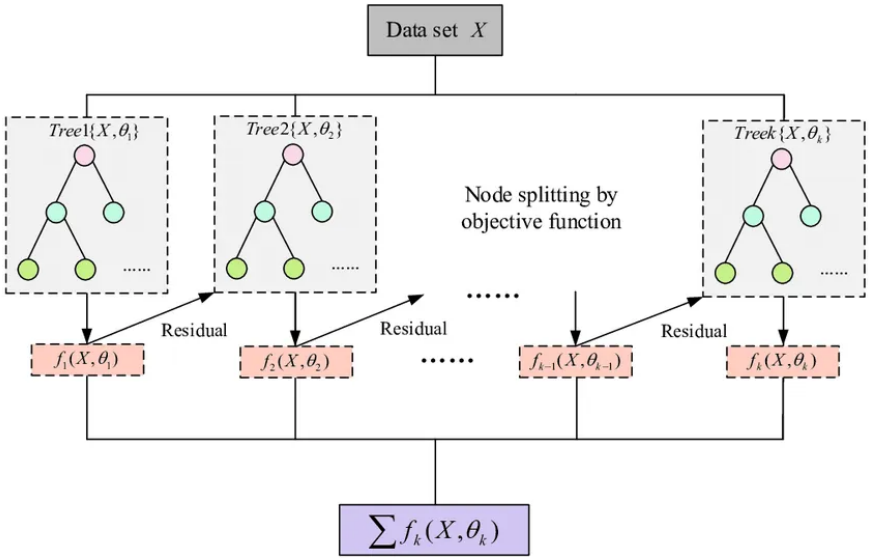


Figura 3.9: Ejemplo de *XGBoost* (Omarzai, 2021)

LightGBM

LightGBM (*Light Gradient Boosting Machine*) es un algoritmo de *gradient boosting* desarrollado por Microsoft, diseñado para ofrecer un entrenamiento rápido y eficiente con bajo uso de memoria. Está optimizado para trabajar con grandes volúmenes de datos y alto número de características (Ke et al., 2017).

Al igual que *XGBoost* y *CatBoost*, *LightGBM* entrena árboles de decisión de forma secuencial, donde cada nuevo árbol intenta corregir los errores del conjunto anterior. Su ventaja principal radica en técnicas únicas de construcción de árboles y manejo eficiente de datos escasos (Kareem, 2020).

- **Crecimiento de árbol orientado a hojas (*leaf-wise*):** a diferencia del crecimiento por niveles (*leaf-wise*) de otros algoritmos, *LightGBM* expande el nodo con mayor ganancia de información, lo que mejora la precisión del modelo, aunque puede requerir regularización adicional (Ke et al., 2017).

- **Soporte para valores faltantes y categóricos:** maneja automáticamente datos ausentes y convierte variables categóricas en divisiones eficientes (Microsoft, 2024).

- ***Binning* mediante histogramas:** convierte los valores continuos en intervalos discretos (bins), lo que acelera las comparaciones y reduce el uso de memoria.

- **Control del sobreajuste:** incluye parámetros como *max depth* y *min data* in leaf para regular la complejidad del modelo (Kareem, 2020).

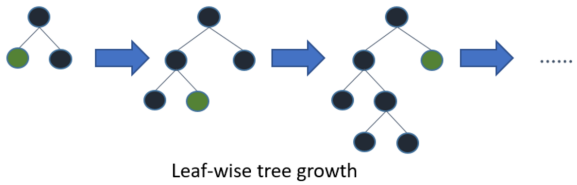


Figura 3.10: Ejemplo de LightGBM (Mandot, 2021)

Perceptrón Multicapa (*MLP*)

El **Perceptrón Multicapa** (*Multilayer Perceptron*, *MLP*) es un tipo de red neuronal artificial y uno de los modelos fundamentales del aprendizaje profundo. Consiste en una secuencia de capas de neuronas conectadas entre sí, en las que cada neurona aplica una función de activación no lineal a una combinación lineal de sus entradas (Goodfellow et al., 2016).

Un *MLP* típico tiene una capa de entrada, una o más capas ocultas y una capa de salida. Todas las capas (excepto la de salida) utilizan funciones de activación no lineales para permitir que la red aprenda representaciones complejas (Bishop, 2006).

- **Modelo completamente conectado:** cada neurona está conectada con todas las neuronas de la capa anterior y la siguiente.
- **Capacidad universal de aproximación:** un *MLP* con al menos una capa oculta y funciones de activación adecuadas puede aproximar cualquier función continua (Hornik et al., 1989).
- **Entrenamiento mediante retropropagación:** se utiliza el algoritmo de *backpropagation* junto con métodos de optimización como descenso por gradiente (*SGD*, *Adam*, etc.) para actualizar los pesos.
- **Requiere normalización y ajuste de hiperparámetros:** los *MLPs* suelen ser sensibles a la escala de los datos y a la elección del número de capas, unidades ocultas, tasa de aprendizaje y regularización.
- **Funciones de activación comunes:** ReLU, sigmoid y tanh son ampliamente usadas para introducir no linealidad.
- **Arquitectura de capas:** La cantidad de capas ocultas y el número de unidades por capa determinan la capacidad de representación del modelo.
- **Regularización:** Para evitar el sobreajuste, se utilizan técnicas como *dropout*, *early stopping* y regularización L_1 o L_2.
- **Limitaciones:** Aunque potentes, los MLPs pueden ser ineficientes para datos estructurados como imágenes o secuencias, donde arquitecturas especializadas (CNN, RNN) son preferibles.

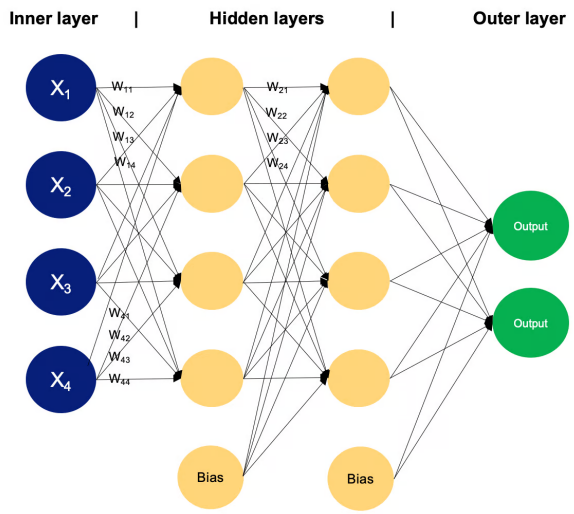


Figura 3.11: Ejemplo de un *MLP* (DataCamp, 2023)

Máquina de Soporte Vectorial (*Support Vector Machine (SVM)*

La **Máquina de Soporte Vectorial** (*Support Vector Machine (SVM)*) es un algoritmo de aprendizaje supervisado utilizado principalmente para tareas de *clasificación*, aunque también puede aplicarse a *regresión* (SVR). Su objetivo es encontrar el hiperplano que maximiza el margen entre dos clases en un espacio de características, logrando así una separación óptima (Cortes y Vapnik, 1995).

En el caso lineal, *SVM* encuentra un hiperplano tal que la distancia entre este y los puntos de entrenamiento más cercanos (vectores de soporte) sea máxima. En problemas no lineales, *SVM* utiliza funciones núcleo (*kernel*) para proyectar los datos a un espacio de mayor dimensión donde la separación sea linealmente posible (Bishop, 2006).

- **Máximo margen:** busca el hiperplano que deja el mayor margen posible entre clases, lo que mejora la generalización del modelo.
- **Robustez frente al sobreajuste:** gracias al uso del margen y la regularización, es menos propensa al sobreajuste en problemas de baja dimensión (Vapnik, 1998).
- **Soporte para clasificación binaria y multiclase (mediante estrategias como uno-contra-uno o uno-contra-resto).**
- **Función de decisión:** en la versión lineal, la predicción se hace como:

$$f(\mathbf{x}) = \text{sign}(\mathbf{w}^\top \mathbf{x} + b)$$

- **Parámetros clave:**
  - C: controla el equilibrio entre el margen y el error en entrenamiento.
  - γ: parámetro del *kernel* RBF que controla la influencia de cada muestra.
- **Limitaciones:** su entrenamiento puede ser computacionalmente costoso en grandes conjuntos de datos o en alta dimensionalidad.

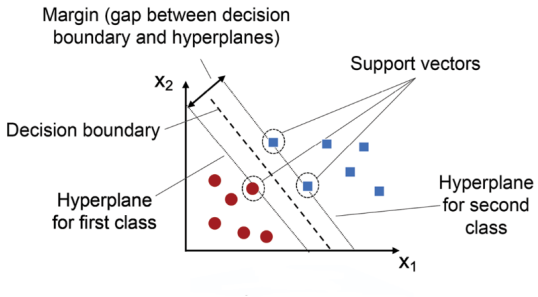


Figura 3.12: Ejemplo de hiperplano de SVM (Gupta, 2023)

Redes Neuronales

Las **Redes Neuronales Artificiales** (*Artificial Neural Networks*, *ANNs*) son un conjunto de modelos computacionales inspirados en la estructura del cerebro humano. Están compuestas por unidades llamadas *neuronas*, organizadas en capas, que procesan información mediante conexiones ponderadas (Bishop, 2006).

Las *ANNs* son especialmente efectivas para problemas donde las relaciones entre variables son no lineales, complejas o poco estructuradas. Se utilizan ampliamente en tareas de clasificación, regresión, detección de anomalías, procesamiento de lenguaje natural, visión por computador, entre otras (Goodfellow et al., 2016).

- **Modelo flexible y no lineal:** gracias al uso de funciones de activación no lineales como *ReLU*, *tanh* o *sigmoide*.
- **Capacidad de aprendizaje jerárquico:** las redes profundas pueden aprender representaciones progresivamente más abstractas a partir de los datos.
- **Alto rendimiento en tareas complejas:** las redes neuronales han alcanzado el estado del arte en diversas aplicaciones del aprendizaje automático.
- **Sensibilidad a la configuración:** requieren cuidado en la selección de hiperparámetros (tasa de aprendizaje, número de capas, regularización, etc.) (DataCamp, 2023).
- **Arquitectura:** una *ANN* básica consta de una capa de entrada, una o más capas ocultas y una capa de salida. Cada neurona realiza una transformación del tipo:

$$z = \mathbf{w}^\top \mathbf{x} + b, \quad a = \sigma(z)$$

  donde σ es la función de activación.
- **Regularización y prevención del sobreajuste:** se aplican técnicas como *dropout*, *batch normalization*, y *early stopping*.
- **Limitaciones:** el entrenamiento puede ser costoso en términos computacionales, y el modelo puede ser difícil de interpretar.

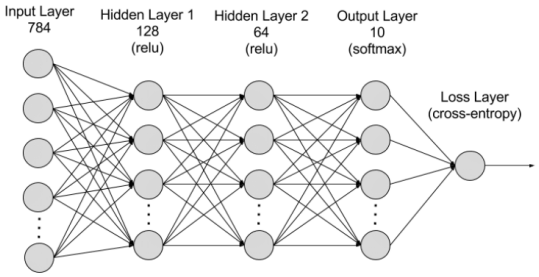


Figura 3.13: Ejemplo de una red neuronal (Amazon Web Services, 2024)

En el contexto de las redes neuronales, las **funciones de activación** desempeñan un papel fundamental al introducir *no linealidad* en el modelo. Sin funciones de activación no lineales, una red neuronal profunda colapsaría en una simple transformación lineal, sin capacidad para representar relaciones complejas entre las variables (Goodfellow et al., 2016).

Entre las funciones más utilizadas se encuentran:

- **Sigmoide:** convierte cualquier valor real en el intervalo $(0, 1)$. Es útil para tareas de clasificación binaria, pero puede sufrir del problema del *desvanecimiento del gradiente*.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- **Tangente hiperbólica (tanh):** similar a la sigmoide, pero con salida en el intervalo $(-1, 1)$. Centra los datos, lo cual puede ayudar al entrenamiento.

- **ReLU (Rectified Linear Unit):** es actualmente la función más popular en redes profundas. Devuelve cero para valores negativos y la identidad para valores positivos:

$$f(x) = \text{máx}(0, x)$$

  Su principal ventaja es que mitiga el desvanecimiento del gradiente, aunque puede provocar neuronas inactivas (*dying ReLUs*) (Glorot et al., 2011).

- **Leaky ReLU y variantes:** permiten un pequeño gradiente cuando la entrada es negativa, lo que evita la inactivación completa de neuronas.

- **Softmax:** convierte un vector de valores reales en una distribución de probabilidad. Se utiliza comúnmente en la capa de salida para clasificación multiclase:

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

La elección de la función de activación depende del tipo de problema, la arquitectura de la red y la función de pérdida utilizada. Las funciones no lineales permiten que las redes neuronales aprendan representaciones complejas y realicen tareas que los modelos lineales no podrían resolver (Bishop, 2006).

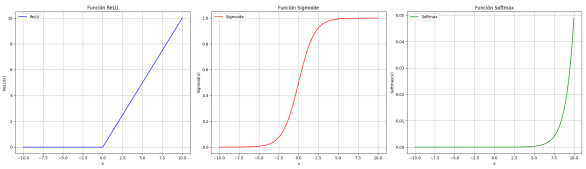


Figura 3.14: Funciones de activación (ReLU, Sigmoide, Softmax)

Dropout es una de las técnicas de regularización más populares para redes neuronales profundas. Fue propuesta inicialmente por Geoffrey Hinton en 2012 y posteriormente detallada por Srivastava et al. en 2014. Esta técnica ha demostrado ser altamente efectiva: incluso redes neuronales de última generación pueden mejorar su precisión entre un 1 % y 2 % simplemente añadiendo Dropout. Aunque parezca modesto, por ejemplo, mejorar de un 95 % a un 97 % de precisión representa una reducción del 40 % en la tasa de error (de 5 % a 3 %) (Géron, 2019).

El algoritmo es sencillo: en cada paso del entrenamiento, cada neurona (incluyendo las de entrada, pero excluyendo las de salida) tiene una probabilidad p de ser *desactivada temporalmente*, es decir, se la ignora completamente durante ese paso de entrenamiento, aunque puede estar activa en el siguiente.

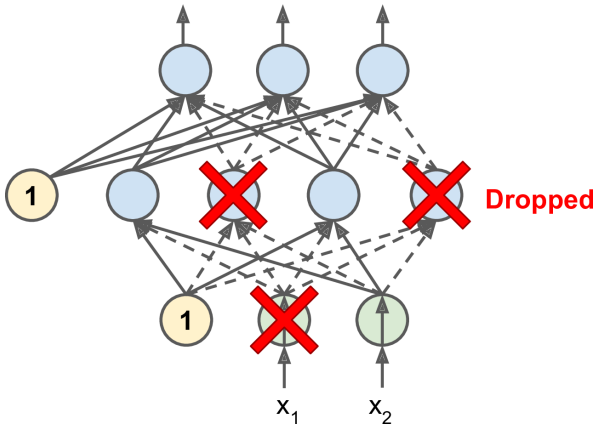


Figura 3.15: Ejemplo de *dropout* que elimina 3 neuronas (Géron, 2019)

3.1.3. Métodos de balanceo

En tareas de clasificación, los conjuntos de datos desbalanceados presentan un reto significativo: los modelos tienden a favorecer la clase mayoritaria, ignorando patrones importantes en la clase minoritaria. Este sesgo perjudica métricas relevantes como la sensibilidad, el F1-score o el área bajo la curva ROC (Fernández et al., 2018). Para abordar este problema, se utilizan dos estrategias complementarias: **sobremuestreo** y **submuestreo**.

Sobremuestreo

El **sobremuestreo** busca aumentar la representatividad de la clase minoritaria generando nuevas instancias. Uno de los métodos más conocidos es:

- **SMOTE (*Synthetic Minority Over-sampling Technique*):** propuesto por Chawla et al. (Chawla et al., 2002), este método crea ejemplos sintéticos en vez de duplicar instancias reales. Para cada ejemplo de la clase minoritaria, se seleccionan k vecinos más cercanos. Luego, se generan nuevas muestras interpolando aleatoriamente entre el punto original y alguno de sus vecinos. Esto permite expandir el espacio de representación de la clase minoritaria de forma más general, evitando el sobreajuste asociado al sobremuestreo aleatorio (learn Developers, 2024a).

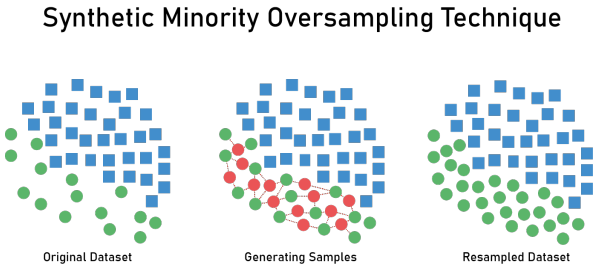


Figura 3.16: Ejemplo de muestras con SMOTE (Ghosh, 2020)

Submuestreo

El **submuestreo** intenta equilibrar la distribución eliminando ejemplos de la clase mayoritaria. Puede hacerse de forma aleatoria o dirigida. Entre los métodos más conocidos se encuentran:

- ***Random UnderSampling* (RUS):** consiste en eliminar instancias de la clase mayoritaria de forma aleatoria hasta igualar su tamaño con la clase minoritaria. Aunque simple y rápido, puede eliminar ejemplos representativos e importantes, reduciendo la capacidad predictiva del modelo (learn Developers, 2024d).

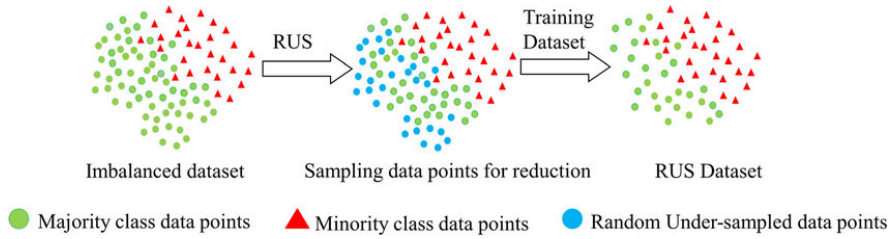


Figura 3.17: Ejemplo de balanceo con *Random UnderSampling* (ResearchGate, 2020a)

- ***Edited Nearest Neighbours* (ENN):** propuesto por Wilson (Wilson, 1972), este método examina cada muestra y sus k vecinos más cercanos. Si la clase

de una muestra difiere de la mayoría de sus vecinos, se elimina del conjunto. ENN mejora la calidad de la frontera de decisión, eliminando puntos ambiguos o ruidosos, lo que resulta útil para modelos sensibles al ruido (learn Developers, 2024c).

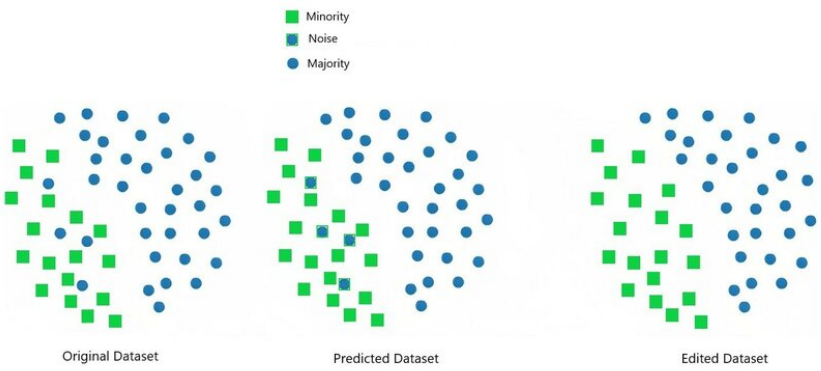


Figura 3.18: Ejemplo de balanceo con *Edited Nearest Neighbors* (ResearchGate, 2023)

- **Cluster Centroids:** este método, descrito por Japkowicz y Stephen (Japkowicz y Stephen, 2002), aplica un algoritmo de clustering (como K-means) a la clase mayoritaria y reemplaza los grupos por sus centroides. Así, se reduce el número de instancias manteniendo una representación estructurada de la clase. Es una técnica más informada que RUS, aunque introduce una fase de agrupamiento computacionalmente más costosa (learn Developers, 2024b).

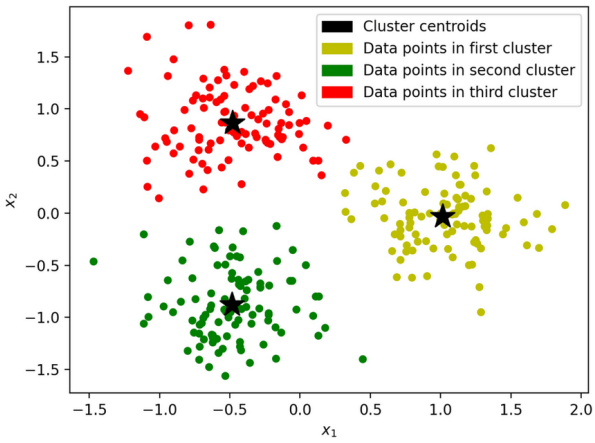


Figura 3.19: Ejemplo de balanceo con Cluster Centroids (ResearchGate, 2020b)

3.1.4. Métricas de evaluación en clasificación

La evaluación del rendimiento de un modelo de clasificación no debe limitarse a la *exactitud* (*accuracy*), especialmente en contextos con clases desbalanceadas. Existen métricas complementarias que ofrecen una visión más detallada del comportamiento del modelo (Powers, 2011).

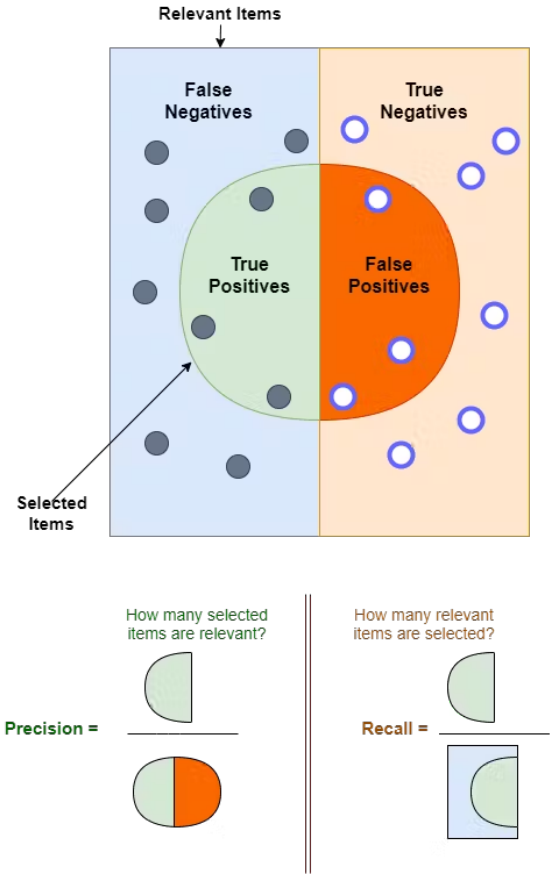


Figura 3.20: Precision y *Recall* de manera gráfica (Encord, 2023)

Precisión (Precision)

La **precisión** mide la proporción de verdaderos positivos entre todos los casos que el modelo ha clasificado como positivos:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Es útil cuando el coste de un falso positivo es alto (por ejemplo, detección de spam).

Exhaustividad (*Recall* o Sensibilidad)

El ***recall*** mide la proporción de verdaderos positivos entre todos los casos reales positivos:

$$Recall = \frac{TP}{TP + FN}$$

Es esencial cuando los falsos negativos son costosos, como en el diagnóstico médico.

F1-Score

El **F1-score** es la media armónica entre precisión y recall:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Se recomienda cuando se busca un equilibrio entre precisión y recall, especialmente en contextos desbalanceados.

Exactitud (*Accuracy*)

La **exactitud** representa la proporción total de predicciones correctas sobre el total de muestras:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Aunque es intuitiva, puede ser engañosa si las clases están desbalanceadas.

Curva ROC y AUC

La **curva ROC** (*Receiver Operating Characteristic*) muestra la relación entre el *true positive rate* (recall) y el *false positive rate*:

$$\text{FPR} = \frac{FP}{FP + TN}$$

El área bajo la curva (AUC) cuantifica el rendimiento general del modelo. Un AUC cercano a 1 indica una excelente capacidad de discriminación.

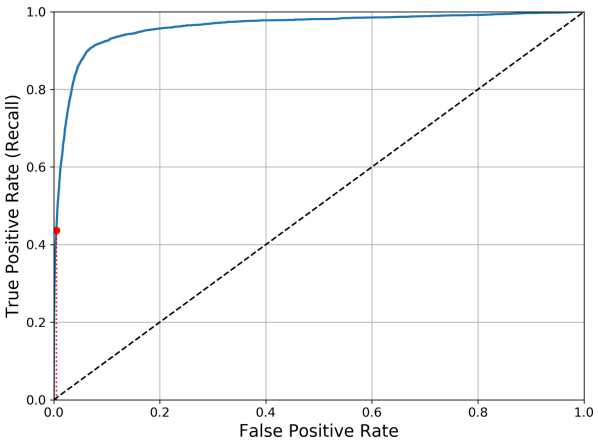


Figura 3.21: Ejemplo de curva ROC (Géron, 2019)

3.1.5. Matriz de confusión

La **matriz de confusión** es una herramienta fundamental para evaluar el rendimiento de un modelo de clasificación. Resume los resultados de las predicciones comparándolos con las verdaderas etiquetas de clase, permitiendo identificar errores específicos y calcular múltiples métricas asociadas (Powers, 2011).

Para una clasificación binaria, la matriz se estructura como sigue:

	Predicho Positivo	**Predicho Negativo**
Real Positivo	TP (*True Positive*)	FN (*False Negative*)
Real Negativo	FP (*False Positive*)	TN (*True Negative*)

Interpretación

- **TP (Verdadero Positivo):** el modelo predijo positivo y era correcto.
- **TN (Verdadero Negativo):** el modelo predijo negativo y era correcto.
- **FP (Falso Positivo):** el modelo predijo positivo, pero era negativo (error tipo I).
- **FN (Falso Negativo):** el modelo predijo negativo, pero era positivo (error tipo II).

Relación con métricas

A partir de la matriz de confusión se derivan muchas métricas clave:

$$Accuracy = \frac{TP + TN}{TP + FP + FN + TN}$$

$$Precision = \frac{TP}{TP + FP}, \quad Recall = \frac{TP}{TP + FN}, \quad F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

La matriz de confusión también se puede extender a problemas de clasificación multiclase, donde cada fila y columna representa una clase distinta, permitiendo detectar en qué clases el modelo comete más errores.

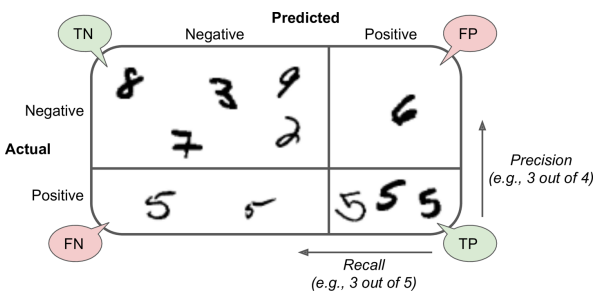


Figura 3.22: Ejemplo de matriz de confusión (Géron, 2019)

3.1.6. Cross-Validation

La **validación cruzada** es una técnica estadística utilizada para evaluar el rendimiento de un modelo de aprendizaje automático y estimar su capacidad de generalización a nuevos datos. Es especialmente útil cuando se dispone de un conjunto de datos limitado, ya que permite reutilizar de manera eficiente los datos disponibles (Arlot y Celisse, 2010). Sirve también para evitar el *Overfitting*.

Cross-validation k-fold

El esquema más común es la **validación cruzada k-fold**, que consiste en dividir el conjunto de datos en k subconjuntos (o *folds*) de tamaño similar. El procedimiento es el siguiente:

1. Se reservan uno a uno los k subconjuntos como conjunto de validación.
2. Se entrena el modelo con los $k - 1$ subconjuntos restantes.
3. Se evalúa el modelo sobre el subconjunto reservado.
4. Este proceso se repite k veces, y se promedian los resultados.

Esta técnica proporciona una estimación más estable y robusta del rendimiento del modelo que una simple partición entrenamiento/prueba.

La validación cruzada también puede usarse para ajustar hiperparámetros mediante *búsqueda en rejilla* (*grid search*) o *optimización bayesiana*.

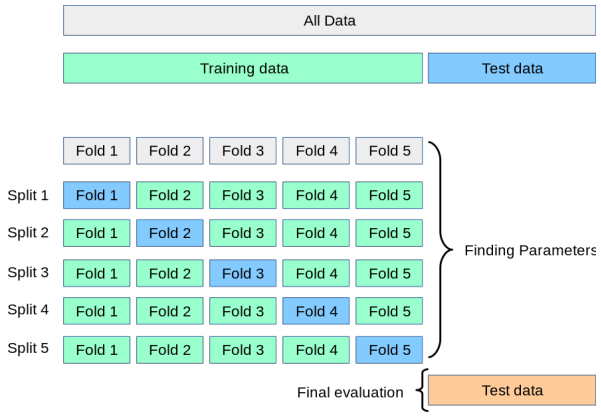


Figura 3.23: *Cross-Validation* con 5-*fold* (learn Developers, 2024e)

3.1.7. Sobreajuste y subajuste

A la hora de realizar el entrenamiento de los diferentes algoritmos utilizados para el proyecto, los resultados pueden dar lugar a confusión debido a que los datos proporcionados no son de calidad o no se encuentran balanceados. Es por ello, que se debe de prestar una gran atención porque puede dar dar lugar a sobreajuste (*overfitting*) o subajuste (*underfitting*).

Sobreajuste

Supongamos que visitas un país extranjero y un taxista te estafa. Podrías sentirte tentado a pensar que todos los taxistas de ese país son ladrones. Esta **sobregeneralización** es un error común en los humanos, y lamentablemente, las máquinas pueden caer en la misma trampa si no se tiene cuidado (Géron, 2019).

En **aprendizaje automático**, este fenómeno se conoce como *sobreajuste* (*overfitting*): ocurre cuando un modelo se ajusta muy bien a los datos de entrenamiento, pero no generaliza adecuadamente a datos nuevos.

Modelos complejos, como las redes neuronales profundas, tienen la capacidad de detectar patrones sutiles en los datos. Sin embargo, si el conjunto de entrenamiento es ruidoso o demasiado pequeño (lo que introduce *ruido de muestreo*), es probable que el modelo detecte patrones que no existen realmente, es decir, patrones en el ruido.

Por ejemplo, supón que introduces muchas variables en un modelo que predice el nivel de satisfacción de vida, incluidas variables irrelevantes como el nombre del país. En ese caso, un modelo complejo podría identificar un patrón como: `todos los países en el conjunto de entrenamiento que contienen una w en su nombre tienen una satisfacción mayor a 7`:

- *New Zealand* (7.3)
- *Norway* (7.4)
- *Sweden* (7.2)
- *Switzerland* (7.5)

¿Con qué grado de confianza puedes asegurar que esta regla también se aplica a países como *Rwanda* o *Zimbabwe*? Es evidente que este patrón surgió por simple azar dentro del conjunto de entrenamiento, pero el modelo no tiene forma de saber si es real o solo ruido.

El sobreajuste sucede cuando el modelo es **demasiado complejo en relación con la cantidad y calidad (nivel de ruido) de los datos de entrenamiento**.

Existen varias estrategias para reducir el sobreajuste:

- **Simplificar el modelo**: seleccionando uno con menos parámetros (por ejemplo, un modelo lineal en lugar de uno polinómico de alto grado), reduciendo el número de atributos, o restringiendo la flexibilidad del modelo.
- **Aumentar el tamaño del conjunto de entrenamiento**.
- **Reducir el ruido en los datos**: corrigiendo errores, eliminando valores atípicos y limpiando datos irrelevantes.

Subajuste

El **subajuste** (*underfitting*) es lo opuesto al sobreajuste: ocurre cuando el modelo es demasiado simple para aprender la estructura subyacente de los datos (Géron, 2019).

Por ejemplo, un modelo lineal para predecir la satisfacción con la vida puede tender al subajuste, ya que la realidad es generalmente más compleja que lo que el modelo puede capturar. En consecuencia, sus predicciones serán inexactas incluso sobre los propios datos de entrenamiento.

A continuación se presentan algunas estrategias comunes para abordar el subajuste:

- **Seleccionar un modelo más potente**, con mayor capacidad (más parámetros o arquitectura más compleja).
- **Proporcionar mejores características** al algoritmo de aprendizaje, mediante técnicas de *ingeniería de características* (*feature engineering*).
- **Reducir las restricciones impuestas al modelo**, por ejemplo, disminuyendo el hiperparámetro de regularización.



Figura 3.24: Comportamiento de *Overfitting* y *Underfitting* (Patel, 2022)

3.2. Malware

En esta sección se dará una visión general del significado del *malware* y de la importancia que tiene hoy en día. Se hablará, desde sus orígenes hasta la actualidad, los diferentes tipos de *malware* que hay y sus propósitos. También mencionaremos las características principales y las formas de mitigar estos ataques que producen consecuencias bastante duras.

3.2.1. ¿Qué es el *malware*?

El término *malware* proviene de la abreviatura inglesa de *malicious software*, es decir, *software* malicioso. Se refiere a cualquier programa informático diseñado con la

intención de dañar, interrumpir, robar o, en general, causar un comportamiento no deseado en sistemas, redes o dispositivos (Stallings, 2020).

Históricamente, el *malware* ha evolucionado desde simples virus informáticos en los años 80 hasta sofisticadas amenazas modernas como el *ransomware* (Skoudis y Zeltser, 2003).

Es importante distinguir entre *malware* y otros términos relacionados como *adware* (*software* que muestra publicidad no deseada) y *spyware* (*software* que recopila información sin consentimiento) (Security, 2020).

El propósito general del *malware* puede incluir robo de información, sabotaje de sistemas, espionaje industrial o gubernamental, y extorsión económica.

3.2.2. Clasificación del *malware*

El *malware* se presenta en diversas formas, cada una con características particulares en cuanto a su funcionamiento, forma de propagación y objetivos. A continuación, se describen las principales categorías:

Virus

Un virus informático es un tipo de *malware* que se adjunta a archivos ejecutables y requiere la intervención del usuario para activarse y propagarse. Al ejecutarse, puede replicarse infectando otros archivos o sistemas.

Un ejemplo histórico es el virus CIH (conocido como *Chernobyl*), que apareció en 1998 y causó graves daños al sobrescribir la BIOS del sistema (Center, 1999).

Gusanos(*Worms*)

Los gusanos son programas capaces de replicarse y propagarse automáticamente a través de redes sin intervención del usuario y sin necesidad de archivos anfitriones. Su capacidad para expandirse rápidamente los convierte en amenazas especialmente peligrosas.

Un ejemplo notable es *WannaCry*, un gusano con funcionalidad de ransomware que en 2017 afectó a miles de organizaciones en todo el mundo explotando vulnerabilidades del sistema operativo *Windows* (Lab, 2017).

Troyanos

Los troyanos se presentan como software legítimo para engañar al usuario. Una vez ejecutados, permiten a los atacantes acceder de forma remota al sistema, crear puertas traseras (*backdoors*) o instalar más *malware*.

Este tipo de *malware* suele ser utilizado para espionaje, robo de credenciales o toma de control de sistemas (Skoudis y Zeltser, 2003).

Ransomware

El *ransomware* cifra los archivos del usuario y exige el pago de un rescate para restaurar el acceso. Su impacto económico ha sido significativo en la última década.

Entre los ejemplos más conocidos se encuentran *NotPetya*, que afectó infraestructuras críticas, y *Locky*, difundido principalmente por campañas de correo electrónico (Response, 2016; Group, 2017).

Spyware

El *spyware* es un tipo de *malware* que espía la actividad del usuario sin su conocimiento, recopilando información como contraseñas, hábitos de navegación y datos personales.

Se utiliza frecuentemente para fines de publicidad o robo de identidad (Security, 2020).

Rootkits

Los *rootkits* son herramientas diseñadas para ocultar la presencia de otros programas maliciosos en un sistema, interfiriendo con funciones del sistema operativo para evitar la detección.

Pueden modificar partes del kernel del sistema operativo y son extremadamente difíciles de detectar y eliminar (Hoglund y Butler, 2005).

Keyloggers

Los *keyloggers* registran las pulsaciones del teclado, permitiendo a los atacantes capturar información confidencial como nombres de usuario, contraseñas o números de tarjetas de crédito (Skoudis y Zeltser, 2003).

Botnets

Las *botnets* son redes de dispositivos infectados (conocidos como *bots* o *zombies*) que son controlados de forma remota por un atacante, a menudo para lanzar ataques distribuidos de denegación de servicio (DDoS), enviar spam o propagar *malware* (Singh y De, 2016).

3.2.3. Características del *malware*

El *malware* moderno se caracteriza por una serie de capacidades que le permiten cumplir sus objetivos de manera efectiva y, a menudo, sigilosa. Estas características varían en función del tipo de *malware*, pero en general pueden agruparse en los siguientes aspectos:

Persistencia

La persistencia es la capacidad del *malware* de mantenerse activo en el sistema incluso después de reinicios o intentos de eliminación. Para ello, puede modificar claves del registro, servicios del sistema o utilizar técnicas más avanzadas como la creación de tareas programadas o el uso de *rootkits* (Hoglund y Butler, 2005).

Capacidad de ocultamiento

El *malware* emplea diversas técnicas para evitar ser detectado por el usuario o por soluciones de seguridad. Esto incluye el empaquetado de código, el uso de *polimorfismo*, cifrado de su contenido, inyección en procesos legítimos, o modificación del núcleo del sistema operativo (Stallings, 2020).

Propagación automática

Algunos tipos de *malware*, como los gusanos, tienen la capacidad de replicarse y propagarse automáticamente a otros sistemas mediante vulnerabilidades de red, dispositivos USB o técnicas de ingeniería social. Esto permite una expansión rápida y, en muchos casos, masiva (Lab, 2017).

Daño o impacto

El *malware* puede generar una amplia gama de daños, desde la corrupción o cifrado de datos, la ralentización del sistema, hasta la pérdida de control total sobre el dispositivo afectado. En entornos empresariales, puede derivar en pérdida financiera, robo de propiedad intelectual o daño reputacional (Response, 2016).

Evasión de mecanismos de defensa

Los creadores de *malware* diseñan sus programas con la capacidad de evadir sistemas de defensa como antivirus, firewalls, o sistemas de detección de intrusos (IDS). Para ello se utilizan técnicas como la ofuscación de código, cifrado, comportamientos condicionales (ejecutarse solo bajo ciertas condiciones), o la detección de entornos de análisis (*sandboxing*) para evitar ser examinados (Skoudis y Zeltser, 2003; Hoglund y Butler, 2005).

3.2.4. Detección y análisis de *malware*

La detección y el análisis del *malware* son componentes fundamentales de la ciberseguridad. Permiten identificar amenazas antes de que causen daños graves, así como entender su funcionamiento para mitigar futuras infecciones. Las principales técnicas de detección se agrupan en:

Técnicas de detección

Detección basada en firmas: esta técnica identifica *malware* mediante la comparación de fragmentos de código (firmas) con una base de datos conocida. Es eficaz para detectar malware ya catalogado, pero ineficaz frente a nuevas variantes o *malware* polimórfico (Stallings, 2020).

Detección basada en comportamiento: analiza las acciones que realiza un programa en tiempo de ejecución, como accesos al sistema de archivos, conexiones de red, o modificaciones al registro. Es útil contra *malware* desconocido o sin firma, aunque puede generar falsos positivos (Egele et al., 2012).

Detección heurística: esta técnica aplica reglas predefinidas y modelos estadísticos para evaluar si un programa tiene características típicas de *malware*. Es un enfoque proactivo que puede detectar amenazas nuevas, pero también presenta riesgos de clasificación errónea (You y Yim, 2010).

Análisis de *malware*

Análisis estático: consiste en examinar el código del *malware* sin ejecutarlo. Puede implicar ingeniería inversa, revisión del código ensamblador o análisis de cadenas embebidas. Es seguro, pero requiere experiencia técnica y puede ser ineficaz contra *malware* ofuscado o cifrado (Egele et al., 2012).

Análisis dinámico: implica ejecutar el *malware* en un entorno controlado o *sandbox* para observar su comportamiento en tiempo real. Permite identificar conexiones salientes, archivos creados y modificaciones en el sistema. Aunque es efectivo, algunos tipos de *malware* pueden detectar que están en un entorno virtualizado y alterar su comportamiento (Wueest, 2013).

4. Herramientas y tecnologías utilizadas

En este apartado se describirán las diferentes herramientas y tecnologías utilizadas a la hora de desarrollar el proyecto. Se explicará el *framework* utilizado para desarrollar el código, así como el lenguaje de programación y las librerías importantes dentro de este lenguaje de programación. Es importante mencionar esto debido a que muchas de estas librerías se componen de funciones principales y necesarias para la obtención de los resultados.

4.1. Python

Python es un lenguaje de programación potente y fácil de aprender. Posee eficientes estructuras de datos de alto nivel y un enfoque sencillo pero eficaz de la programación orientada a objetos. La elegante sintaxis de *Python* y su tipado dinámico, junto con su naturaleza interpretada, lo convierten en un lenguaje ideal para la creación de scripts y el desarrollo rápido de aplicaciones en muchos ámbitos y en la mayoría de las plataformas. En la mayoría de los proyectos relacionados con inteligencia artificial se utiliza este lenguaje debido a su facilidad de integración de librerías y funciones que se pueden utilizar. (Foundation, 2024)

4.2. Librerías de python

4.2.1. Numpy

NumPy es el paquete fundamental para la computación científica con *Python*. Es una librería de *Python* que proporciona una estructura de datos eficiente mediante el objeto `ndarray`, que representa arreglos multidimensionales homogéneos. Estos arreglos permiten realizar operaciones matemáticas, estadísticas, lógicas, de álgebra lineal, transformadas de Fourier, simulaciones aleatorias, entre otras, de forma optimizada y concisa (The NumPy Community, 2025).

A diferencia de las secuencias estándar de *Python* (como listas), los arreglos de *NumPy* presentan ventajas importantes:

- Tienen un tamaño fijo al momento de su creación.
- Todos sus elementos son del mismo tipo de datos, lo que permite una representación más eficiente en memoria.
- Soportan **vectorización** y ***broadcasting***, lo que elimina la necesidad de bucles explícitos.

Estas características han convertido a *NumPy* en el núcleo de muchas bibliotecas científicas y de machine learning en *Python*, como *SciPy*, *Pandas*, *Scikit-learn*, *TensorFlow*, entre otras.

4.2.2. Pandas

Pandas es una biblioteca de código abierto para el lenguaje de programación *Python*, especializada en la manipulación y análisis de datos. Proporciona estructuras de datos y operaciones para manipular tablas numéricas y series temporales. (The pandas Development Team, 2025)

Las principales estructuras de datos que ofrece pandas son:

- **Series**: Arreglos unidimensionales etiquetados capaces de contener cualquier tipo de dato (enteros, cadenas, números de punto flotante, objetos de *Python*, etc.).
- ***DataFrame***: Estructura bidimensional con etiquetas en los ejes (filas y columnas), similar a una hoja de cálculo o una tabla SQL.

Entre las funcionalidades destacadas de pandas se incluyen:

- Lectura y escritura de datos en diversos formatos como *CSV*, *Excel*, *JSON*, *HDF5* y bases de datos SQL.
- Manejo eficiente de datos faltantes representados como `NaN`.
- Indexación y selección de datos mediante etiquetas o posiciones.
- Agrupación de datos para realizar operaciones de agregación y transformación.
- Reestructuración y pivotado de conjuntos de datos.
- Integración con bibliotecas como *NumPy* y *Matplotlib* para análisis numérico y visualización.

Estas características hacen de pandas una herramienta esencial para científicos de datos, analistas y cualquier persona que trabaje con datos estructurados en *Python*.

4.2.3. Matplotlib

Matplotlib es una biblioteca de código abierto para la visualización de datos en *Python*. Proporciona una amplia gama de herramientas para crear gráficos estáticos, animados e interactivos de alta calidad. Su diseño modular y su API orientada a objetos permiten una personalización detallada de cada elemento del gráfico, desde las figuras y ejes hasta los estilos y anotaciones (The Matplotlib Development Team, 2025).

Entre las características más destacadas de Matplotlib se incluyen:

- **Versatilidad en los tipos de gráficos**: permite crear gráficos de líneas, dispersión, barras, histogramas, diagramas de caja, gráficos de sectores, mapas de calor, gráficos 3D, entre otros.

- **Integración con otras bibliotecas**: se integra de manera eficiente con bibliotecas como *NumPy* y *Pandas*, facilitando la visualización directa de estructuras de datos complejas.
- **Compatibilidad con múltiples entornos**: puede utilizarse en scripts de *Python*, entornos interactivos como Jupyter Notebook y aplicaciones con interfaces gráficas de usuario.
- **Personalización avanzada**: ofrece control total sobre los elementos del gráfico, incluyendo estilos de línea, colores, etiquetas, leyendas y más.

Estas capacidades hacen de *Matplotlib* una herramienta fundamental para científicos, ingenieros, analistas de datos y cualquier profesional que requiera representar visualmente datos de manera efectiva.

4.2.4. Scikit-learn

Scikit-learn es una biblioteca de aprendizaje automático de código abierto que admite el aprendizaje supervisado y no supervisado. También proporciona varias herramientas para el ajuste de modelos, el preprocesamiento de datos, la selección de modelos, la evaluación de modelos y muchas otras utilidades. (Scikit-learn Developers, nd)

- **Modelos de *machine learning***: incluye algoritmos clásicos como regresión logística, máquinas de soporte vectorial (SVM), árboles de decisión, k-NN, *random forests*, entre otros.
- ***Pipeline* de trabajo**: permite encadenar múltiples pasos (como preprocesamiento y modelado) mediante objetos de tipo `Pipeline`, promoviendo buenas prácticas y reproducibilidad.
- **Evaluación y selección de modelos**: incluye métricas estándar (*accuracy*, F1, ROC-AUC) y estrategias de validación cruzada para comparar modelos de forma fiable.
- **Preprocesamiento**: herramientas para escalado de datos, imputación de valores faltantes, codificación de variables categóricas, etc.
- **Interfaz coherente**: todos los modelos utilizan los métodos `fit()`, `predict()` y `score()`, lo que permite cambiar fácilmente de algoritmo sin reescribir el flujo de trabajo.

Scikit-learn es ampliamente adoptada en la industria y la academia por su simplicidad, documentación extensa y comunidad activa.

4.2.5. Seaborn

Seaborn es una biblioteca de visualización de datos en *Python*, construida sobre `matplotlib` y estrechamente integrada con `pandas`. Proporciona una interfaz de alto

nivel para crear gráficos estadísticos atractivos y bien estructurados con pocas líneas de código (Waskom y the Seaborn Development Team, 2025).

Su principal objetivo es facilitar la exploración y comprensión de datos mediante gráficos informativos y estéticamente agradables.

- **Gráficos estadísticos integrados**: incluye funciones para crear gráficos de dispersión, barras, cajas, violín, histogramas, mapas de calor, gráficos de regresión, entre otros.
- **Agrupación automática**: permite agregar y visualizar automáticamente estadísticas por categorías usando variables categóricas.
- **Soporte para *DataFrames* de *pandas***: facilita la visualización directa de datos tabulares, accediendo a columnas por nombre.
- **Estilos predefinidos y temas**: proporciona estilos visuales profesionales listos para usar, mejorando la estética por defecto de *matplotlib*.

Seaborn es ideal para análisis exploratorio de datos, ya que permite identificar relaciones, distribuciones y patrones visuales rápidamente.

4.2.6. Imbalanced-learn

Imbalanced-learn es una biblioteca de código abierto para *Python*, diseñada para abordar el problema de conjuntos de datos desbalanceados en tareas de aprendizaje automático. Se integra completamente con ***scikit-learn*** y proporciona una variedad de técnicas de remuestreo para equilibrar las clases en un conjunto de datos (The imbalanced-learn Developers, 2025).

- **Técnicas de sobremuestreo**: Métodos como ***RandomOverSampler***, ***SMOTE*** (Synthetic Minority Over-sampling Technique) y ***ADASYN*** generan nuevas muestras sintéticas para la clase minoritaria.
- **Técnicas de submuestreo**: Métodos como ***RandomUnderSampler***, ***ENN*** y `ClusterCentroids` eliminan muestras de la clase mayoritaria para equilibrar el conjunto de datos.
- **Combinación de técnicas**: Métodos como ***SMOTEENN*** y ***SMOTETomek*** combinan sobremuestreo y submuestreo para mejorar la calidad del conjunto de datos equilibrado.

Estas funcionalidades hacen de ***imbalanced-learn*** una herramienta esencial para mejorar el rendimiento de modelos de clasificación en conjuntos de datos con clases desbalanceadas.

4.2.7. Keras

Keras es la API de alto nivel de la plataforma *TensorFlow*, diseñada para proporcionar una interfaz accesible y altamente productiva para resolver problemas de

aprendizaje automático, con un enfoque en el aprendizaje profundo moderno. *Keras* cubre cada paso del flujo de trabajo de aprendizaje automático, desde el procesamiento de datos hasta el ajuste de hiperparámetros y el despliegue. Fue desarrollada con un enfoque en permitir una experimentación rápida (TensorFlow Developers, 2025).

- **Simplicidad y coherencia**: Ofrece interfaces simples y consistentes, minimizando el número de acciones requeridas para casos de uso comunes.
- **Mensajes de error claros**: Proporciona mensajes de error claros y accionables.
- **Divulgación progresiva de la complejidad**: Es fácil comenzar, y se pueden completar flujos de trabajo avanzados aprendiendo sobre la marcha.
- **Código conciso y legible**: Ayuda a escribir código conciso y legible.

Parte II

Desarrollo

5. Metodología

En este apartado se explicarán todos los pasos que se han realizado a la hora de obtener resultados. Se describirán detalladamente cada una de las etapas en las que se divide, desde que se obtienen los datos hasta la hora de obtener los resultados finales. Se acompañarán de diferentes bloques de código con su adecuada explicación.

Todo el código fuente desarrollado y los resultados obtenidos, que se detallan a lo largo de este trabajo, se encuentran disponibles para su descarga y consulta en el siguiente repositorio de GitHub: `https://github.com/albertodm11/DeteccionMalware_TFM.git`

5.1. Obtención de datos

Uno de los objetivos principales del trabajo era la de poder obtener una fuente de datos a partir de internet relacionada con *malware* debido al ámbito en el que se enfoca el trabajo que es la `ciberseguridad` y poder complementarlos mediante diferentes técnicas de análisis dinámico o estático de cada uno de los *malware* recolectados.

Para ello, el primer paso es obtener una serie de instancias de *malware* a partir de un repositorio de internet. Estos datos se descargan a través de `MalwareBazaar`, una plataforma de internet donde las personas pueden cargar archivos que pueden ser *malware* y también se encuentra formado por una base de datos de información que puede ser valiosa. Se pueden establecer alertas para realizar un seguimiento de los nuevos programas maliciosos observados, utilizar las API para enviar o recibir señales sin problemas y automatizar las consultas masivas (abuse.ch, nd).

El siguiente paso es acceder a la página y exportar los datos que se encuentren dentro de la base de datos. Se descargará un fichero `zip` y dentro estará disponible un fichero con la extensión `csv` dónde aparecerán diferentes instancias de *malware* subido por la comunidad. Se exportará el fichero y se trabajará con él.

Las instancias de *malware* con las que se han trabajado datan del 14 de noviembre de 2024 o días anteriores. Una vez se accede dentro del fichero se puede apreciar que está formado por un total de 14 columnas. La Tabla 5.1 muestra algunos ejemplos de muestra que se encuentran dentro del fichero `csv`.

- **Fecha (UTC)**: indica el momento exacto en que la muestra fue vista por primera vez en la plataforma, en formato `YYYY-MM-DD hh:mm:ss`. Es útil para estudios de tendencias temporales en la distribución de *malware*.
- **SHA256**: hash SHA-256 de la muestra. Se trata de una huella digital única de 256 bits que identifica inequívocamente el fichero. Es uno de los identificadores criptográficos más usados por su seguridad y baja tasa de colisiones.

- **MD5**: hash MD5 del fichero. Aunque es menos seguro que SHA256 debido a vulnerabilidades conocidas, aún se utiliza por compatibilidad en sistemas antiguos.
- **SHA1**: otra función hash criptográfica, intermedia entre MD5 y SHA256. Se encuentra en desuso progresivo, pero todavía presente en bases de datos antivirus o entornos heredados.
- **Reportero (reporter)**: usuario o sistema que subió o reportó el archivo. Si es `anonymous`, se desconoce el origen. En algunos casos puede ser una fuente confiable o una honeypot.
- **Nombre del archivo (file_name)**: nombre original con el que el fichero fue cargado, el cual puede revelar intentos de ingeniería social o campañas específicas (e.g., `factura.pdf.exe`).
- **Tipo de archivo (file_type_guess)**: tipo del archivo según la estructura interna (no solo por extensión). Por ejemplo: `exe`, `zip`, `js`.
- **MIME Type**: tipo MIME estándar del fichero (Multipurpose Internet Mail Extensions), útil para el manejo de archivos en entornos web y detección de extensiones engañosas.
- **Firma (signature)**: familia de *malware* o tipo detectado, cuando está disponible. Por ejemplo: `RemcosRAT`, `AsyncRAT`, `MassLogger`.
- **ClamAV**: resultado de análisis del antivirus ClamAV (si está disponible). Puede mostrar el nombre de la familia o indicar que no ha sido detectado (`n/a`).
- **VT % (vtpercent)**: porcentaje de motores antivirus en VirusTotal que detectaron la muestra como maliciosa. Es un valor numérico clave para evaluar la peligrosidad del archivo.
- **Imphash**: hash derivado de la tabla de importaciones de un ejecutable PE (Portable Executable). Útil para identificar variantes similares de *malware* con estructuras internas parecidas.
- **SSDEEP**: hash difuso (fuzzy hash) calculado con el algoritmo context triggered piecewise hashing (CTPH). Sirve para detectar similitud parcial entre archivos, incluso si no son idénticos.
- **TLSH**: otra técnica de fuzzy hashing: Trend Micro Locality Sensitive Hash. Más robusta que SSDEEP para ciertos usos de agrupación y clasificación de *malware* (Tridgell, 2009; Micro, nd).

Fecha (UTC)	SHA256	Archivo	Tipo	Firma	VT %	...
2024-11-14 12:37:21	0461e6...c90c	FileCoder.zip	zip	n/a	n/a	...
2024-11-14 12:36:18	0e248e...7c8d	Quotation.exe	exe	RemcosRAT	23.61	...
2024-11-14 12:35:53	f5bc7a...fe09	Ziraat Bankası...exe	exe	MassLogger	27.40	...
2024-11-14 12:35:29	390945...4cf9	9541327...98.js	js	n/a	17.46	...
2024-11-14 12:34:53	d6cb94...b1fb	dekont.pdf.exe	exe	n/a	33.33	...
2024-11-14 12:34:27	e1ad2d...cdb9	(İTOSAM)...exe	exe	AgentTesla	40.28	...
2024-11-14 12:25:17	d4394a...ac7e	A64734...52A.exe	exe	AsyncRAT	70.83	...
...						

Tabla 5.1. *Ejemplos de muestras que se encuentran en el fichero descargado.*

El fichero inicial se compone de un total de 834.049 filas con instancias de *malware* publicado por la comunidad. Para nuestro trabajo se reducirá el número de filas con el que trabajaremos. El siguiente paso, después de obtener información, es complementar la información de la que se compone el dataset con más información a través de otras diversas fuentes fiables.

Para el proyecto, a la información inicial se le ha añadido información proveniente de la API de virustotal, una página web que inspecciona los elementos introducidos con más de 70 escáneres antivirus y servicios de bloqueo de URL/dominios, además de un montón de herramientas para extraer señales del contenido estudiado (VirusTotal, 2024).

Para poder complementar la información, lo primero que se ha realizado ha sido iniciar sesión/registrarse en la página web oficial de virustotal. Una vez que la sesión se encuentra iniciada, la propia página proporciona una `API-KEY` para poder realizar peticiones. Se pueden realizar un máximo de 4 peticiones por minuto, 500 al día y 15.500 mensuales. Teniendo en cuenta estos valores, el proceso ha sido un poco lento debido a que no se ha podido acceder a la versión premium proporcionada a instituciones importantes.



Figura 5.1: Página inicial de virustotal

Para cada petición, es necesario introducir un valor correspondiente al *malware*. Esto puede ser el `SHA256`, `MD5` o el `SHA1`, valores únicos asociados al fichero que se quiere

evaluar. Además, para cada petición, se debe de añadir la `API-KEY`, correspondiente al usuario que desea realizar la petición. La página proporciona el código necesario para poder ejecutarlo en diferentes lenguajes como `shell`, `nodeJS`, `Ruby`, `PHP` o `Python`. Para poder ejecutar la petición en python, es necesario instalar el paquete *request*, que es muy popular a la hora de hacer peticiones HTTP (J2Logo, 2024).

```
1  url = "https://www.virustotal.com/api/v3/files/id"
2
3  headers = {
4      "accept": "application/json",
5      "x-apikey": "api-key"
6  }
7
8  response = requests.get(url, headers=headers)
9
10 print(response.text)
```

Extracto de código 5.1: Consulta a la API de VirusTotal

En el bloque de código 5.1 se puede ver el fragmento de código general a la hora de hacer las peticiones a la API de virustotal. Dónde se encuentra la variable `id` se establecen los diferentes `hashes` de los ficheros, mientras que donde aparece `api-key` se añade el `API-KEY` del usuario.

Como resultado de la petición, devuelve una respuesta en formato *JSON* con una serie de atributos y valores que nos pueden servir a la hora de complementar el dataset inicial. Proporciona una cantidad elevada de información estructurada sobre un archivo enviado, centrada en su análisis antivirus, comportamiento en entornos de ejecución controlados (sandbox), reputación comunitaria y otros metadatos relevantes.

Entre los datos más destacados se encuentran:

- **Datos de análisis antivirus**: la API incluye los resultados del análisis del archivo por parte de múltiples motores antivirus. Se proporciona tanto el resultado detallado por motor como un resumen estadístico que indica cuántos motores lo consideraron malicioso, inofensivo, sospechoso, etc. Esto ayuda a confirmar la naturaleza del archivo y a minimizar falsos positivos.
- **Resultados de análisis en sandbox**: cuando el archivo ha sido ejecutado en entornos controlados, VirusTotal recopila los resultados de estos análisis dinámicos. Se ofrecen veredictos, confianza, categorías de comportamiento (por ejemplo, sospechoso, malicioso) y posibles familias de *malware* asociadas. La ejecución en un entorno controlado ofrece acciones sospechosas y la interacción real del archivo con el sistema, indicando su verdadera intención.
- **Reputación y votaciones**: se muestra la puntuación basada en votos de la comunidad de usuarios de VirusTotal, indicando si el archivo es considerado seguro o peligroso por los usuarios. La opinión de profesionales y usuarios puede alertar sobre amenazas emergentes o reafirmar la peligrosidad de un archivo.
- **Información estructural y hashes**: se incluyen identificadores como MD5, SHA1, SHA256, vhash, permhash o tlsh, útiles para la identificación unívoca y

comparación de archivos. También se informa sobre el tamaño del archivo, su extensión, tipo MIME y otros atributos técnicos.

- **Etiquetas y nombres**: se listan todos los nombres con los que el archivo ha sido detectado, así como etiquetas descriptivas o características extraídas de su análisis (por ejemplo, si es ejecutable, si tiene comportamiento sospechoso, etc.). Las etiquetas y nombres permiten entender de un vistazo sus características y posibles implicaciones.
- **Datos temporales**: la API devuelve fechas relevantes como el momento de la primera y última subida, última modificación y fecha del análisis más reciente, todas en formato timestamp UTC. Permiten rastrear la antigüedad del archivo y su historial de detección, lo cual es útil para evaluar la relevancia y la novedad de una amenaza.
- **Resultados de inteligencia colectiva**: se pueden incluir coincidencias con reglas Sigma, reglas YARA de la comunidad, análisis automáticos con IA, y relaciones del archivo con otros objetos (por ejemplo, archivos similares, archivos relacionados, archivos embebidos, etc.).
- **Indicadores de amenazas**: también se incluyen indicadores booleanos sobre si el archivo está vinculado a amenazas conocidas, vulnerabilidades (CVEs), actores maliciosos, o colecciones sospechosas.

Enfocado a nuestro objetivo, los atributos que se han obtenido de la petición son:

- **Sandbox verdicts (`sandbox_verdicts`)**: se recogen los resultados generados por entornos de ejecución controlados. Para cada sandbox que haya emitido un veredicto se extraen los siguientes subatributos: `category` (categoría del veredicto), `malware_classification` (clasificación de *malware*), `malware_names` (familias de *malware* detectadas) y `sandbox_name` (nombre del entorno sandbox).
- **Votaciones de la comunidad (`total_votes`)**: se obtienen las cantidades de votos positivos y negativos emitidos por usuarios de la comunidad, a través de los campos `harmless` y `malicious`.
- **Resumen del análisis antivirus (`last_analysis_stats`)**: se extrae un resumen estadístico que incluye el número de motores que clasificaron el archivo en cada una de las siguientes categorías: `malicious`, `suspicious`, `undetected`, `harmless`, `timeout`, `confirmed-timeout`, `failure` y `type-unsupported`.
- **Etiquetas del archivo (`tags`)**: se recupera la lista de etiquetas que describen características relevantes del archivo, las cuales se presentan concatenadas para su inclusión en el fichero de salida.
- **Contexto de IDS (`crowdsourced_ids_results.alert_context`)**: se incorporan datos de tráfico de red extraídos de reglas IDS colaborativas, incluyendo direcciones IP y puertos de origen y destino (`src_ip`, `src_port`, `dest_ip`, `dest_port`), así como el nombre del host y la URL implicada en la alerta.

Esta información es obtenida para cada instancia de *malware* que se encuentra dentro del dataset inicial. Todas estas peticiones no se han realizado y se han ido aña-

diendo manualmente. Por ello, se ha creado un *script* en *Python* que permita realizar el trabajo de una manera automatizada. El nuevo dataset estará formado por la información inicial más la información que ha sido añadida y que ha resultado de un carácter importante. La nueva cabecera sería:

- `first_seen_utc`
- `sha256_hash`, `md5_hash`, `sha1_hash`
- `reporter`, `file_name`, `file_type_guess`, `mime_type`, `signature`
- `clamav`, `vtpercent`, `imphash`, `ssdeep`, `tlsh`
- `sandbox_names`, `sandbox_categories`, `sandbox_malware_classifications`, `sandbox_malware_names`
- `total_votes_harmless`, `total_votes_malicious`
- `last_analysis_malicious`, `last_analysis_suspicious`, `last_analysis_undetected`, `last_analysis_harmless`
- `last_analysis_timeout`, `last_analysis_confirmed_timeout`, `last_analysis_failure`, `last_analysis_type_unsupported`
- `tags`
- `src_ip`, `src_port`, `hostname`, `url`, `dest_ip`, `dest_port`

```
url = f"https://www.virustotal.com/api/v3/files/{sha256_hash}"
headers = {
    "accept": "application/json",
    "x-apikey": "TU_API_KEY"
}

response = requests.get(url, headers=headers)
data = response.json()
attributes = data.get("data", {}).get("attributes", {})

sandbox_verdicts = attributes.get("sandbox_verdicts", {})
total_votes = attributes.get("total_votes", {})
last_analysis_stats = attributes.get("last_analysis_stats", {})
tags = attributes.get("tags", [])
```

Extracto de código 5.2: Consulta a la API de VirusTotal y extracción de atributos clave

Toda esta información se debe de guardar en un fichero `csv` para poder trabajar con él posteriormente. Para ello, el fichero de entrada es el descargado de la página de `MalwareBazaar` y el fichero de salida se le ha asignado el nombre `malwarebazaar_processed.csv` . Las tablas 5.2,5.3, 5.4 muestran 5 muestras de los datos obtenidos a partir de la búsqueda en la API de virustotal. También la Figura 5.2 muestra los pasos que se han llevado a cabo para completar el dataset final.

Tabla 5.2. *Información general de archivos analizados con VirusTotal*

Fecha	SHA256 (resumido)	MD5	SHA1	Archivo	Tipo	Firma
2024-11-14 12:37:21	0461e6...c90c	be8d17...c337	902f12...1fd	FileCoder.zip	zip	n/a
2024-11-14 12:36:18	0e248e...7c8d	9ddd4a...732	08c58c...fb3	Quotation.exe	exe	RemcosRAT
2024-11-14 12:35:53	f5bc7a...fe09	82471b...55e	bdcb15...6db	Ziraat Bankası...exe	exe	MassLogger
2024-11-14 12:35:29	390945...4cf9	37f712...80b	449e6e...1aa	9541327...98.js	js	n/a
2024-11-14 12:34:53	d6cb94...b1fb	e50308...f397	af2a32...33c	dekont.pdf.exe	exe	n/a

Tabla 5.3. *Estadísticas de análisis y comportamiento en sandbox*

SHA256 (resumido)	VT %	Malicious	Suspicious	Undetected	Tags	Sandboxes
0461e6...c90c	n/a	56	0	13	contains-pe, zip	—
0e248e...7c8d	23.61	57	0	15	detect-debug, macro, 64bits...	CAPE, Zenbox, Dr.Web, Yomi, C2AE
f5bc7a...fe09	27.40	57	0	15	spreader, persistence, overlay...	Zenbox, Yomi, C2AE, Dr.Web
390945...4cf9	17.46	24	0	38	macro-powershell, text, obfuscated	Zenbox
d6cb94...b1fb	33.33	57	0	15	spreader, peexe, calls-wmi...	CAPE, Zenbox, Yomi, C2AE

Tabla 5.4. *Información de red asociada al archivo (IPs y puertos)*

SHA256 (resumido)	IPs destino	Puertos destino
0461e6...c90c	—	—
0e248e...7c8d	23.95.60.82	30908
f5bc7a...fe09	132.226.247.73, 158.101.44.242, ...	80, 443, 53, ...
390945...4cf9	—	—
d6cb94...b1fb	8.8.8.8, 8.8.8.8	53, 53

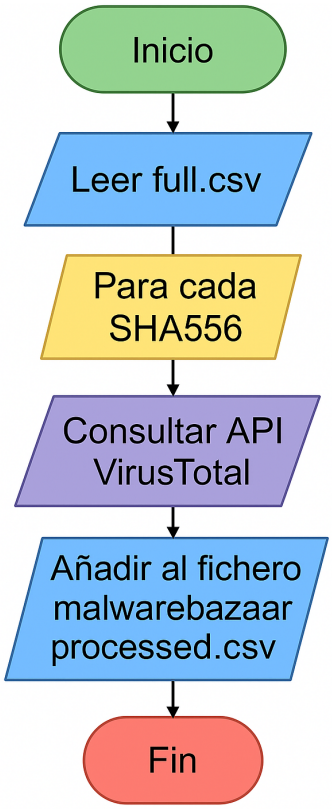


Figura 5.2: Flujo a la hora de crear el dataset

5.2. Preprocesamiento de los datos

Una vez obtenidos los datos, el siguiente paso será el de preprocesar los datos. Esto quiere decir que debemos de limpiar los valores repetidos de *malware* en el caso de que existan, así como generar nuevos atributos a partir de los datos existentes o eliminar atributos que no sirvan para nada y no aporten valor.

Para la realización de todo el código en *Python* se ha utilizado `JupyterNotebook` de `AnacondaNavigator`, que es una aplicación de creación de cuadernos simplificada y forma parte del Proyecto Jupyter, un gran proyecto paraguas centrado en el objetivo de proporcionar herramientas (y estándares) para la computación interactiva con cuadernos computacionales (Project Jupyter, 2024).

5.2.1. Inclusión de datos complementarios

Aunque el uso de herramientas externas como antivirus y sandbox pueda parecer redundante, su inclusión en el modelo aporta beneficios clave:

1. Categorías de sandbox como `file drop`, `evasion`, `registry changes`, etc., ayudan a identificar patrones.

2. Votos de la comunidad y el número de motores que marcan un archivo como malicioso (`last_analysis_*`) aportan una medida que puede ser de utilidad a la hora de predecir si es *malware* o no.

   Estos datos se han obtenido a partir de las consultas en la API de VirusTotal.

5.2.2. Importación del dataset

El primer paso será realizar la importación del fichero `csv` al *notebook* de `Jupyter`. Lo único que se debe de hacer es utilizar la función `read_csv` de la librería `Pandas` e introducir la ruta del fichero.

5.2.3. Eliminación de valores nulos

El preprocesamiento de los datos comienza con el objetivo de suprimir las filas en las que el atributo `vtpercent` se encuentre nulo o vacío debido a que es la variable objetivo que se quiere predecir. También se eliminarán las filas que se encuentren duplicadas. Esto se realizará haciendo uso de diferente funciones de `Python` como son `drop_duplicates` y `dropna`. En el código 5.3 se puede ver el uso de las diferentes funciones.

```
# Borrar duplicados
df = df.drop_duplicates()

# Borrar nulos
df = df.dropna(subset=["vtpercent"]).reset_index(drop=True)
```

Extracto de código 5.3: Eliminación de duplicados y valores nulos

5.2.4. Eliminación de columnas

A partir de las columnas iniciales, después de haber sido completadas con las peticiones a la API de virustotal, se han eliminado ciertas columnas como son:

- **Fecha de subida del archivo**:
  - `first_seen_utc`: la mayoría de los archivos tienen un periodo de subida de 2-3 días. No aporta valor para entrenar modelos, ya que en el caso de que se quiera predecir el porcentaje de virus de un fichero, esta información no tendría relación con el resultado.
- **Hashes de los archivos**:
  - `sha256_hash`, `md5_hash`, `sha1_hash`: son identificadores únicos para cada archivo. Al predecir el porcentaje de virus, un nuevo fichero, estos valores serán distintos, lo que impide extraer patrones útiles.
- **Datos irrelevantes o redundantes**:

- `reporter`: la persona que sube el archivo no aporta información relevante para la detección de *malware* debido a que el reportador no es una persona especializada, puede ser una persona sin conocimientos.
- `file_type_guess`: está relacionado con la extensión del archivo y, por tanto, es redundante.
- `clamav`: contiene únicamente valores nulos, por lo que no aporta información útil.
- `mime_type`: se encuentra relacionada con la extensión del fichero.
- `file_name`: el nombre del fichero, en la mayor de las probabilidades, suele tener nombre distinto. Se han obtenido características a partir de esta columna.

- **Firmas y características de similitud**:
  - `imphash`, `ssdeep`, `tlsh`: funcionan de forma similar a los hashes, y no permiten clsificar patrones relevantes para la predicción.
- **Direcciones IP, nombres de host y URLs**:
  - `src_ip`, `dest_ip`, `hostname`, `url`: se han descartado porque existen miles de valores únicos.

5.2.5. Ingeniería de características y preprocesamiento avanzado

Ingeniería de características - *Feature Engineering* - es el proceso de transformación de datos brutos en información relevante para su uso por modelos de aprendizaje automático. En otras palabras, la ingeniería de características es el proceso de creación de características de modelos predictivos (IBM, nd). Además, se incluyen tareas de limpieza, transformación y categorización de los datos, con el objetivo de enriquecer el *dataset* y adaptarlo a los distintos modelos de aprendizaje automático.

En nuestro proyecto, la ingeniería de características se ha utilizado a la hora de añadir nuevas columnas al dataset a partir de los datos iniciales. Las columnas que se han añadido son:

- **Sandbox features**
  - `has_sandbox_classification`: valor binario que indica si el archivo fue clasificado dinámicamente por un sandbox.
  - `sandbox_contains_*`: indicadores derivados que detectan si la clasificación incluye nombres específicos de *malware*.
- **Votos y análisis de antivirus**
  - `vote_ratio_malicious`: proporción de votos maliciosos sobre el total de votos (maliciosos + benignos + 1).

- `malicious_score`: puntuación calculada como:
  `malicious + 0,5 × suspicious - 0,25 × harmless`.

- **Red: *IP*, *Hostname* y URL**
  - `has_ip`: indica si el archivo contiene direcciones IP.
  - `has_hostname`: indica si dispone de nombre de host.
  - `has_url`: indica si se caracteriza una URL.
- **Tipo de archivo (MIME)**
  - `is_executable`: indica si el archivo es potencialmente ejecutable.
- **Nombre del archivo**
  - `extension`: extensión del archivo.
  - `double_extension`: indicador binario de doble extensión (ej. `.pdf.exe`).
  - `suspicious_name`: valor binario si el nombre contiene palabras asociadas a *malware*.
  - `file_name_length`: longitud del nombre del archivo.
  - `file_has_numbers`: indica si el nombre contiene números.
  - `file_special_chars`: indica si el nombre contiene caracteres especiales.
  - `is_malicious_extension`: indica si la extensión del archivo es maliciosa.
  - `is_compressed_file`: indica si la extensión corresponde a un archivo comprimido.
- **Otras características derivadas**
  - `firma_comportamiento`: indica si el archivo tiene firma digital y comportamiento malicioso.
  - `nombre_ext_sospechoso`: combinación de nombre largo y extensión maliciosa.
  - `total_tags`: suma de todas las columnas `tag_`.
  - `ext_mod_combo`: extensión maliciosa + modificación del sistema.
- **Procesamiento y clasificación de la columna *tags***
  - Se verifica si la columna `tags` está presente y se rellenan los valores nulos con una cadena vacía.
  - Se extraen todas las etiquetas del dataset, dividiendo cada fila por el delimitador "|".
  - Se identifican los 50 tags más frecuentes y se almacenan en el conjunto `common_tags` para uso posterior.
  - Se crea la columna `num_tags`, que representa la cantidad total de etiquetas asociadas a cada archivo.

- Se define un diccionario `tag_categories` que agrupa etiquetas comunes en distintas categorías semánticas:
  - `malware_behavior`, `evasion`, `exploits_vulnerabilities`, `file_types`, `system_modification`, `corruption`, `network_dns`, y `other`.
- Para cada categoría, se crea una nueva columna `tag_nombre_categoría` que contiene el número de etiquetas que pertenecen a dicha categoría en cada archivo.
- Finalmente, se elimina la columna original `tags` tras extraer toda su información relevante.

- **Selección del Puerto Más Frecuente en `src_port` y `dest_port`**
  - En algunos registros, los campos `src_port` y `dest_port` contienen múltiples valores separados por el carácter "|".
  - Se define una función `select_most_common_port` que realiza los siguientes pasos:
    - Convierte el valor a cadena de texto si no lo es.
    - Divide los valores múltiples usando `split("|")`.
    - Si hay más de un valor, utiliza `Counter` para contar las ocurrencias de cada puerto.
    - Devuelve el puerto más común de la lista. Si no hay puertos, devuelve una cadena vacía.
  - Esta función se aplica a las columnas `src_port` y `dest_port` del `DataFrame` para limpiar los datos y conservar únicamente el puerto más representativo por fila.

5.2.6. Resumen de columnas añadidas y eliminadas

Columnas Añadidas	**Columnas Eliminadas**
has_sandbox_classification	sha256_hash
sandbox_contains_*	md5_hash
vote_ratio_malicious	sha1_hash
malicious_score	reporter
has_ip	file_type_guess
has_hostname	clamav
has_url	mime_type
is_executable	file_name
extension	tags
double_extension	vtpercent
suspicious_name	imphash
file_name_length	ssdeep
file_has_numbers	tlsh

file_special_chars	src_ip
is_malicious_extension	dest_ip
is_compressed_file	hostname
firma_comportamiento	url
nombre_ext_sospechoso	
total_tags	
ext_mod_combo	
tag_malware_behavior	
tag_evasion	
tag_exploits_vulnerabilities	
tag_file_types	
tag_system_modification	
tag_corruption	
tag_network_dns	
tag_other	
num_tags	
src_port (procesado)	
dest_port (procesado)	

Tabla 5.5. *Resumen de columnas añadidas y eliminadas*

5.2.7. Clasificación variable objetivo

La variable objetivo del problema es el porcentaje de virus (`vtpercent`), un valor numérico del 0 al 100. Se ha adaptado el trabajo a un problema de clasificación en la que la finalidad es clasificar a un fichero si el riesgo es bajo, medio o alto. Para ello, se ha tenido que transformar la variable `vtpercent` en 5 categorías:

- `Sin riesgo`: valores menores o iguales a 0,99.
- `Riesgo bajo`: valores entre 1 y 25,99.
- `Riesgo medio`: valores entre 26 y 74,99.
- `Riesgo alto`: valores entre 75 y 99,99.
- *`Malware`*: exactamente 100.
- `Desconocido`: cualquier otro valor fuera de rango.

```
def categorize_vtpercent(value):
    if value <= 0.99:
        return "Sin riesgo"
    elif 1 <= value <= 25.99:
        return "Riesgo bajo"
    elif 26 <= value <= 74.99:
        return "Riesgo medio"
    elif 75 <= value <= 99.99:
        return "Riesgo alto"
    elif value == 100:
        return "Malware"
    else:
```

```
        return "Desconocido"

df["vt_category"] = df["vtpercent"].apply(categorize_vtpercent)

print(df["vt_category"].value_counts())

df.drop(columns=["vtpercent"], inplace=True)
```

Extracto de código 5.4: Categorización de vtpercent en niveles de riesgo

5.2.8. Estudio exploratorio de los datos

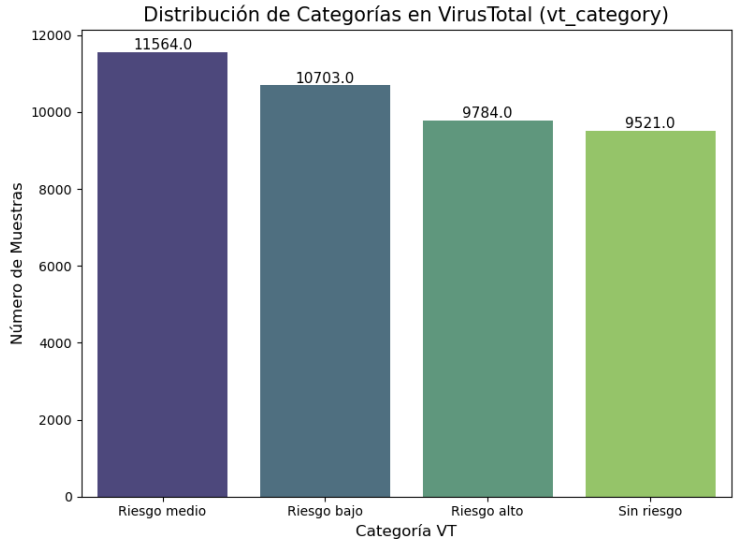


Figura 5.3: Categoríasobtenidas a partir de VirusTotal

La Figura superior dibuja un gráfico de barras verticales titulado “Distribución de Categorías en VirusTotal (vt_category)”. El eje horizontal, denominado “Categoría VT”, muestra las cuatro categorías clasificadas anteriormente: “Riesgo medio”, “Riesgo bajo”, “Riesgo alto” y “Sin riesgo”. El eje vertical, etiquetado como “Número de Muestras”, cuantifica la frecuencia de cada categoría, con una escala que se extiende desde 0 hasta 12.000.

- La categoría “Riesgo medio”, representada por una barra de color morado oscuro, es la que cuenta con mayor número de instancias, alcanzando las 11.564 muestras.
- A esta le sigue la categoría “Riesgo bajo”, con 10.703 muestras, visualizada con una barra de color azul grisáceo.
- La categoría “Riesgo alto” está formada por 9.784 muestras, mostrada con una barra de color verde azulado.
- Por último, la categoría “Sin riesgo”, con 9.521 muestras, es la menos frecuente y se representa con una barra de color verde claro.

La distribución de las muestras entre estas cuatro categorías de riesgo es notablemente equilibrada, por lo tanto, el rebalanceo de clases que se deberá de realizar no será a gran escala.

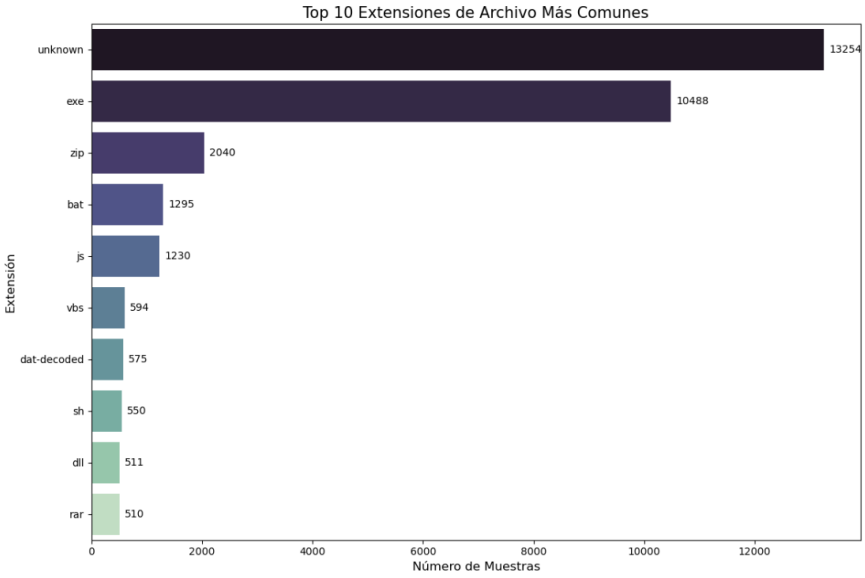


Figura 5.4: Extensiones de archivos más comunes

El gráfico de barras horizontales presentado ilustra las "Top 10 Extensiones de Archivo Más Comunes" dentro del conjunto de datos analizado. El eje vertical, denominado "Extensión", muestra las diez extensiones de archivo más frecuentes, ordenadas de mayor a menor frecuencia de aparición. El eje horizontal, "Número de Muestras", representa el conteo absoluto de archivos para cada una de estas extensiones.

Se observan dos categorías principalmente:

- La categoría "unknown", que representa archivos cuya extensión no pudo ser determinada o no estaba presente, es la más numerosa con 13.254 muestras.
- La extensión ".exe" (ejecutables de Windows) es la segunda más común.

Las siguientes extensiones más frecuentes, aunque con un número de muestras considerablemente menor, incluyen:

- ".zip" (archivos comprimidos) con 2.040 muestras.
- ".bat" (archivos de procesamiento por lotes de Windows) con 1.295 muestras.
- ".js" (archivos JavaScript) con 1.230 muestras.
- ".vbs" (archivos VBScript) con 594 muestras.
- "dat-decoded" (posiblemente archivos .dat que han sido decodificados o renombrados) con 575 muestras.
- ".sh" (scripts de shell) con 550 muestras.
- ".dll" (bibliotecas de enlace dinámico) con 511 muestras.

- ".rar" (archivos comprimidos) con 510 muestras.

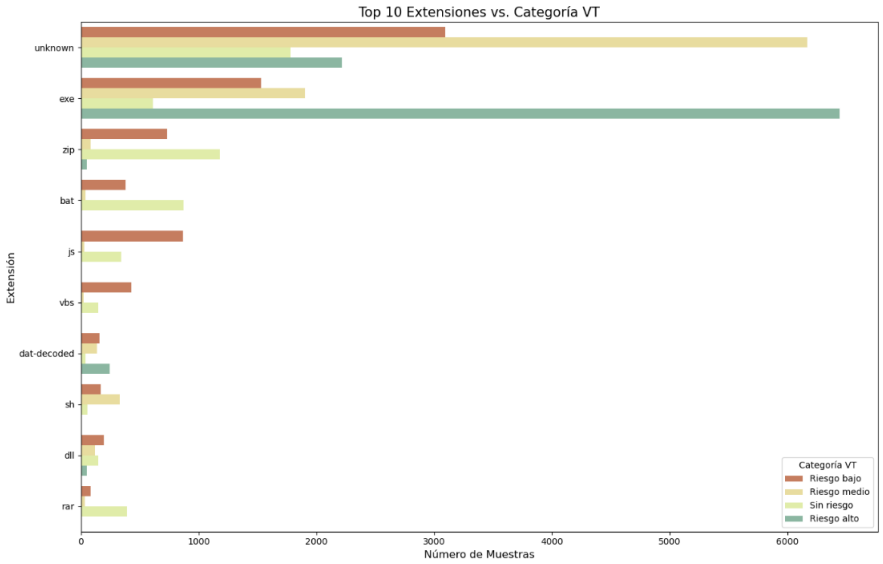


Figura 5.5: Comparativa extensiones existentes con la clasificación de *malware*

Este gráfico de barras horizontales, titulado "Top 10 Extensiones vs. Categoría VT", profundiza en la relación entre las diez extensiones de archivo más frecuentes y su clasificación de riesgo, según VirusTotal. El eje vertical lista las mismas diez extensiones identificadas previamente: "unknown", "exe", "zip", "bat", "js", "vbs", "dat-decoded", "sh", "dll" y "rar". El eje horizontal representa el "Número de Muestras". Para cada extensión, se muestran barras de diferentes colores que indican la cantidad de muestras pertenecientes a cada una de las cuatro categorías de riesgo, según la leyenda proporcionada: "Riesgo bajo" (color marrón), "Riesgo medio" (color beige), "Sin riesgo" (color amarillo) y "Riesgo alto" (color verde).

Se concluye que:

- Para los archivos con extensión "unknown", la categoría predominante es "Riesgo medio", seguida de "Riesgo bajo" y "Sin riesgo", con una menor proporción de "Riesgo alto". Esto nos indica que una gran cantidad de archivos sin extensión identificada son benignos.
- Los archivos con extensión ".exe" muestran una alta concentración en la categoría "Riesgo alto". También presentan una cantidad notable en "Riesgo bajo" y "Riesgo medio", pero muy pocas instancias "Sin riesgo".
- Las extensiones de archivos comprimidos como ".zip" y ".rar" se asocian mayoritariamente con la categoría "Sin riesgo", aunque ".zip" también presenta algunas instancias en "Riesgo bajo".
- Archivos de script como ".bat", ".js" y ".vbs" tienden a concentrarse en las categorías "Riesgo bajo" y "Sin riesgo", aunque ".js" y ".bat" también muestran una presencia de "Riesgo medio". La extensión ".vbs" está casi exclusivamente en "Riesgo bajo".

- La categoría “dat-decoded” muestra una distribución interesante, con una proporción considerable en “Riesgo alto” y “Riesgo bajo”.
- Los scripts de shell (“.sh”) se dividen principalmente entre “Riesgo bajo” y “Sin riesgo”.
- Las bibliotecas de enlace dinámico (“.dll”) presentan muestras en “Riesgo bajo”, “Riesgo alto” y “Sin riesgo”, destacando su potencial uso tanto legítimo como malicioso.

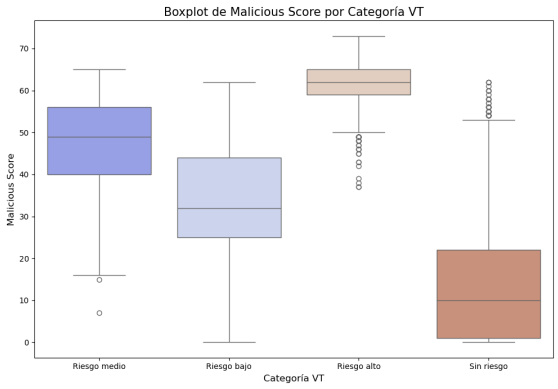


Figura 5.6: Disposición malicious *score* con categoría de *malware*

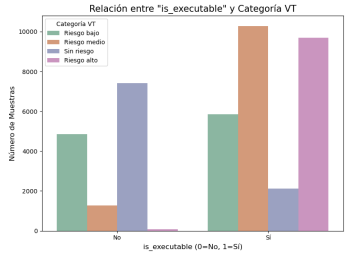
La figura presenta un diagrama de cajas (boxplot) titulado “Boxplot de Malicious Score por Categoría VT”, que complementa la visualización anterior del histograma. El eje horizontal, “Categoría VT”, muestra las cuatro categorías de riesgo: “Riesgo medio”, “Riesgo bajo”, “Riesgo alto” y “Sin riesgo”. El eje vertical representa el “Malicious Score”, con una escala que va aproximadamente de 0 a más de 70. Cada diagrama de caja resume la distribución del `Malicious Score` para la categoría correspondiente, mostrando la mediana (línea dentro de la caja), los cuartiles primero (Q1) y tercero (Q3) (bordes inferior y superior de la caja, respectivamente), y los bigotes que se extienden hasta los valores normales. Los puntos individuales representan valores atípicos (*outliers*).

A partir de los diagramas de caja, se pueden extraer las siguientes conclusiones:

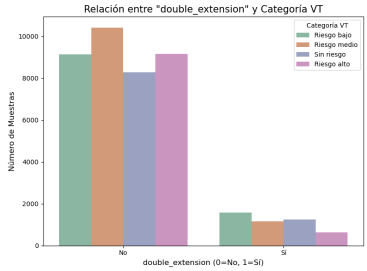
- La categoría “Sin riesgo” (caja de color marrón claro) muestra la distribución de *Malicious Score* más baja. La caja es muy compacta y se sitúa en la parte inferior de la escala, con una mediana cercana a 0. La mayoría de los datos se concentran en puntuaciones bajas, aunque se observan algunos *outliers* con scores más altos.
- La categoría “Riesgo alto” (caja de color beige) muestra la distribución de *Malicious Score* más elevada. La mediana es significativamente alta, y el rango intercuartílico se encuentra en la parte superior de la escala. Se aprecian algunos valores atípicos en la parte inferior, indicando que unas pocas muestras clasificadas como de alto riesgo recibieron puntuaciones bajas.
- Las categorías “Riesgo medio” (caja de color azul oscuro) y “Riesgo bajo” (caja de color azul claro) se sitúan en posiciones intermedias.

    - "Riesgo medio" tiene una mediana y una altura de la caja considerables, con puntuaciones que se extienden desde aproximadamente 40 hasta 57 en su rango intercuartílico, y con algunos *outliers* en la parte inferior.
    - "Riesgo bajo" presenta una mediana inferior a la de "Riesgo medio", pero con una dispersión también amplia. La caja se extiende desde aproximadamente 25 hasta 45.
- Existe un solapamiento notable entre los rangos intercuartílicos de "Riesgo medio" y "Riesgo bajo", así como entre "Riesgo bajo" y algunos valores de "Riesgo alto" (especialmente considerando los bigotes y outliers). Sin embargo, la distinción entre "Sin riesgo" y las demás categorías, y entre "Riesgo alto" y las demás, es bastante clara en términos de sus medianas y rangos principales.

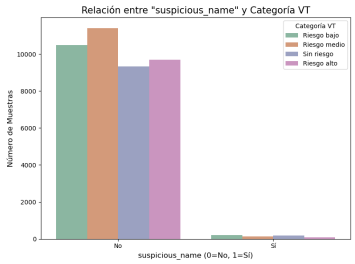
Este diagrama de cajas confirma la utilidad del `Malicious Score` para discriminar entre las diferentes categorías de riesgo, especialmente para identificar las muestras "Sin riesgo" y "Riesgo alto". Con este gráfico, también se puede llegar a la conclusión de que las clasificaciones de "Riesgo medio" y "Riesgo bajo" son muy equivalentes y la distinción entre ellas puede dar lugar a problemas.



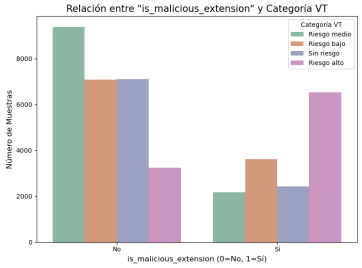
(a) `is_executable`



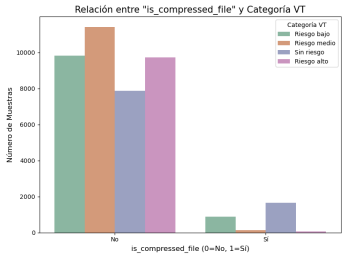
(b) `double_extension`



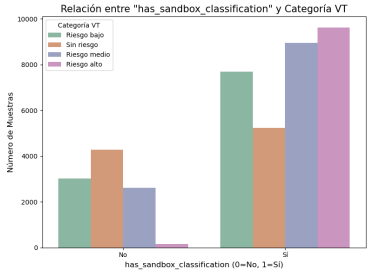
(c) `suspicious_name`



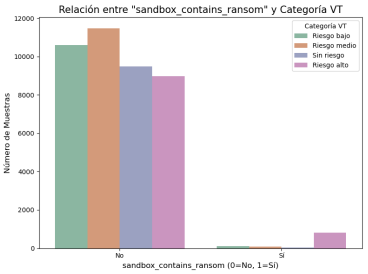
(d) `is_malicious_extension`



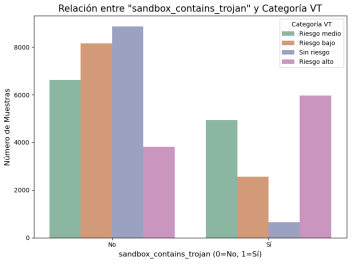
(e) `is_compressed_file`



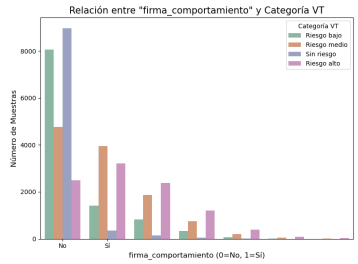
(f) `has_sandbox_classification`



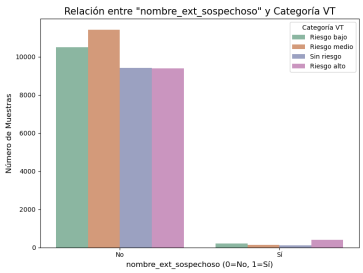
(g) `sandbox_contains_ransom`



(h) `sandbox_contains_trojan`



(i) `firma_comportamiento`



(j) `nombre_ext_sospechoso`

Figura 5.7: Relación entre diversas características binarias y la Categoría VT. Cada gráfico ilustra la distribución del número de muestras según la presencia o ausencia de la característica elegida, clasificada por categoría de riesgo.

La serie de diez gráficos de barras (Figura 5.7) agrupadas establece la relación entre diversas características binarias y la clasificación de riesgo de las muestras según la variable `vt_category`. En cada gráfico, el eje horizontal representa la ausencia (etiquetado como "No" o 0) o presencia (etiquetado como "Sí" o 1) de la característica. El eje vertical cuantifica el "Número de Muestras". Dentro de cada categoría del eje horizontal ("No" o "Sí"), las barras agrupadas por color corresponden a las cuatro categorías de riesgo: "Riesgo bajo", "Riesgo medio", "Sin riesgo" y "Riesgo alto".

Tras realizar un análisis general se puede concluir que:

- **Indicadores fuertes de alto riesgo:** la presencia de características como ser un archivo ejecutable (`is_executable`), tener una extensión considerada maliciosa (`is_malicious_extension`), o la detección por sandbox de ransomware (`sandbox_contains_ransom`) o troyanos (`sandbox_contains_trojan`) está fuertemente asociada con las categorías "Riesgo alto" y "Riesgo medio". Por ejemplo, en el gráfico de `is_executable`, la gran mayoría de las muestras no ejecutables se clasifican como "Sin riesgo" o "Riesgo bajo", mientras que las muestras ejecutables predominan en las categorías "Riesgo medio" y "Riesgo alto". De manera similar, ocurre con las demás gráficas mencionadas.

- **Indicadores de bajo riesgo:** la característica `is_compressed_file` muestra una tendencia clara: la mayoría de los archivos comprimidos (valor "Sí") se asocian con la categoría "Sin riesgo".

- **Impacto de la clasificación por sandbox y firmas de comportamiento:** la existencia de una clasificación por sandbox (`has_sandbox_classification`) o la presencia de una firma de comportamiento (`firma_comportamiento`) se asocia con una mayor proporción de archivos clasificados en categorías de riesgo elevadas, en comparación con aquellos que no presentan estas características. Específicamente, cuando `has_sandbox_classification` es "Sí", se observa un aumento en las categorías "Riesgo alto" y "Sin riesgo" (esta última posiblemente debido a que el sandbox confirma que no es *malware*).

- **Características con menor número de instancias positivas:** algunas características como `double_extension`, `suspicious_name`, y `nombre_ext_sospechoso` presentan un número de instancias "Sí" mucho menor que las instancias "No".

    - Para `suspicious_name`, se puede ver que hay muy pocas instancias de un nombre sospechoso (valor "Sí") que da lugar a que la definición de este atributo no es de gran utilidad a la hora de predecir de *malware* en este conjunto de datos.

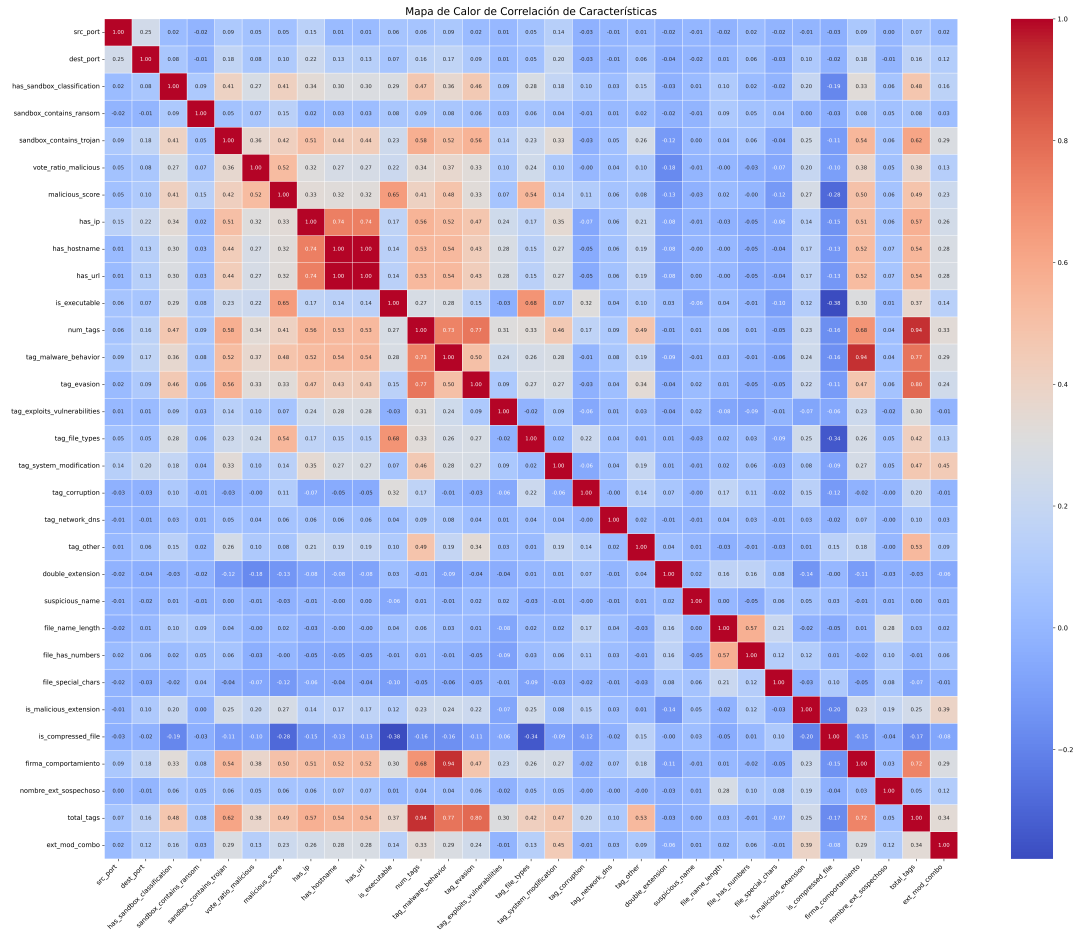


Figura 5.8: Mapa correlación variables

La Figura 5.8 presenta un mapa de calor que visualiza la matriz de correlación de Pearson entre un conjunto seleccionado de características numéricas y binarias del dataset. Cada celda de la matriz representa el coeficiente de correlación entre dos variables, donde la intensidad y el color de la celda indican la fuerza y la dirección de la correlación. La escala de colores, mostrada a la derecha, varía desde el azul intenso (correlación negativa fuerte, cercana a -1) hasta el rojo intenso (correlación positiva fuerte, cercana a +1), con colores pálidos o blancos indicando una correlación débil o nula (cercana a 0). Las variables se listan tanto en el eje horizontal como en el vertical.

El análisis de este mapa de calor permite identificar varias relaciones lineales entre las características:

- **Correlaciones positivas fuertes (valores rojos intensos):**
  - Se observa una correlación positiva muy fuerte entre `num_tags` y `total_tags` (valor de 0,94), lo cual es esperado si `total_tags` es una suma o una medida muy relacionada con `num_tags`.
  - `malicious_score` muestra correlaciones positivas moderadas a fuertes con varias características indicativas de malicia, como `vote_ratio_malicious` (0,52), `tag_malware_behavior` (0,47), `firma_comportamiento` (0,50), `is_-`

`malicious_extension` (0,25), y la presencia de sandbox (`has_sandbox_classification`, 0,41). También con `num_tags` (0,58) y `total_tags` (0,57).

- `vote_ratio_malicious` también se correlaciona positivamente con características similares a `malicious_score`, como `tag_malware_behavior` (0,33), `firma_comportamiento` (0,38), y `num_tags` (0,34).
- Las variables relacionadas con la detección en sandbox, como `has_sandbox_classification`, `sandbox_contains_ransom`, y `sandbox_contains_trojan`, muestran correlaciones positivas entre sí, aunque no extremadamente altas (ej. `has_sandbox_classification` y `sandbox_contains_trojan` con 0,42).
- Varias variables de tipo 'tag' (ej. `tag_malware_behavior`, `tag_evasion`, `tag_system_modification`) muestran correlaciones positivas moderadas entre sí y con `num_tags` y `total_tags`, lo cual es lógico si estas tags contribuyen al conteo total. Por ejemplo, `tag_malware_behavior` y `tag_evasion` (0,43).
- `has_ip`, `has_hostname`, y `has_url` muestran correlaciones positivas entre ellas (ej. `has_ip` y `has_hostname` con 0,32, `has_ip` y `has_url` con 0,34).

- **Correlaciones negativas (valores azules):**
  - Se observan algunas correlaciones negativas, aunque generalmente de magnitud débil a moderada. Por ejemplo, `is_compressed_file` tiene una correlación negativa con `is_executable` (-0,38), lo que sugiere que los archivos comprimidos tienden a no ser ejecutables y viceversa.
  - `is_compressed_file` también muestra correlaciones negativas con `malicious_score` (-0,13), `vote_ratio_malicious` (-0,18) y varias tags de *malware*, lo que podría indicar que los archivos comprimidos por sí mismos tienden a tener scores de malicia más bajos.
- **Correlaciones débiles (valores cercanos a 0, colores pálidos):**
  - Muchas pares de variables muestran correlaciones muy débiles, indicando poca o ninguna relación lineal directa entre ellas. Por ejemplo, `file_name_length` generalmente muestra correlaciones débiles con la mayoría de las otras variables. De manera similar, `src_port` y `dest_port` no muestran correlaciones fuertes con muchas otras características.

Este mapa de calor es fundamental para entender las interdependencias entre las variables del conjunto de datos. Puede ayudar a identificar redundancias (variables altamente correlacionadas que podrían medir aspectos similares) y a seleccionar características para modelos de aprendizaje automático.

5.2.9. Exportación del dataset

Por último, para poder guardar los cambios y poder utilizarlos a la hora de entrenar los modelos se debe de guardar en un fichero `csv`. Para ello se hace uso de la

función `to_csv` de `Python`. El código 5.5 muestra las dos líneas necesarias.

```
output_file = "transformed_dataset_clasificacion.csv"
df.to_csv(output_file, index=False)
```

Extracto de código 5.5: Exportación del DataFrame a CSV

5.3. Pasos generales de entrenamiento de los algoritmos

Este bloque de la memoria del trabajo consistirá en la explicación de todos los pasos necesarios para poder realizar el entrenamiento a los algoritmos explicados anteriormente. Se describirán todas las etapas llevadas a cabo, así como el código utilizado para cada tipo de algoritmo.

Cabe destacar que el código es distinto para cada entrenamiento debido a que cada algoritmo tiene unas ventajas e inconvenientes. No se aportará todo el código, sino lo más importante. Cada algoritmo ha sido entrenado en un *notebook* de Jupyter distinto.

5.3.1. Imputación de valores nulos

Al importar el dataset guardado anteriormente del preprocesamiento de datos, hay una serie de columnas con algunos valores nulos. Estas columnas son los puertos de origen y destino, la firma y la extension. No se pueden eliminar estas columnas debido a que aportan valor a la hora de clasificar las instancias de *malware*, es por ello que estos valores faltantes se deben de imputar.

La estrategia seguida a la hora de imputar estos valores nulos es asignar los puertos origen y destino con el valor `-1` debido a que no existe ningún puerto con este valor. Por otro lado, para la `signature` y `extension` se les ha establecido el valor `unknown`, que quiere decir desconocido y no se conoce estos valores. Esto se debe a que la mayoría de los modelos de *machine learning* no pueden trabajar ni funcionan correctamente con valores faltantes. El código 5.6 realiza los pasos explicados.

```
# Rellenamos los valores nulos en las columnas de puertos con -1 y convertimos a enteros
df['src_port'] = df['src_port'].fillna(-1).astype(int)
df['dest_port'] = df['dest_port'].fillna(-1).astype(int)

# Rellenamos los valores nulos en la columna 'signature' con "unknown"
df['signature'] = df['signature'].fillna("unknown")

# Rellenamos los valores nulos en la columna 'extension' con "unknown"
```

```
df['extension'] = df['extension'].fillna("unknown")

# Verificamos los resultados
print(df[['src_port', 'dest_port', 'signature', 'extension']].head
    ())
```

Extracto de código 5.6: Imputación de valores faltantes

5.3.2. Creación dataset predictor y objetivo

Después de realizar la imputación de los valores como se ha explicado anteriormente, el siguiente paso es separar las variables y dividir el dataset en dos. Por un lado, se elimina del dataset inicial la variable objetivo (`vt_category`) y, por otro lado, se crea otro dataset con dicha variable. Esto es necesario debido a que los modelos de *machine learning* trabajen de esta manera.

```
# Se separan las variables
X = df.drop(columns=['vt_category'])  # Variables predictoras
y = df['vt_category']                 # Variable objetivo

print("Dimensiones de X:", X.shape)
print("Dimensiones de y:", y.shape)
```

Extracto de código 5.7: Separación de variables predictoras y objetivo

5.3.3. Tratamiento variables categóricas

Algunas de las variables predictoras son categóricas, es decir, los valores que guardan no son númericos. Para los modelos de *machine learning* es necesario transformar estas variables a un tipo número. De los algoritmos que se utilizan para predecir la categoría del virus, hay algunos que tratan internamente esta transformación por las características del algoritmo pero, sin embargo, hay otros algoritmos que no.

Para los algoritmos que tratan internamente estas variables, lo único que se debe de hacer es añadir a una lista el nombre de estas variables. El código 5.8 muestra las líneas que se deben de programar.

```
# Se seleccionan las columnas categóricas
categorical_cols = X.select_dtypes(include=["object", "category"]).
    columns.tolist()
```

Extracto de código 5.8: Selección de columnas categóricas

Para los algoritmos que no tratan internamente este formato se debe de realizar la codificación de las variables. Para ello, en estos algoritmos se ha utilizado `Label Encoder` que codifica las variables con valores entre 0 y el número de clases máximo - 1. Esta codificación solo se hace en las variables predictoras, es decir sobre las columnas del dataset `X` (scikit-learn developers, 2023a). Un ejemplo puede ser si tenemos

`[Rojo, Azul, Verde, Azul]`, un `Label Encoder` podría transformarlo a `[2, 0, 1, 0]`, asignando un entero único a cada categoría (ej. Azul→0, Verde→1, Rojo→2).

```
# Aplicar Label Encoding a variables categóricas
le_signature = LabelEncoder()
le_extension = LabelEncoder()

df["signature"] = le_signature.fit_transform(df["signature"])
df["extension"] = le_extension.fit_transform(df["extension"])
```

Extracto de código 5.9: Aplicación de Label Encoding a variables categóricas

5.3.4. División de datos en entrenamiento y test

Tras realizar la imputación de los valores faltantes y separación de las variables predictoras y objetivo, se disponen de dos datasets `X` e `y` con un total de 41.572 filas y 33 columnas predictoras. Todas son variables numéricas menos la variable objetivo que es de tipo categórico. El siguiente paso es la división de los datos dentro de cada dataset en entrenamiento y test. Es necesario separarlo en estos conjuntos debido a que así se evalúa el rendimiento de los algoritmos. Se entrenan con los valores de *train* de los dos dataset y se evalúa con datos que no han sido vistos como son los de *test*.

Los porcentajes utilizados para la división son del 80 % de los datos para el entrenamiento mientras que un 20 % de los datos totales son para el *test*. A esta división se le llama *Hold-out*. Divide aleatoriamente el conjunto de datos disponibles en dos subconjuntos: un conjunto de entrenamiento y un conjunto de prueba. De este modo, el modelo puede entrenarse y probarse con datos diferentes. El modelo se ajusta en el conjunto de entrenamiento, y el modelo ajustado se utiliza para predecir las respuestas de las observaciones del conjunto de validación. El error resultante del conjunto de validación proporciona una estimación del error de la prueba. El código 5.10 muestra los pasos necesarios.

```
# Hold-Out. 80% de entrenamiento y 20% prueba.
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size
    =0.2, random_state=42, stratify=y)

print("Tamaño del conjunto de entrenamiento:", X_train.shape,
    y_train.shape)
print("Tamaño del conjunto de prueba:", X_test.shape, y_test.shape)
```

Extracto de código 5.10: Hold-Out: división 80 % entrenamiento y 20 % prueba

5.3.5. Normalización

La normalización es una técnica muy utilizada a la hora de desarrollar *machine learning*. El objetivo es ajustar los datos numéricos a un rango específico. El método utilizado para realizar la normalización es el llamado ***Standard Scaler***, que transforma los datos para que tengan una media de 0 y una desviación estándar de 1. Hay que

destacar que la normalización solo se aplicará en los algoritmos en los que sea necesario y también es importante añadir que solo se utiliza en el dataset predictor. La fórmula de estandarización es la siguiente (scikit-learn developers, 2023b):

$$X_{\text{scaled}} = \frac{X - \mu}{\sigma} \tag{5.1}$$

Donde:

- X es el valor original de los datos.
- μ es la media de los datos.
- σ es la desviación estándar de los datos.
- X_{scaled} es el valor estandarizado.

El código 5.11 es el utilizado en los cuadernos de Jupyter.

```
# Inicializar el escalador
scaler = StandardScaler()

# Ajustar el escalador con los datos de entrenamiento y transformar
X_train_scaled = scaler.fit_transform(X_train)

# Transformar el conjunto de prueba usando los mismos parámetros
    del entrenamiento
X_test_scaled = scaler.transform(X_test)

# Mostrar confirmación de transformación
print("Escalado completado.")
print("Dimensiones después del escalado:")
print(f"- X_train_scaled: {X_train_scaled.shape}")
print(f"- X_test_scaled: {X_test_scaled.shape}")
```

Extracto de código 5.11: Aplicación de StandardScaler a los datos

5.3.6. Balanceo de clases

Ciertos algoritmos no tratan el balanceo de la clase objetivo automáticamente. Para ello se debe de realizar un balanceo de clases en ellos. En nuestro trabajo realizamos cuatro rebalanceos de clases distintos como son `SMOTE`, `RandomUndersampling`, `Cluster Centroids` y `Edited Nearest Neighbors (ENN)`. Se adjuntarán los diferentes códigos para cada tipo de balanceo.

SMOTE

Antes de aplicar SMOTE, se imprimen las dimensiones del conjunto de entrenamiento y la distribución de clases original, para así verificar si existe un desbalance en

los datos. A continuación, se utiliza la técnica SMOTE para generar muestras sintéticas de la clase minoritaria con el objetivo de equilibrar las clases objetivos.

Posteriormente, se vuelven a imprimir las dimensiones del conjunto de entrenamiento y la nueva distribución de clases para comprobar que el proceso de sobremuestreo se ha realizado correctamente.

```
# Mostrar tamaños antes de SMOTE
print("Tamaño antes de SMOTE:", X_train_scaled.shape, y_train.shape)
print("Distribución de clases antes de SMOTE:")
class_distribution_before = pd.Series(y_train).value_counts().sort_index()
print(class_distribution_before)

# Aplicar SMOTE
smote = SMOTE(random_state=42)
X_train_sm, y_train_sm = smote.fit_resample(X_train_scaled, y_train)

# Mostrar tamaños después de SMOTE
print("Tamaño después de SMOTE:", X_train_sm.shape, y_train_sm.shape)
print("Distribución de clases después de SMOTE:")
class_distribution_after = pd.Series(y_train_sm).value_counts().sort_index()
print(class_distribution_after)
```

Extracto de código 5.12: Aplicación de SMOTE para balanceo de clases

RandomUndersampling

Antes de aplicar la técnica de *undersampling*, se muestra la cantidad de valores del conjunto de entrenamiento y la distribución de clases, para confirmar el desbalance. El *Random Undersampling* reduce el tamaño de la clase mayoritaria eliminando instancias al azar, equilibrando así la distribución de clases.

Tras aplicar el *undersampling*, se vuelve a mostrar el tamaño del nuevo conjunto de entrenamiento y su distribución de clases para comprobar que el balance ha sido realizado de la manera correcta.

```
# Mostrar tamaños antes de Undersampling
print("Tamaño antes de Undersampling:", X_train_scaled.shape, y_train.shape)
print("Distribución de clases antes de Undersampling:")
class_distribution_before = pd.Series(y_train).value_counts().sort_index()
print(class_distribution_before)

# Aplicar Undersampling sobre el train escalado
undersampler = RandomUnderSampler(random_state=42)
```

```
X_train_us, y_train_us = undersampler.fit_resample(X_train_scaled,
    y_train)

# Mostrar tamaños después de Undersampling
print("Tamaño después de Undersampling:", X_train_us.shape,
    y_train_us.shape)
print("Distribución de clases después de Undersampling:")
class_distribution_after = pd.Series(y_train_us).value_counts().
    sort_index()
print(class_distribution_after)
```

Extracto de código 5.13: Aplicación de Random Undersampling para balanceo de clases

Cluster Centroids

La técnica Cluster Centroids es una estrategia de *undersampling* que busca reducir la cantidad de muestras de la clase mayoritaria mejor que el submuestreo aleatorio. Se conserva una representación general de los datos originales.

En el código, se muestra primero el tamaño y la distribución de clases antes del balanceo. Luego se aplica Cluster Centroids y se imprimen las dimensiones del nuevo conjunto de entrenamiento y su distribución, evidenciando que se ha logrado un balance entre clases.

```
# Cluster Centroids: Balanceo de Datos
print("Tamaño antes del balanceo (Cluster Centroids):",
    X_train_scaled.shape, y_train.shape)
cc = ClusterCentroids(random_state=42)
X_train_bal, y_train_bal = cc.fit_resample(X_train_scaled, y_train)

print("Tamaño después del balanceo:", X_train_bal.shape,
    y_train_bal.shape)
print("Distribución de clases después del balanceo:")
print(pd.Series(y_train_bal).value_counts().sort_index())
```

Extracto de código 5.14: Aplicación de Cluster Centroids para balanceo de clases

ENN

El método *Edited Nearest Neighbours (ENN)* es una técnica de limpieza de datos basada en el concepto de vecinos más cercanos. ENN elimina aquellas muestras del conjunto de entrenamiento que no coinciden con la mayoría de sus vecinos más cercanos (según el parámetro `n_neighbors`).

Se imprime primero el tamaño y la distribución de clases original. Posteriormente, se aplica ENN para realizar el filtrado y se visualiza el nuevo tamaño y distribución del conjunto resultante, permitiendo verificar su efecto sobre la estructura del dataset.

```python
# Aplicar ENN correctamente
print("Tamaño antes del balanceo (ENN):", X_train_scaled.shape, y_train.shape)
enn = EditedNearestNeighbours(n_neighbors=3)
X_train_bal, y_train_bal = enn.fit_resample(X_train_scaled, y_train)

print("Tamaño después del balanceo:", X_train_bal.shape, y_train_bal.shape)
print("Distribución de clases después del balanceo:")
print(pd.Series(y_train_bal).value_counts().sort_index())
```

Extracto de código 5.15: Aplicación de Edited Nearest Neighbours (ENN) para balanceo de clases

5.3.7. Cross-Validation

Después de haber explicado y dónde se pueden aplicar todos los conceptos anteriormente, el siguiente paso es realizar *Cross-Validation* para evitar el *overfitting* en los diferentes algoritmos y así poder ver como se comporta con diferentes datos.

Un ejemplo a modo general sin especificar el algoritmo se puede ver en el código 5.16.

```python
# X: Dataset predictoras
# y: Dataset objetivo

#  Calcular pesos de clase si hay desbalanceo para las clases que lo realizan automáticamente
# class_weights = compute_class_weight('balanced', classes=np.unique(y), y=y)

base_model = AlgorithmClassifier(
    # Hiperparámetros específicos del modelo
    # class_weights=class_weights, # Si se aplica
    # cat_features=categorical_cols, # Si el modelo lo soporta
    random_seed=42,
    # verbose=0 # Para controlar la salida del modelo
)

cv_strategy = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Listas para almacenar métricas de cada fold
accuracies = []
mccs = []
roc_aucs= []

```

```python
for fold_number, (train_indices, test_indices) in enumerate(cv_strategy.split(X, y)):
    print(f"\n=== Fold {fold_number + 1} ===")

    X_train_fold, X_test_fold = X.iloc[train_indices], X.iloc[test_indices]
    y_train_fold, y_test_fold = y.iloc[train_indices], y.iloc[test_indices]

    # Crear una nueva instancia o copia del modelo para cada fold
    model =  base_model.copy()
    model.fit(X_train_fold, y_train_fold)

    y_predictions_fold = model.predict(X_test_fold)

    # Cálculo de métricas
    acc = accuracy_score(y_test_fold, y_predictions_fold)
    mcc = matthews_corrcoef(y_test_fold, y_predictions_fold)

    try:
        y_probabilities_fold = current_model.predict_proba(X_test_fold)
        roc_auc = roc_auc_score(y_test_fold, y_probabilities_fold, multi_class='ovo', average='macro')
    except AttributeError:
        roc_auc = None # O manejar de otra forma

    print(f"Accuracy: {acc:.4f}")
    print(f"MCC: {mcc:.4f}")
    if roc_auc is not None:
        print(f"ROC AUC (macro): {roc_auc:.4f}")
    print("Classification Report:\n",classification_report(y_test_fold, y_predictions_fold))

    accuracies.append(acc)
    mccs.append(mcc)
    if roc_auc is not None:
        roc_aucs.append(roc_auc)

print("\n========================================")
print("Resultados Finales de Validación Cruzada")
print(f"Promedio de Accuracy: {np.mean(accuracies):.4f}")
print(f"Promedio de MCC: {np.mean(mccs):.4f}")
if roc_aucs:
    print(f"Promedio de ROC AUC (macro): {np.mean(roc_aucs):.4f}")
```

Extracto de código 5.16: Estructura general de un proceso de validación cruzada para modelos de clasificación.

1. **Preparación inicial:** se definen los datos de entrada (`X`, `y`) y se calcula los pesos para las clases desbalanceadas.

2. **Definición del modelo base:** se selecciona el algoritmo de clasificación deseado con un conjunto de hiperparámetros. Este modelo se utilizará para cada iteración de la validación cruzada.
3. **Configuración de la validación cruzada:** se establece la estrategia para dividir el conjunto de datos. `StratifiedKFold` es una elección común para problemas de clasificación, ya que busca mantener la proporción original de las clases en cada una de las `n_splits` divisiones (pliegues o folds).
4. **Bucle de validación cruzada:**
   - Los datos se dividen en subconjuntos de entrenamiento y prueba específicos para el pliegue actual.
   - Se hace una copia del modelo base utilizando únicamente los datos de entrenamiento del pliegue que toque en esa iteración del bucle. Es importante no reutilizar un modelo ya entrenado en pliegues anteriores para evitar la fuga de información.
   - El modelo entrenado se utiliza para generar predicciones sobre el subconjunto de prueba del pliegue.
   - Se calculan diversas métricas de rendimiento (como Accuracy, Coeficiente de Correlación de Matthews (MCC), Área Bajo la Curva ROC (ROC AUC)) comparando las predicciones con las etiquetas verdaderas del conjunto de prueba.
   - Estas métricas se almacenan para su posterior utilización.
5. **Resultados:** finalmente, se calculan estadísticas descriptivas (generalmente la media) de las métricas obtenidas en cada pliegue. Esto proporciona una estimación más fiable y menos sesgada del rendimiento esperado del modelo en datos no vistos, en comparación con una única división de entrenamiento/prueba, y así se puede ver si hay mucha diferencia entre los datos o no para comprender el *overfitting*

5.3.8. Entrenamiento del modelo

Por último, después de haber entrenado el modelo con *Cross-Validation* y se ha detectado que no existe *overfitting*, lo único que queda es entrenar el modelo creado anteriormente y exponer los resultados.

```
start_time = time.time()

# Se entrena el modelo final con el conjunto de entrenamiento.
trained_model = base_model.copy()
trained_model.fit(X_train, y_train)

# Se generan predicciones sobre los conjuntos de entrenamiento y
    prueba.
y_train_predictions = trained_model.predict(X_train)
y_test_predictions = trained_model.predict(X_test)
```

```
10
11 # Precisión general (Accuracy)
12 train_accuracy = accuracy_score(y_train, y_train_predictions)
13 test_accuracy = accuracy_score(y_test, y_test_predictions)
14
15 # Reportes de clasificación detallados (precisión, recall, F1-score
       por clase)
16 train_detailed_report = classification_report(y_train,
       y_train_predictions, digits=4)
17 test_detailed_report = classification_report(y_test,
       y_test_predictions, digits=4)
18
19 # Matriz de Confusión (para el conjunto de prueba)
20 test_confusion_matrix = confusion_matrix(y_test, y_test_predictions
       )
21
22 # Coeficiente de Correlación de Matthews (MCC)
23 test_mcc = matthews_corrcoef(y_test, y_test_predictions)
24
25 # Área Bajo la Curva ROC (ROC AUC) - para clasificación multiclase
       (OVR)
26 try:
27     y_test_probabilities = trained_model.predict_proba(X_test)
28     # Binarizar las etiquetas verdaderas para el cálculo ROC AUC
           multiclase
29     y_test_binarized = label_binarize(y_test, classes=np.unique(
           y_test))
30     test_roc_auc = roc_auc_score(y_test_binarized,
           y_test_probabilities, average="macro", multi_class="ovr")
31 except AttributeError:
32     test_roc_auc = None # Si el modelo no soporta predict_proba
33
34 print("="*50)
35 print("Resultados del Modelo Final")
36 print("="*50)
37 print(f"Accuracy en Entrenamiento: {train_accuracy:.4f}")
38 print("Reporte de Clasificación en Entrenamiento:")
39 print(train_detailed_report)
40
41 print("="*50)
42 print(f"Accuracy en Prueba: {test_accuracy:.4f}")
43 print("Reporte de Clasificación en Prueba:")
44 print(test_detailed_report)
45 print(f" MCC en Prueba: {test_mcc:.4f}")
46 if test_roc_auc is not None:
47     print(f" ROC AUC (macro) en Prueba: {test_roc_auc:.4f}")
48 print("="*50)
49
50 end_time = time.time()
51 print(f"Tiempo de ejecución total: {end_time - start_time:.2f}
```

```
    segundos")
```

Extracto de código 5.17: Proceso de entrenamiento y evaluación final de un modelo de clasificación.

1. **Entrenamiento del modelo:** se entrena el modelo con el conjunto de datos de entrenamiento (`X_train`, `y_train`).
2. **Generación de Predicciones:** se utilizan las predicciones del modelo tanto para el conjunto de entrenamiento como para el de prueba.
3. **Cálculo de métricas clave:** se calculan y recopilan métricas de rendimiento esenciales como la precisión (*accuracy*), el coeficiente de correlación de Matthews (MCC), y el área bajo la curva ROC (ROC AUC) para el conjunto de prueba. Además, se generan reportes de clasificación detallados que proporcionan métricas por clase como precisión, recall y F1-score.
4. **Impresión de resultados:** finalmente, se muestran los resultados consolidados de las métricas en ambos conjuntos y el tiempo de ejecución.

5.4. Entrenamiento específico de los algoritmos

En este apartado de la memoria se informará sobre los pasos seguidos en cada uno de los algoritmos utilizados a la hora de clasificar los distintos resultados.

5.4.1. CatBoost

Lo primero que hay que realizar para entrenar el algoritmo de clasificación `CatBoost` es importar el dataset con el que se trabajará. Una vez se importe el dataset, se debe de realizar la imputación de los valores faltantes y posteriormente separar las variables predictoras y objetivo. `CatBoost` al ser un algoritmo que trata internamente las columnas categóricas se deben de guardar en una lista para después realizar distintas operaciones. Se dividen los distintos dataset en entrenamiento y pruebas y por último se calcula los pesos de las diferentes clases objetivos (*class_weight*) para llevar a cabo el balanceo automáticamente y se crea el modelo para entrenarlo con *Cross-Validation* y después entrenarlo para obtener los resultados. La figura muestra los pasos realizados para entrenar el modelo.

5.4.2. Árbol de decisión

Para entrenar el algoritmo de clasificación `DecisionTree`, el primer paso es importar el dataset. Seguidamente, se debe de realizar la imputación de los valores faltantes. `DecisionTree` al ser un algoritmo que no trata internamente las columnas categóricas, se debe de realizar el codificado de los datos utilizando *Label Encoder* y se separan las variables predictoras y objetivo. Se dividen los distintos dataset en entrenamiento y pruebas. Para abordar el posible desbalance en las clases de la variable objetivo, se

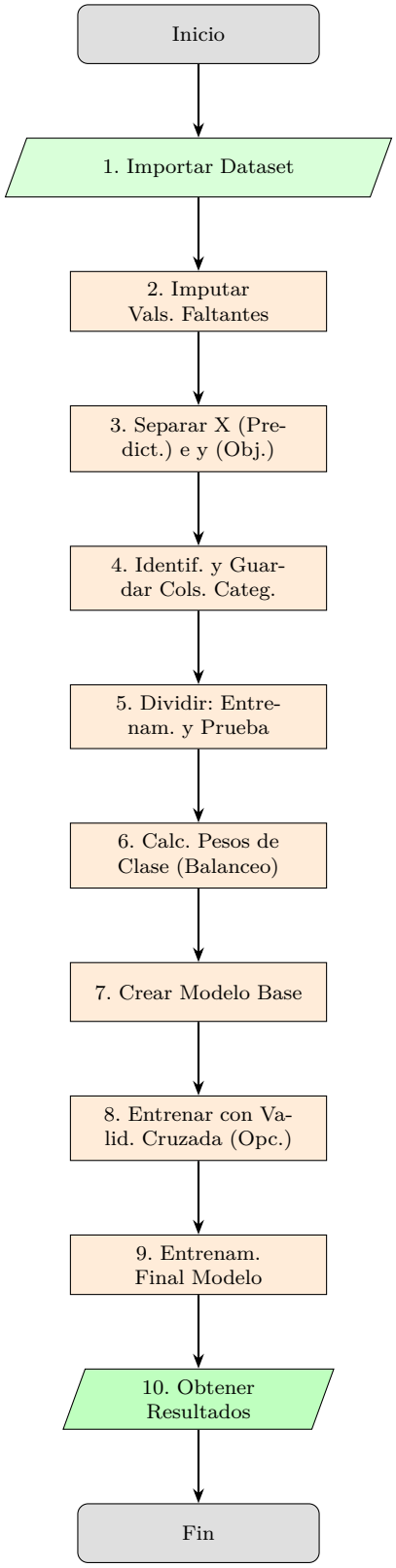


Figura 5.9: Diagrama de flujo compacto del preprocesamiento y configuración para `CatBoost`.

Hiperparámetro	Valor	Propósito
`iterations`	300	Número total de árboles entrenados.
`depth`	6	Profundidad máxima de cada árbol.
`learning_rate`	0,03	Tasa de aprendizaje (pasos de optimización más finos con valores bajos).
`colsample_bylevel`	0,8	Proporción de *features* usadas en cada nivel (submuestreo).
`l2_leaf_reg`	1	Penalización L2 que reduce el sobreajuste.
`class_weights`	*dict*	Compensa el desbalance entre clases.
`cat_features`	`categorical_cols`	Columnas categóricas procesadas de forma específica.
`random_seed`	42	Semilla para reproducibilidad.
`verbose`	0	Entrenamiento en modo silencioso.

Tabla 5.6. *Hiperparámetros y valores en* `CatBoostClassifier`

pueden aplicar dos enfoques: primero, calcular los pesos de clase (*class_weight*) para que el algoritmo los considere durante el entrenamiento y realice un balanceo automático; segundo, aplicar técnicas de muestreo manual para balancear las clases antes del entrenamiento. Entre estas técnicas manuales se incluyen *SMOTE*, *ENN*, Cluster Centroids y *Random Undersampling*. Finalmente, se crea el modelo `DecisionTreee`, se entrena usando Validación Cruzada para optimización y se evalúa para obtener los resultados, comparando los efectos de los diferentes métodos de balanceo.

Hiperparámetro	Valor	Propósito
`max_depth`	6	Profundidad máxima del árbol (controla la complejidad).
`criterion`	`gini`	Función de impureza usada para dividir los nodos.
`splitter`	`best`	Selecciona la mejor división disponible en cada nodo.
`min_samples_split`	2	Mínimo de muestras requerido para subdividir un nodo.
`min_samples_leaf`	1	Mínimo de muestras que debe contener una hoja.
`class_weight`	*class_weights*	Pondera las clases para tratar el desbalance de datos.
`random_state`	42	Semilla para reproducibilidad.

Tabla 5.7. *Hiperparámetros y valores en* `DecisionTreeClassifier` *sin balanceo*

Hiperparámetro	Valor	Propósito
`max_depth`	6	Profundidad máxima del árbol (controla la complejidad).
`criterion`	`gini`	Función de impureza empleada para dividir los nodos.
`splitter`	`best`	Selecciona la mejor división disponible en cada nodo.
`min_samples_split`	2	Mínimo de muestras requerido para subdividir un nodo.
`min_samples_leaf`	1	Mínimo de muestras que debe contener una hoja.
`random_state`	42	Semilla que garantiza la reproducibilidad.

Tabla 5.8. *Hiperparámetros y valores en* `DecisionTreeClassifier` *con balanceo*

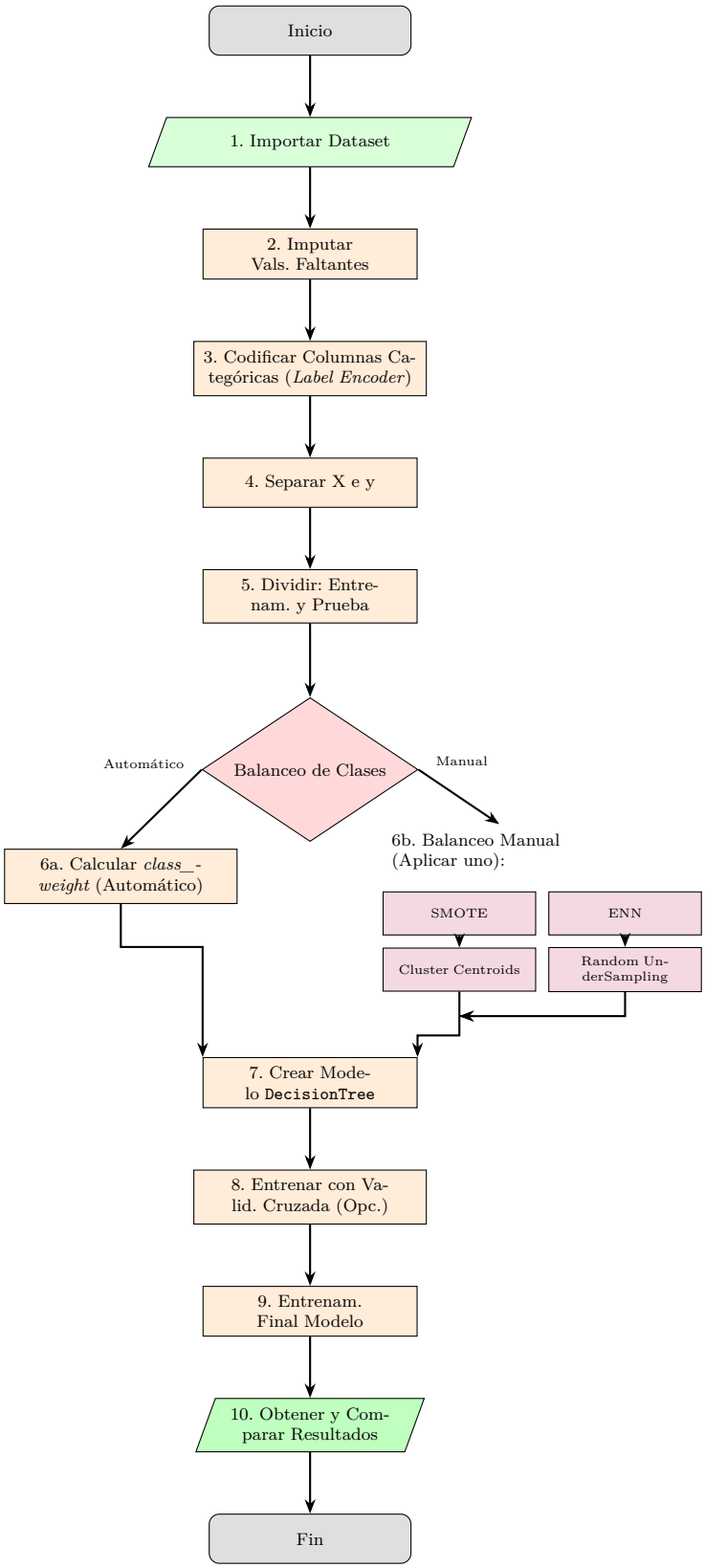


Figura 5.10: Diagrama de flujo del preprocesamiento para `DecisionTree` (corregido origen de flechas manuales).

5.4.3. KNN

El algoritmo `KNN` se caracteriza por su distancia entre los diferentes puntos a la hora de clasificar una entrada. Se importa el dataset y se rellenan los valores faltantes. No transforma internamente las columnas categóricas, es por ello que se deben de codificar utilizando *Label Encoder*. Se separan variables predictoras y objetivo y se dividen en entrenamiento y prueba. Como el algoritmo se fundamenta en la distancia, los valores numéricos en el dataset se deben de escalar con *Standard Scaler*. El siguiente paso es balancear las clases con los cuatro métodos vistos y después entrenarlos con *Cross-Validation* y concluir con el entreno final del modelo.

Hiperparámetro	**Valor**	**Propósito**
`n_neighbors`	`best_k`	Número de vecinos considerados para la predicción.
`weights`	`distance`	Pondera la influencia de cada vecino inversamente proporcional a su distancia.

Tabla 5.9. *Hiperparámetros y valores en `KNeighborsClassifier`*

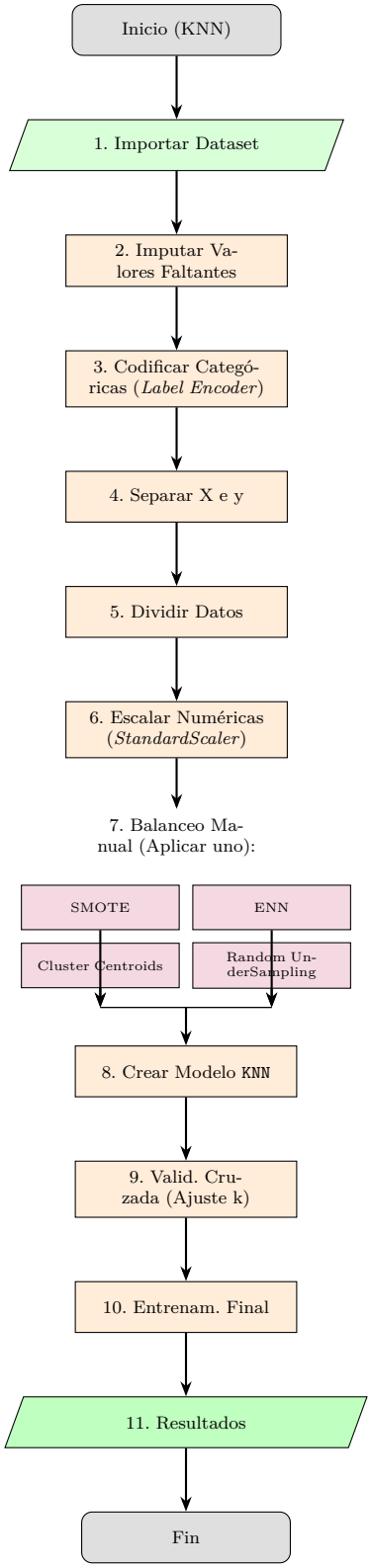


Figura 5.11: Diagrama de flujo para `KNN` con balanceo manual (corregido y simplificado).

5.4.4. LigthGBM

El proceso inicial para entrenar el algoritmo de clasificación `LightGBM` comienza con la importación del conjunto de datos. Una vez se carga, se procede a la imputación de los valores faltantes. A continuación, se separan las variables predictoras de la variable objetivo. `LightGBM` puede manejar características categóricas de manera interna, por lo tanto, se identifican estas variables en una lista. Posteriormente, los datos se dividen en conjuntos de entrenamiento y prueba. Finalmente, se calcula el parámetro *scale_pos_weight* y se define el modelo `LightGBM`. Este se entrena utilizando Cross-Validation para evitar *overfitting*, antes del entrenamiento definitivo para la obtención de resultados.

Hiperparámetro	**Valor**	**Propósito**
`n_estimators`	300	Número total de árboles que se entrenan.
`max_depth`	6	Profundidad máxima de cada árbol.
`learning_rate`	0,03	Tasa de aprendizaje que controla el aporte de cada árbol.
`colsample_bytree`	0,8	Proporción de *features* usadas para construir cada árbol.
`reg_lambda`	1	Regularización L2 para prevenir sobreajuste.
`class_weight`	*class_weigths*	Compensación del desbalance de clases.
`random_state`	42	Semilla para reproducibilidad.
`verbose`	-1	Modo silencioso (suprime avisos).

Tabla 5.10. *Hiperparámetros y valores en `LGBMClassifier`*

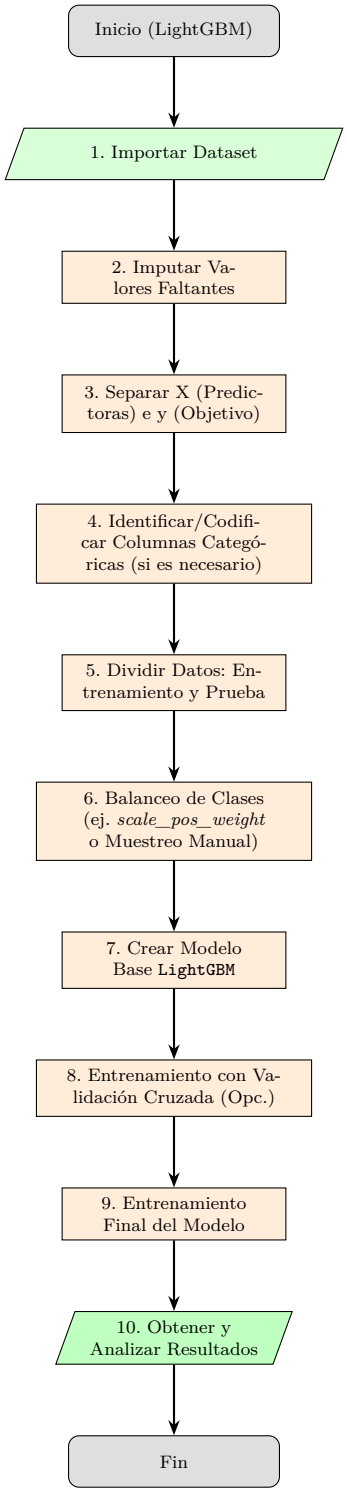


Figura 5.12: Diagrama de flujo del proceso de preprocesamiento y configuración para el modelo `LightGBM`.

5.4.5. MLP

El Perceptrón Multicapa (`MLPClassifier`), un tipo de red neuronal artificial, comienza con la importación del conjunto de datos, seguido de la imputación de valores faltantes. Dado que los `MLPClassifier` operan con entradas numéricas, las características categóricas deben ser transformadas; se utilizará *Label Encoder*. Una vez codificadas las categóricas, se separan las variables predictoras de la variable objetivo y los datos se dividen en subconjuntos de entrenamiento y prueba. Son sensibles a la escala de las características de entrada, es por ello que se debe de escalar todas las variables numéricas utilizando *StandardScaler*. Posteriormente, para abordar el desequilibrio de clases, se pueden aplicar los cuatro métodos de balanceo vistos. Finalmente, se define la arquitectura del MLPClassifier, se entrena con *Cross-Validation* para ajustar hiperparámetros como el número de capas ocultas, neuronas, o la tasa de aprendizaje y se evalúa para obtener los resultados.

Hiperparámetro	**Valor**	**Propósito**
`hidden_layer_sizes`	`best_hidden_layers`	Arquitectura de la red (n.º de neuronas por capa oculta).
`activation`	`relu`	Función de activación usada en las capas ocultas.
`solver`	`adam`	Algoritmo de optimización para entrenar la red.
`max_iter`	500	Número máximo de iteraciones (épocas) de entrenamiento.
`random_state`	42	Semilla para resultados reproducibles.

Tabla 5.11. *Hiperparámetros y valores en `MLPClassifier`*

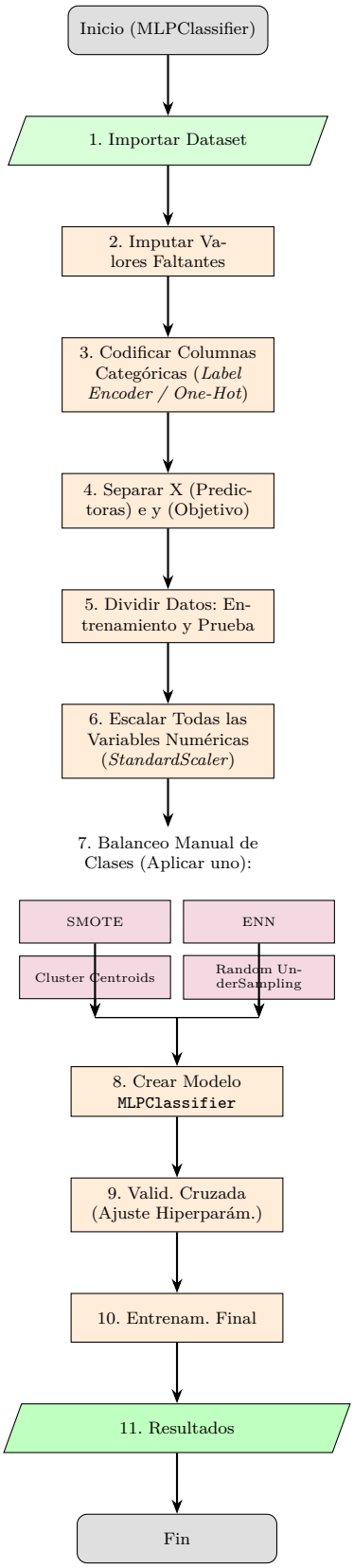


Figura 5.13: Diagrama de flujo para `MLPClassifier` (corregido).

5.4.6. `RandomForest`

Para entrenar un modelo de clasificación basado en `RandomForest`, el proceso inicial implica la importación del conjunto de datos. Posteriormente, se realiza la imputación de valores faltantes. Dado que los `RandomForest`, al igual que los árboles de decisión, no manejan intrínsecamente las características categóricas, estas deben ser transformadas a un formato numérico, usando *Label Encoder*. Una vez codificadas, se separan las variables predictoras de la variable objetivo. Los datos se dividen luego en subconjuntos de entrenamiento y prueba. Para abordar el posible desbalance en las clases de la variable objetivo, se pueden aplicar dos enfoques principales: primero, utilizar el parámetro *class_weight* para que el algoritmo pondere las muestras durante el entrenamiento, realizando un balanceo automático; segundo, aplicar técnicas de muestreo manual para balancear las clases directamente en los datos antes del entrenamiento. Entre estas técnicas manuales se incluyen SMOTE, ENN, Cluster Centroids y Random Undersampling. Finalmente, se define y crea el modelo `RandomForest`, se entrena, a menudo utilizando *Cross-Validation* para ver cómo se comporta con el *overfitting*, y se evalúa para obtener los resultados.

Hiperparámetro	**Valor**	**Propósito**
`n_estimators`	300	Número total de árboles que componen el bosque.
`max_depth`	6	Profundidad máxima permitida para cada árbol.
`criterion`	`gini`	Función de impureza usada para escoger las divisiones.
`min_samples_split`	2	Muestras mínimas requeridas para dividir un nodo.
`min_samples_leaf`	1	Muestras mínimas que debe contener una hoja.
`class_weight`	*class_weights*	Ajuste de pesos para mitigar el desbalance de clases.
`random_state`	42	Semilla que garantiza resultados reproducibles.
`n_jobs`	-1	Usa todos los núcleos disponibles para el entrenamiento.

Tabla 5.12. *Hiperparámetros y valores en `RandomForestClassifier` balanceo automático*

Hiperparámetro	**Valor**	**Propósito**
`n_estimators`	300	Número total de árboles que componen el bosque.
`max_depth`	6	Profundidad máxima permitida para cada árbol.
`criterion`	`gini`	Función de impureza usada para elegir las divisiones.
`min_samples_split`	2	Muestras mínimas requeridas para dividir un nodo.
`min_samples_leaf`	1	Muestras mínimas que debe contener una hoja.
`random_state`	42	Semilla que garantiza la reproducibilidad.
`n_jobs`	-1	Usa todos los núcleos disponibles para el entrenamiento.

Tabla 5.13. *Hiperparámetros y valores en `RandomForestClassifier`*

‘‘

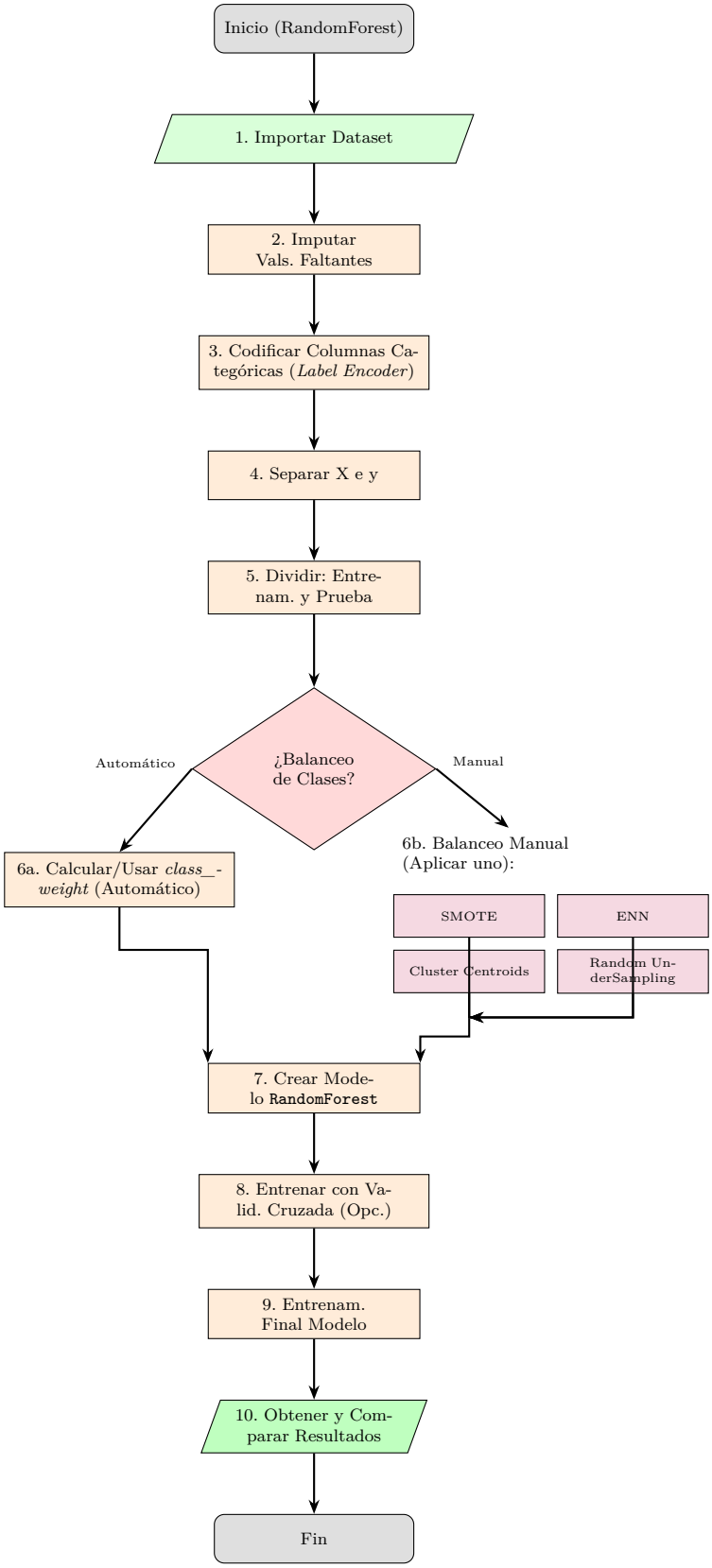


Figura 5.14: Diagrama de flujo del preprocesamiento y configuración para `RandomForestClassifier` con opciones de balanceo.

5.4.7. SVM

El clasificador de Máquinas de Soporte Vectorial (`SVM`) se inicia con la importación del conjunto de datos, tras lo cual se realiza la imputación de los valores ausentes. Dado que los algoritmos `SVM` operan fundamentalmente con entradas numéricas, las características categóricas se transforman con *Label Encoder*. Después del codificado de los datos, se separan las variables predictoras (características) de la variable objetivo. Los datos se dividen en conjuntos de entrenamiento y prueba. Los `SVM`, que son muy sensibles a la escala de las características de entrada y es por eso que se debe de aplicar *StandardScaler*. A continuación, para manejar el desequilibrio de clases, se aplican los cuatro métodos de muestreo manual. Finalmente, se define el modelo `SVM`, se entrena, utilizando a menudo Validación Cruzada y se evalúa para obtener los resultados finales.

Hiperparámetro	**Valor**	**Propósito**
`C`	`C_fixed`	Parámetro de penalización que controla el equilibrio entre margen ancho y errores de clasificación.
`kernel`	`rbf`	Tipo de núcleo (función de similitud) utilizado para proyectar los datos en un espacio de mayor dimensión.
`gamma`	`scale`	Anchura del núcleo RBF; `scale` ajusta $1/(\text{n_features}\,\sigma^2)$.
`probability`	`True`	Habilita la estimación de probabilidades mediante Platt scaling.
`random_state`	42	Semilla que garantiza la reproducibilidad de los resultados.

Tabla 5.14. *Hiperparámetros y valores en `SVC`*

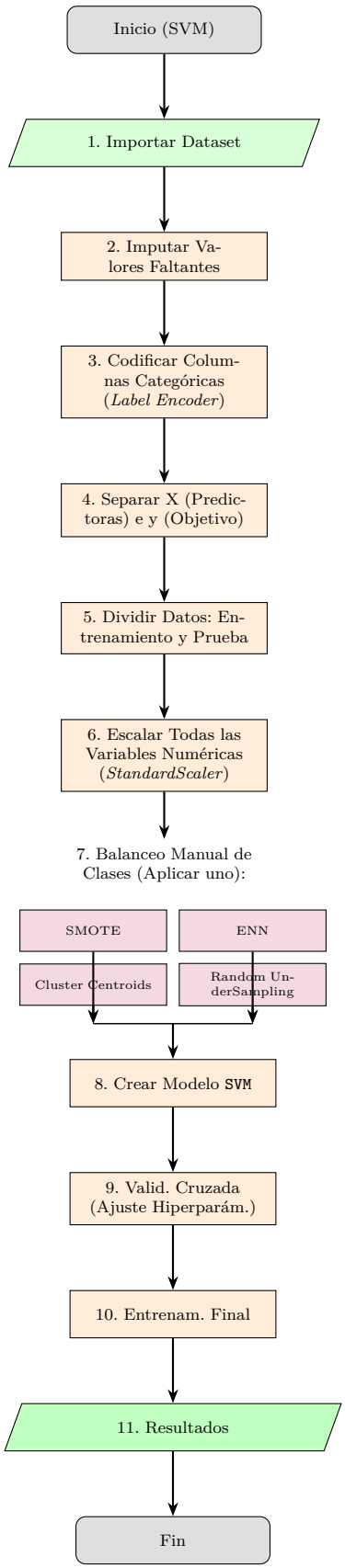


Figura 5.15: Diagrama de flujo del preprocesamiento y configuración para `SVM` con balanceo manual (corregido).

5.4.8. XGBoost

El algoritmo `XGBoost` (Extreme Gradient Boosting), primero de todo, realiza la importación del conjunto de datos. A continuación, es importante rellenar los valores faltantes. Las características categóricas,se transforman previamente a un formato numérico mediante *Label Encoder*. Una vez preparadas las características, se separan las variables predictoras de la variable objetivo. Los datos se dividen luego en subconjuntos de entrenamiento y prueba. Para abordar el desequilibrio en las clases de la variable objetivo, se pueden considerar dos enfoques: primero, XGBoost ofrece el parámetro *scale_pos_weight* que permite ponderar la clase positiva, o se pueden calcular pesos de clase personalizados para problemas multiclase; segundo, se pueden aplicar técnicas de muestreo manual (como SMOTE, ENN, Cluster Centroids y Random Undersampling) a los datos antes del entrenamiento. Finalmente, se configura y crea el modelo `XGBoost`, se entrena utilizando *Cross-Validation* y se evalúa para obtener los resultados, comparando los efectos de los diferentes métodos de balanceo.

Hiperparámetro	**Valor**	**Propósito**
`n_estimators`	300	Número total de árboles (rondas de boosting).
`max_depth`	6	Profundidad máxima de cada árbol.
`learning_rate`	0,03	Tasa de aprendizaje (paso de actualización de cada iteración).
`colsample_bytree`	0,8	Proporción de *features* muestreadas por árbol.
`subsample`	0,8	Proporción de instancias muestreadas por árbol.
`reg_alpha`	0,1	Regularización L1 que favorece la sparsidad.
`reg_lambda`	1	Regularización L2 para evitar sobreajuste.
`objective`	`multi:softprob`	Función objetivo para clasificación multiclase (probabilidades).
`num_class`	`len(le.classes_)`	Número total de clases en la variable objetivo.
`use_label_encoder`	`False`	Desactiva el codificador interno antiguo de XGBoost.
`random_state`	42	Semilla para reproducibilidad.
`n_jobs`	-1	Utiliza todos los núcleos disponibles.
`eval_metric`	`mlogloss`	Métrica de evaluación durante el entrenamiento.

Tabla 5.15. *Hiperparámetros y valores en `XGBClassifier`*

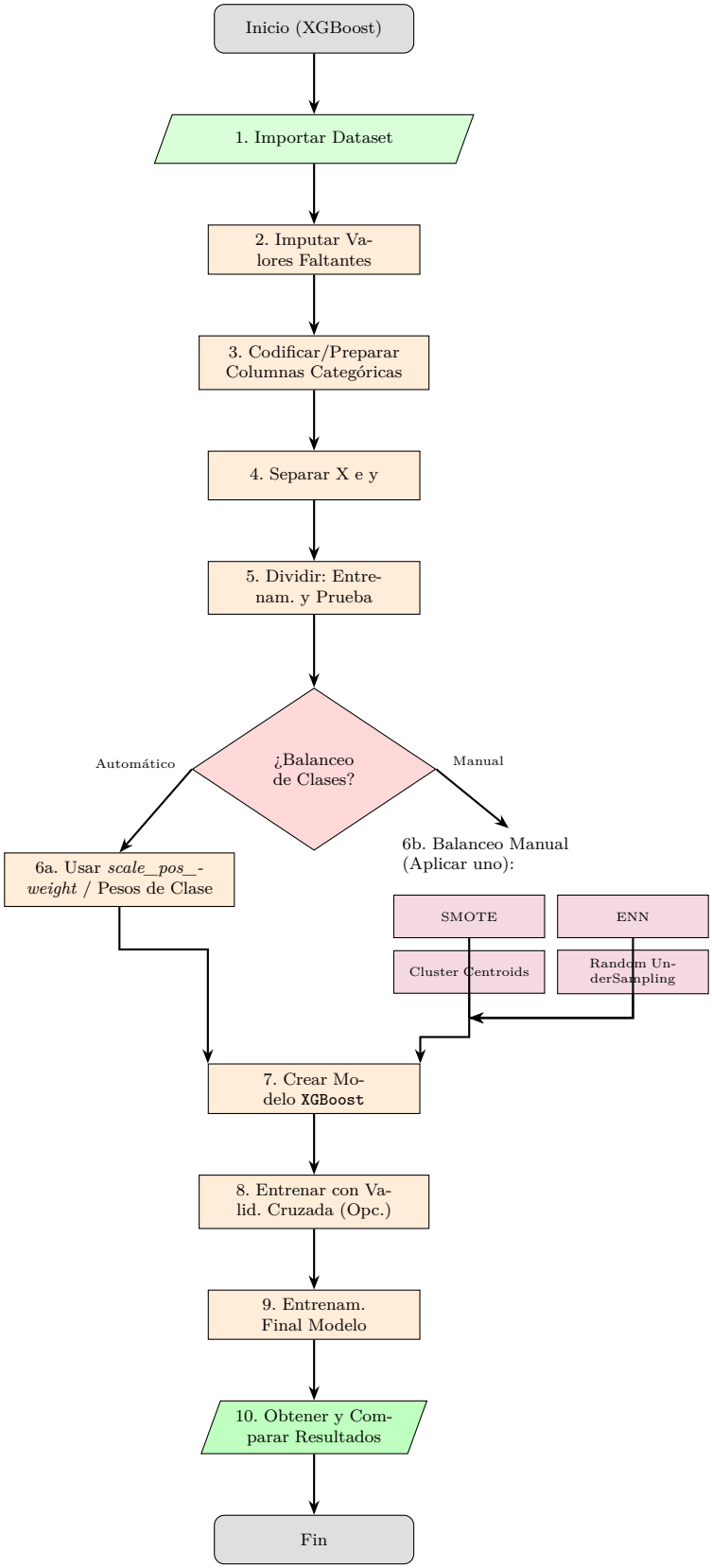


Figura 5.16: Diagrama de flujo del preprocesamiento y configuración para `XGBoost` con opciones de balanceo.

5.4.9. Redes neuronales

Para el proceso de entrenamiento de una red neuronal se comienza con la importación de los datos junto con la imputación de los valores faltantes. Todas las características categóricas deben ser transformadas; se utilizará (*One-Hot Encoder*). Una vez codificadas las categorias, se separan las variables predictoras de la variable objetivo y los datos se dividen en subconjuntos de entrenamiento y prueba. Son sensibles a la escala de las características de entrada, es por ello que se debe de escalar todas las variables numéricas utilizando *StandardScaler*. Posteriormente, para abordar el desequilibrio de clases, se pueden aplicar los cuatro métodos de balanceo vistos. Finalmente, se define la arquitectura de la red neuronal, se entrena y se evalúa para obtener los resultados. Se han realizado dos modelos con las redes neuronales: uno de ellos con dos capas ocultas de 32 y 64 capas, mientras que en el otro modelo se han utilizado 64 y 128 como número de neuronas.

Componente/Hiperparámetro	**Especificación/Valor**	**Propósito**
Arquitectura del Modelo (Sequential)		
Capa Densa 1 (`Dense`)	64 (32) neuronas, activación `relu`	Primera capa oculta. `input_shape` derivado de `X_train_cc.shape[1]`.
`Dropout` 1	Tasa 0.3	Regularización para prevenir sobreajuste después de la capa densa 1.
Capa Densa 2 (`Dense`)	128 (64) neuronas, activación `relu`	Segunda capa oculta.
`Dropout` 2	Tasa 0.3	Regularización para prevenir sobreajuste después de la capa densa 2.
Capa de Salida (`Dense`)	`num_classes` neuronas, activación `softmax`	Capa de salida para clasificación multiclase.
Configuración de Compilación (`model.compile`)		
Optimizador (`optimizer`)	`adam`	Algoritmo para actualizar los pesos de la red durante el entrenamiento.
Función de Pérdida (`loss`)	`categorical_crossentropy`	Mide el error del modelo, adecuada para clasificación multiclase con etiquetas one-hot.
Métricas (`metrics`)	`['accuracy']`	Métrica(s) para monitorear el rendimiento durante el entrenamiento y la evaluación.
Callbacks de Entrenamiento (`EarlyStopping`)		
`EarlyStopping`	Parámetros abajo:	Técnica para detener el entrenamiento prematuramente si no hay mejora.
`patience`	5	Número de épocas a esperar sin mejora en la métrica monitoreada antes de detener.
`restore_best_weights`	`True`	Restaura los pesos del modelo correspondientes a la mejor época observada.

Tabla 5.16. *Hiperparámetros y configuración para el modelo de Red Neuronal.*

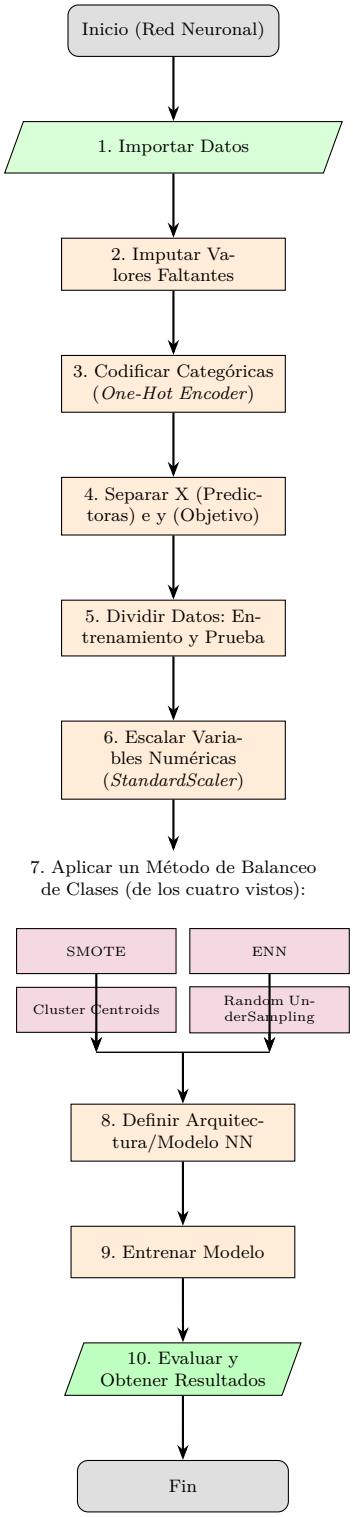


Figura 5.17: Diagrama de flujo para el entrenamiento de Redes Neuronales.

6. Resultados

En esta sección se proporcionarán los diferentes resultados obtenidos del entrenamiento de los diferentes modelos. Además de exponerlos, se explicará el porqué de esos resultados y porque un modelo obtiene mejores resultados que otro. Se mencionará también la forma en que se pueden mejorar estos resultados y como varían.

Con el objetivo de mantener la comparabilidad entre modelos, se aplicaron técnicas de rebalanceo en todos aquellos algoritmos que no cuentan con mecanismos internos robustos para tratar el desbalanceo de clases. En los modelos que sí disponen de estas funcionalidades (como CatBoost o LightGBM), se optó por utilizar sus opciones nativas de balanceo de pesos, ya que representan la forma más eficiente y estandarizada de tratamiento del problema en dichos algoritmos.

6.1. *CatBoost*

Las siguientes tablas resumen las métricas obtenidas, permitiendo visualizar de manera clara el rendimiento global y por clase. Se incluyen tanto los resultados de la validación cruzada como los del conjunto de prueba, lo que facilita comprobar la consistencia y la capacidad de generalización del modelo.

Métrica	Valor Promedio
Accuracy	0,8405
MCC	0,7872
ROC AUC (*Macro*)	0,9677

Tabla 6.1. *Resultados promedio de cross-validation (5 folds)*

Clase	*Precision*	Recall	*F1-score*	*Support*
Riesgo alto	0,9334	0,9583	0,9457	7827
Riesgo bajo	0,7750	0,7395	0,7568	8563
Riesgo medio	0,8570	0,8461	0,8515	9251
Sin riesgo	0,8576	0,8916	0,8742	7617
Accuracy			0,8555	33258
Macro Avg	0,8558	0,8589	0,8571	33258
Weighted Avg	0,8540	0,8555	0,8545	33258

Tabla 6.2. *Reporte de clasificación en el conjunto de entrenamiento (CatBoost)*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,9231	0,9509	0,9368	1957
Riesgo bajo	0,7436	0,7276	0,7355	2140
Riesgo medio	0,8383	0,8137	0,8258	2313
Sin riesgo	0,8509	0,8755	0,8631	1904
Accuracy			0,8380	8314
Macro Avg	0,8390	0,8419	0,8403	8314
Weighted Avg	0,8368	0,8380	0,8372	8314

Tabla 6.3. *Reporte de clasificación en el conjunto de prueba (*CatBoost*)*

Métrica	Valor
MCC	0,7838
ROC AUC (*Macro*)	0,9657
Tiempo de ejecución (s)	22,77

Tabla 6.4. *Métricas adicionales del conjunto de prueba y tiempo de ejecución (CatBoost)*

Un *Accuracy* de **0,8380** indica que el modelo clasifica correctamente aproximadamente el 84 % de las instancias de prueba. El ROC AUC (*Macro*), tanto en *cross-validation* (**0,9677**) como en el conjunto de prueba (**0,9657**), es excelente, lo que muestra que el modelo diferencia bien entre las diferentes clases. El Coeficiente de Correlación de Matthews (MCC) en el conjunto de prueba es de **0,7838**, un buen indicador del rendimiento global del clasificador multiclase.

Las métricas promedio de la *cross-validation* (*Avg Accuracy*: 0,8405, Avg MCC: 0,7872, Avg ROC AUC: 0,9677) son notablemente cercanas a las obtenidas en el conjunto de prueba final, lo que indica que el modelo generaliza bien a datos no vistos y que los resultados de la validación cruzada son una estimación fiable del rendimiento esperado. El modelo no se encuentra sobreajustado.

La clase Riesgo alto y Sin riesgo muestran un rendimiento bastante bueno tanto en entrenamiento como en prueba, con *Precision*, *Recall* y *F1-score* consistentemente altos. Por otro lado, se observa una pequeña disminución en el rendimiento del conjunto de entrenamiento al de prueba (e.g., Train *Accuracy*: 0,8555 vs. Test *Accuracy*: 0,8380).

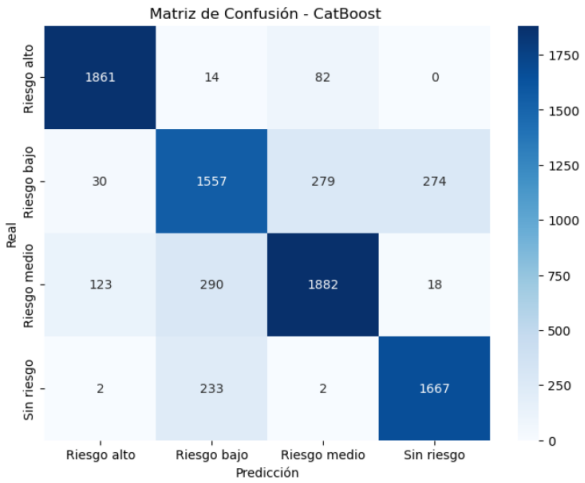


Figura 6.1: Matriz de confusión - *CatBoost*

Si se atiende ahora a los resultados de la matriz de confusión superior, se puede obtener que el modelo es eficaz identificando la clase "Riesgo alto", con muy pocas instancias erróneas hacia categorías de menor riesgo y, notablemente, ninguna hacia "Sin riesgo". La clase **"Sin riesgo"** también es bien identificada, aunque un número considerable (233) se clasifica erróneamente como "Riesgo bajo".

Las mayores áreas de **confusión** se encuentran entre las clases adyacentes. En concreto, "Riesgo bajo" es frecuentemente confundido con "Riesgo medio" (279 instancias) y "Sin riesgo" (274 instancias), mientras que "Riesgo medio" es a su vez confundido con "Riesgo bajo" (290 instancias).

6.2. *DecisionTree*

6.2.1. Sin balanceo

Las tablas siguientes recogen de manera detallada las métricas globales y por clase obtenidas para el modelo *DecisionTree* sin *SMOTE* y sin escalado. Se incluyen tanto los resultados de validación cruzada como los del conjunto de prueba, lo que permite comprobar la consistencia entre ellos y confirmar la capacidad de generalización del modelo.

De forma complementaria, también se muestran las métricas específicas por clase (*Precision*, *Recall* y *F1-score*), que ayudan a identificar en qué categorías el clasificador tiene un mejor rendimiento y en cuáles presenta más dificultades.

Métrica	Valor Promedio
Accuracy	0,7652
MCC	0,6863
ROC AUC (*Macro*)	0,9333

Tabla 6.5. *Resultados promedio de cross-validation (5 folds) - DecisionTree*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,9275	0,9057	0,9165	7827
Riesgo bajo	0,6343	0,6304	0,6323	8563
Riesgo medio	0,7515	0,7699	0,7606	9251
Sin riesgo	0,7998	0,8010	0,8004	7617
Accuracy			0,7730	33258
Macro Avg	0,7783	0,7767	0,7774	33258
Weighted Avg	0,7738	0,7730	0,7734	33258

Tabla 6.6. *Reporte de clasificación en el conjunto de entrenamiento - DecisionTree sin SMOTE sin escalado.*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,9204	0,8922	0,9061	1957
Riesgo bajo	0,6217	0,6290	0,6253	2140
Riesgo medio	0,7362	0,7566	0,7463	2313
Sin riesgo	0,7963	0,7841	0,7902	1904
Accuracy			0,7620	8314
Macro Avg	0,7686	0,7655	0,7670	8314
Weighted Avg	0,7639	0,7620	0,7628	8314

Tabla 6.7. *Reporte de clasificación en el conjunto de prueba - DecisionTree sin SMOTE sin escalado.*

Métrica	Valor
MCC	0,6819
ROC AUC (*Macro*)	0,9315
Tiempo de ejecución (s)	1

Tabla 6.8. *Métricas adicionales del conjunto de prueba y tiempo de ejecución - DecisionTree sin SMOTE sin escalado.*

Los resultados para el modelo *DecisionTree* sin *SMOTE* y sin escalado muestran un rendimiento decente, aunque con áreas mejorables. Un `Accuracy` de **0,7620** indica que el modelo clasifica correctamente más del 76 % de las instancias de prueba. El `ROC AUC (Macro)`, tanto en *cross-validation* (**0,9333**) como en el conjunto de prueba (**0,9315**), es bastante bueno. El Coeficiente de Correlación de Matthews (`MCC`) en el conjunto de prueba es de **0,6819**, lo que indica un rendimiento predictivo aceptable para un clasificador multiclase. Las métricas entre *cross-validation*, entrenamiento y

prueba son muy parecidas, lo que aporta una gran ventaja, ya que sugiere que el modelo *DecisionTree*, en esta configuración, no está sobreajustando los datos de entrenamiento y generaliza de manera muy predecible a datos no vistos.

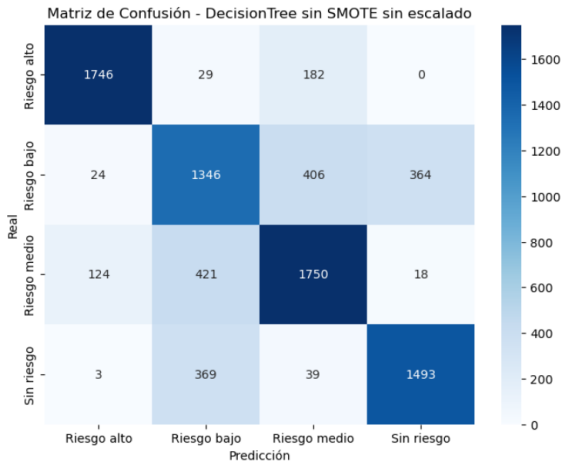


Figura 6.2: Matriz de confusión - *DecisionTree* (Sin balanceo)

El modelo *DecisionTree*, sin *SMOTE* ni escalado, sigue siendo eficaz en evitar la clasificación errónea de "Riesgo alto" como "Sin riesgo", lo cual es positivo. La clase "Riesgo bajo" es claramente el punto más débil del modelo. Un total de 406 + 364 = 770 instancias de "Riesgo bajo" son mal clasificadas como "Riesgo medio" o "Sin riesgo". Existe una confusión significativa entre "Riesgo bajo" y "Riesgo medio": 406 de "Riesgo bajo" van a "Riesgo medio", y 421 de "Riesgo medio" van a "Riesgo bajo". Esto sugiere que el modelo tiene serias dificultades para distinguir estas dos categorías. La clasificación errónea de "Sin riesgo" como "Riesgo bajo" (369 instancias) es también un error notable, indicando que el modelo tiende a ser algo más conservador o impreciso al identificar instancias que realmente no tienen riesgo.

6.2.2. Oversampling

Las tablas que se presentan a continuación muestran de forma detallada las métricas globales y por clase obtenidas para el modelo *DecisionTree* con *SMOTE* y sin escalado. Se incluyen tanto los resultados de validación cruzada como los del conjunto de prueba, lo que permite comprobar si el uso de esta técnica de sobremuestreo mejora realmente la capacidad de generalización del modelo.

Asimismo, se presentan las métricas específicas por clase (*Precision*, *Recall* y *F1-score*), que permiten apreciar en qué medida el balanceo de datos influye en el rendimiento de cada categoría en particular.

Métrica	Valor Promedio
Accuracy	0,7719
MCC	0,6960
ROC AUC (*Macro*)	0,9362

Tabla 6.9. *Resultados promedio de cross-validation (5 folds) - DecisionTree con SMOTE.*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,9328	0,9132	0,9229	9251
Riesgo bajo	0,6467	0,5872	0,6155	9251
Riesgo medio	0,7352	0,7652	0,7499	9251
Sin riesgo	0,7837	0,8402	0,8110	9251
Accuracy			0,7765	37004
Macro Avg	0,7746	0,7765	0,7748	37004
Weighted Avg	0,7746	0,7765	0,7748	37004

Tabla 6.10. *Reporte de clasificación en el conjunto de entrenamiento - DecisionTree con SMOTE y sin escalado.*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,9186	0,8942	0,9063	1957
Riesgo bajo	0,6323	0,5883	0,6095	2140
Riesgo medio	0,7367	0,7536	0,7450	2313
Sin riesgo	0,7563	0,8151	0,7846	1904
Accuracy			0,7582	8314
Macro Avg	0,7610	0,7628	0,7614	8314
Weighted Avg	0,7572	0,7582	0,7572	8314

Tabla 6.11. *Reporte de clasificación en el conjunto de prueba - DecisionTree con SMOTE y sin escalado.*

Métrica	Valor
MCC	0,6774
ROC AUC (*Macro*)	0,9304
Tiempo de ejecución (s)	1,06

Tabla 6.12. *Métricas adicionales del conjunto de prueba y tiempo de ejecución - DecisionTree con SMOTE y sin escalado.*

La aplicación de *SMOTE* al modelo *DecisionTree* (sin escalado) presenta resultados interesantes, y no necesariamente una mejora directa en todas las métricas del conjunto de prueba en comparación con la versión sin *SMOTE*. Las métricas entre *cross-validation* (CV), entrenamiento y prueba siguen siendo bastante consistentes, aunque las cifras de CV son ligeramente mejores que las de prueba. Esto indica que el modelo, incluso con *SMOTE*, no sobreajusta excesivamente y generaliza de forma predecible, aunque el rendimiento en prueba es ligeramente inferior al del entrena-

miento. La clase `Riesgo alto` mantiene un buen rendimiento en prueba (P: 0,9186, R: 0,8942, F1: 0,9063), similar a la versión sin *SMOTE*. Por otro lado, la clase `Sin riesgo` muestra un *Recall* ligeramente mejorado en prueba (0,8151 con SMOTE vs. 0,7841 sin *SMOTE*), aunque con una ligera caída en *Precision*.

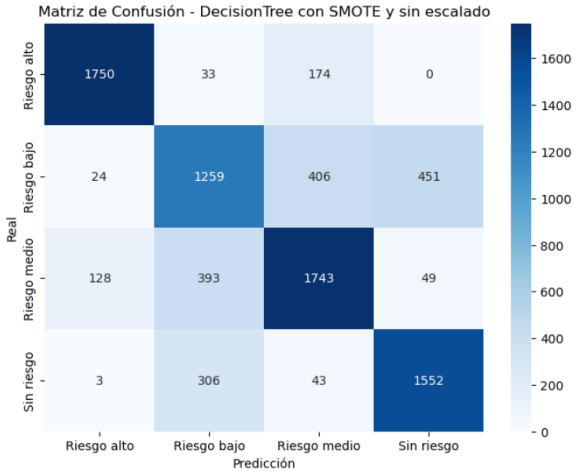


Figura 6.3: Matriz de confusión - *DecisionTree* (SMOTE)

La aplicación de *SMOTE* fue perjudicial, empeorando la clasificación de la clase "Riesgo bajo", que era la que más necesitaba mejora. El principal efecto negativo fue un aumento en la confusión de "Riesgo bajo" con "Sin riesgo", un error considerable. En resumen, la técnica introdujo más ruido que claridad, creando fronteras de decisión más ambiguas para el modelo.

6.2.3. *Undersampling*

Las tablas que se presentan a continuación recogen las métricas globales y por clase obtenidas al aplicar técnicas de submuestreo. Se comparan los resultados de validación cruzada y del conjunto de prueba, lo que permite observar cómo varía el rendimiento de ENN frente al submuestreo aleatorio.

Estas tablas permiten analizar con detalle el comportamiento de cada técnica, destacando especialmente el efecto en la clase "Riesgo bajo", que resulta ser la más afectada por la calidad del rebalanceo aplicado.

Métrica	Valor Promedio
Accuracy	0,7703
MCC	0,6943
ROC AUC (*Macro*)	0,9352

Tabla 6.13. *Resultados promedio de cross-validation (5 Folds) - DecisionTree con Undersampling.*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,9323	0,9077	0,9198	7617
Riesgo bajo	0,6188	0,6566	0,6371	7617
Riesgo medio	0,7319	0,7685	0,7498	7617
Sin riesgo	0,8398	0,7687	0,8027	7617
Accuracy			0,7754	30468
Macro Avg	0,7807	0,7754	0,7774	30468
Weighted Avg	0,7807	0,7754	0,7774	30468

Tabla 6.14. *Reporte de clasificación en el conjunto de entrenamiento - DecisionTree con Undersampling.*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,9187	0,8947	0,9065	1957
Riesgo bajo	0,6154	0,6617	0,6377	2140
Riesgo medio	0,7344	0,7544	0,7443	2313
Sin riesgo	0,8250	0,7500	0,7857	1904
Accuracy			0,7626	8314
Macro Avg	0,7734	0,7652	0,7686	8314
Weighted Avg	0,7679	0,7626	0,7645	8314

Tabla 6.15. *Reporte de clasificación en el conjunto de prueba - DecisionTree con Undersampling.*

Métrica	Valor
MCC	0,6828
ROC AUC (*Macro*)	0,9305
Tiempo de ejecución (s)	0,74

Tabla 6.16. *Métricas adicionales (prueba) y tiempo - DecisionTree con Undersampling.*

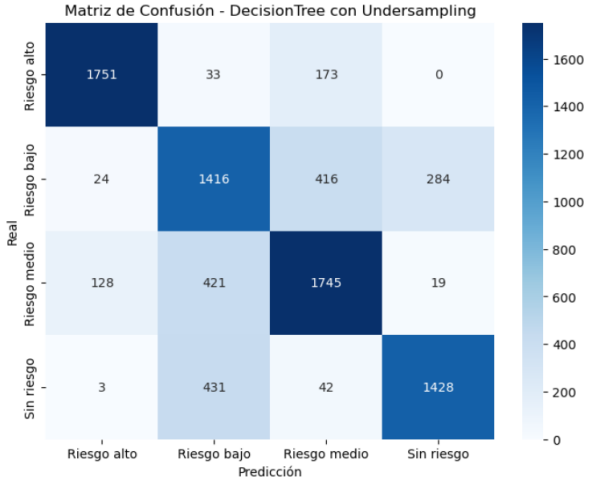


Figura 6.4: Matriz de confusión - *DecisionTree* (*Undersampling*)

Métrica	Valor promedio
Accuracy	0,8596
MCC	0,8093
ROC AUC (*Macro*)	0,9639

Tabla 6.17. *Resultados promedio de cross-validation (5 Folds) - DecisionTree con ENN.*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,9048	0,9044	0,9046	7617
Riesgo bajo	0,6972	0,4395	0,5392	7617
Riesgo medio	0,6930	0,7843	0,7358	7617
Sin riesgo	0,7292	0,9028	0,8068	7617
Accuracy			0,7578	30468
Macro Avg	0,7560	0,7578	0,7466	30468
Weighted Avg	0,7560	0,7578	0,7466	30468

Tabla 6.18. *Reporte de clasificación en el conjunto de entrenamiento - DecisionTree con ENN.*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,8855	0,8968	0,8911	1957
Riesgo bajo	0,6701	0,4262	0,5210	2140
Riesgo medio	0,7054	0,7773	0,7396	2313
Sin riesgo	0,7007	0,8913	0,7846	1904
Accuracy			0,7412	8314
Macro Avg	0,7404	0,7479	0,7341	8314
Weighted Avg	0,7376	0,7412	0,7293	8314

Tabla 6.19. *Reporte de clasificación en el conjunto de prueba - DecisionTree con ENN.*

Métrica	Valor
MCC	0,6607
ROC AUC (*Macro*)	0,9216
Tiempo de ejecución (s)	0,45

Tabla 6.20. *Métricas adicionales (prueba) y tiempo - DecisionTree con ENN.*

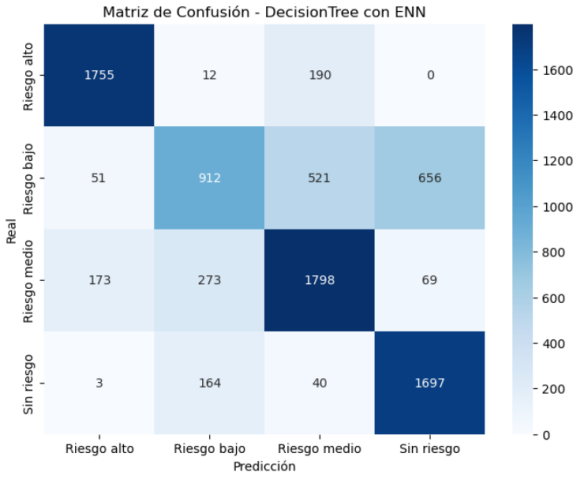


Figura 6.5: Matriz de confusión - *DecisionTree* (ENN)

Métrica	Valor promedio
Accuracy	0,7489
MCC	0,6661
ROC AUC (*Macro*)	0,9266

Tabla 6.21. *Resultados promedio de cross-validation (5 Folds) - DecisionTree con Cluster Centroids.*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,9234	0,9022	0,9127	7617
Riesgo bajo	0,6121	0,6228	0,6174	7617
Riesgo medio	0,7302	0,7764	0,7526	7617
Sin riesgo	0,8237	0,7762	0,7992	7617
Accuracy			0,7694	30468
Macro Avg	0,7724	0,7694	0,7705	30468
Weighted Avg	0,7724	0,7694	0,7705	30468

Tabla 6.22. *Reporte de clasificación en el conjunto de entrenamiento - DecisionTree con Cluster Centroids.*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,9045	0,8958	0,9001	1957
Riesgo bajo	0,5997	0,6168	0,6082	2140
Riesgo medio	0,7426	0,7570	0,7497	2313
Sin riesgo	0,8019	0,7652	0,7831	1904
Accuracy			0,7555	8314
Macro Avg	0,7622	0,7587	0,7603	8314
Weighted Avg	0,7575	0,7555	0,7563	8314

Tabla 6.23. *Reporte de clasificación en el conjunto de prueba - DecisionTree con Cluster Centroids.*

Métrica	Valor
MCC	0,6732
ROC AUC (*Macro*)	0,9281
Tiempo de ejecución (s)	0,44

Tabla 6.24. *Métricas adicionales (prueba) y tiempo - DecisionTree con Cluster Centroids.*

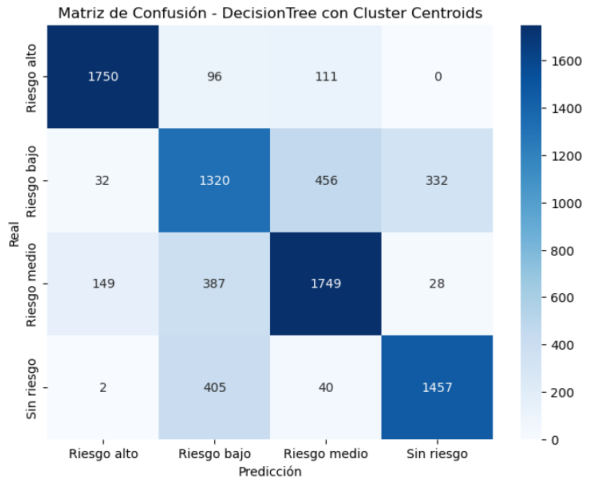


Figura 6.6: Matriz de confusión - *DecisionTree* (CC)

En validación cruzada (CV), la técnica ENN muestra una superioridad notable, alcanzando las mejores métricas promedio con una *Accuracy* de 0,8596, un MCC de 0,8093 y un ROC AUC de 0,9639. Sin embargo, en el conjunto de prueba (Test), ENN, a pesar de su éxito en CV, obtiene el peor rendimiento con una *Accuracy* de 0,7412 y un MCC de 0,6607. En contraste, el submuestreo aleatorio se muestra como el más robusto, liderando con una *Accuracy* de 0,7626 y un MCC de 0,6828, lo que indica un claro sobreajuste por parte de ENN. El análisis detallado por clase confirma este problema: su rendimiento es especialmente deficiente para la clase "Riesgo bajo", registrando un *Recall* de apenas 0,4262, lo que significa que falla en identificar correctamente a más de la mitad de las instancias de esta categoría. El submuestreo aleatorio, en cambio, ofrece el mejor compromiso, alcanzando un *Recall* de 0,6617 y un *F1-score* de 0,6377 para esta

misma clase. En conjunto, el submuestreo aleatorio demuestra la mayor consistencia entre entrenamiento y prueba, posicionándose como la opción más equilibrada, mientras que ENN se descarta por su severo sobreajuste y su impacto negativo en clases críticas.

6.3. KNN

6.3.1. *Oversampling*

Las tablas que se muestran a continuación recogen las métricas globales y por clase obtenidas para el modelo KNN con $k = 4$ y *SMOTE*. Se incluyen los resultados tanto en entrenamiento como en prueba, lo que permite observar con claridad el fuerte sobreajuste del modelo y, al mismo tiempo, la mejora en la clasificación de la clase "Riesgo bajo", que aunque sigue siendo la más complicada, alcanza un *F1-score* superior respecto a otras configuraciones.

Métrica de Optimización	Valor
Mejor valor de k (según CV)	4
F1-*Macro* promedio en CV (k=4)	0,8190

Tabla 6.25. *Optimización del Hiperparámetro k para KNN mediante cross-validation.*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,9991	0,9991	0,9991	9251
Riesgo bajo	0,9884	0,9918	0,9901	9251
Riesgo medio	0,9935	0,9922	0,9929	9251
Sin riesgo	0,9974	0,9952	0,9963	9251
Accuracy			0,9946	37004
Macro Avg	0,9946	0,9946	0,9946	37004
Weighted Avg	0,9946	0,9946	0,9946	37004

Tabla 6.26. *Reporte de clasificación en el conjunto de entrenamiento - KNN con SMOTE (k=4).*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,8434	0,8585	0,8508	1957
Riesgo bajo	0,7198	0,6899	0,7045	2141
Riesgo medio	0,7895	0,7899	0,7897	2313
Sin riesgo	0,8314	0,8545	0,8428	1904
Accuracy			0,7951	8315
Macro Avg	0,7960	0,7982	0,7970	8315
Weighted Avg	0,7938	0,7951	0,7943	8315

Tabla 6.27. *Reporte de clasificación en el conjunto de prueba - KNN con SMOTE (k=4).*

Métrica	Valor
MCC	0,7264
ROC AUC (*Macro*)	0,9275
Tiempo de ejecución (s)	4,45

Tabla 6.28. *Métricas adicionales del conjunto de prueba y tiempo de ejecución - KNN con SMOTE (k=4).*

La característica más destacada del modelo es un gran sobreajuste. Esto se puede ver en un rendimiento casi perfecto en el conjunto de entrenamiento (*Accuracy* de 0,9946) y un resultado inferior en el de prueba (*Accuracy* de 0,7951). Esta diferencia informa que el modelo, una combinación de KNN con k=4 y SMOTE, ha memorizado los datos de entrenamiento en lugar de aprender patrones generalizables. A pesar del sobreajuste, el rendimiento en datos no vistos es sólido y competitivo, con un MCC de 0,7264 y un ROC AUC *Macro* de 0,9275. Un resultado muy positivo es la mejora sustancial en la clasificación de la clase "Riesgo bajo". Alcanzar un *F1-score* de 0,7045 en prueba para esta categoría es un avance significativo. Aun así, sigue siendo la clase con el rendimiento más bajo, lo que confirma su complejidad en este conjunto de datos.

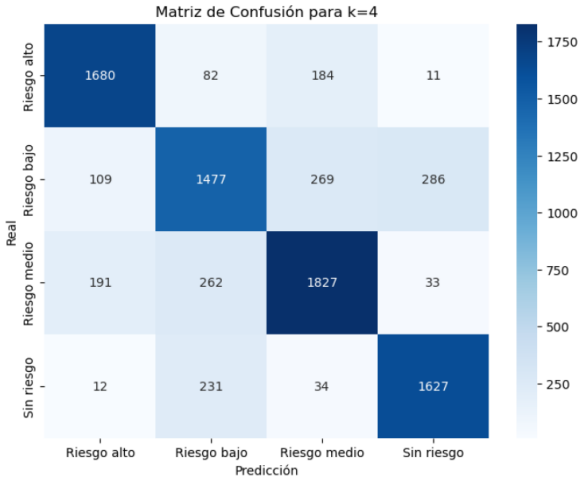


Figura 6.7: Matriz de confusión - KNN (SMOTE)

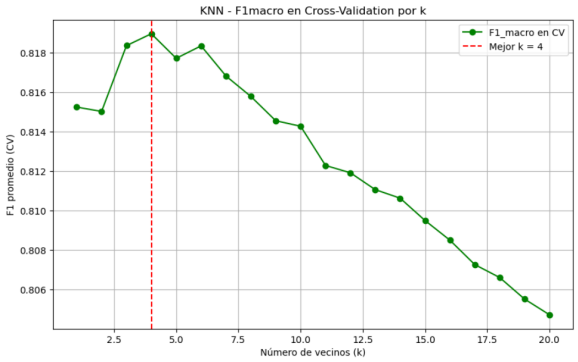


Figura 6.8: F1*Macro* en CV - KNN

La gráfica de optimización confirma que k=4 es la elección óptima, al ser el punto que maximiza el F1-*Macro* promedio durante la validación cruzada, justificando así el hiperparámetro seleccionado. A pesar de la optimización y el uso de SMOTE, la matriz revela que la clase "Riesgo bajo" sigue siendo el principal punto débil, con una fuga considerable de 286 instancias hacia la categoría "Sin riesgo". El modelo comete un error particularmente grave al clasificar 11 instancias de "Riesgo alto" como "Sin riesgo". Este tipo de error, aunque poco frecuente, es una consecuencia directa de un modelo que podría estar sobreajustado y es menos conservador. Se mantiene el patrón de confusión entre clases adyacentes (Bajo ↔ Medio, Medio ↔ Alto), lo que indica que el modelo KNN, al igual que los anteriores, lucha por definir fronteras nítidas entre categorías contiguas.

6.3.2. *Undersampling*

Las tablas y figuras que se presentan a continuación permite visualizar en detalle los errores de clasificación cometidos por el modelo y comparar el impacto de las distintas técnicas de submuestreo aplicadas. En particular, resulta útil para identificar los fallos críticos en la detección de "Riesgo alto" y para evaluar hasta qué punto cada estrategia mejora la clasificación de la clase "Riesgo bajo", que históricamente ha resultado la más difícil de diferenciar.

Métrica de Optimización	Valor
Mejor valor de k (según CV)	6
F1-*Macro* promedio en CV (k=6)	0,7953

Tabla 6.29. *Optimización de k para KNN con Undersampling.*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,9989	0,9992	0,9991	7617
Riesgo bajo	0,9854	0,9926	0,9890	7617
Riesgo medio	0,9945	0,9895	0,9920	7617
Sin riesgo	0,9974	0,9947	0,9961	7617
Accuracy			0,9940	30468
Macro Avg	0,9940	0,9940	0,9940	30468
Weighted Avg	0,9940	0,9940	0,9940	30468

Tabla 6.30. *Reporte de clasificación (entrenamiento) - KNN con Undersampling (k=6).*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,8287	0,8579	0,8431	1957
Riesgo bajo	0,7265	0,6861	0,7057	2141
Riesgo medio	0,7892	0,7851	0,7872	2313
Sin riesgo	0,8296	0,8566	0,8429	1904
Accuracy			0,7931	8315
Macro Avg	0,7935	0,7965	0,7947	8315
Weighted Avg	0,7916	0,7931	0,7921	8315

Tabla 6.31. *Reporte de clasificación (prueba) - KNN con Undersampling (k=6).*

Métrica	Valor
MCC	0,7240
ROC AUC (*Macro*)	0,9369
Tiempo de ejecución (s)	4,27

Tabla 6.32. *Métricas adicionales (prueba) y tiempo - KNN con Undersampling (k=6).*

Métrica de Optimización	Valor
Mejor valor de k (según CV)	8
F1-*Macro* promedio en CV (k=8)	0,7748

Tabla 6.33. *Optimización de k para KNN con Cluster Centroids.*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,9988	0,9997	0,9993	7617
Riesgo bajo	0,9853	0,9928	0,9890	7617
Riesgo medio	0,9958	0,9888	0,9923	7617
Sin riesgo	0,9967	0,9951	0,9959	7617
Accuracy			0,9941	30468
Macro Avg	0,9941	0,9941	0,9941	30468
Weighted Avg	0,9941	0,9941	0,9941	30468

Tabla 6.34. *Reporte de clasificación (entrenamiento) - KNN con Cluster Centroids (k=8).*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,8445	0,8549	0,8497	1957
Riesgo bajo	0,7293	0,6908	0,7095	2141
Riesgo medio	0,7777	0,8003	0,7888	2313
Sin riesgo	0,8401	0,8498	0,8449	1904
Accuracy			0,7963	8315
Macro Avg	0,7979	0,7989	0,7982	8315
Weighted Avg	0,7953	0,7963	0,7956	8315

Tabla 6.35. *Reporte de clasificación (prueba) - KNN con Cluster Centroids (k=8).*

Métrica	Valor
MCC	0,7279
ROC AUC (*Macro*)	0,9424
Tiempo de ejecución (s)	13,40

Tabla 6.36. *Métricas adicionales (prueba) y tiempo - KNN con Cluster Centroids (k=8).*

Métrica de Optimización	Valor
Mejor valor de k (según CV)	1
F1-*Macro* promedio en CV (k=1)	0,9662

Tabla 6.37. *Optimización de k para KNN con ENN.*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	1	1	1	5209
Riesgo bajo	0,9978	1	0,9989	3704
Riesgo medio	1	1	1	5830
Sin riesgo	1	0,9989	0,9995	7617
Accuracy			0,9996	22360
Macro Avg	0,9995	0,9997	0,9996	22360
Weighted Avg	0,9996	0,9996	0,9996	22360

Tabla 6.38. *Reporte de clasificación (entrenamiento) - KNN con ENN (k=1).*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,8059	0,8273	0,8164	1957
Riesgo bajo	0,7525	0,5213	0,6159	2141
Riesgo medio	0,7770	0,7549	0,7658	2313
Sin riesgo	0,6929	0,9375	0,7969	1904
Accuracy			0,7536	8315
Macro Avg	0,7571	0,7602	0,7487	8315
Weighted Avg	0,7583	0,7536	0,7462	8315

Tabla 6.39. *Reporte de clasificación (prueba) - KNN con ENN (k=1).*

Métrica	Valor
MCC	0,6777
ROC AUC (*Macro*)	0,8392
Tiempo de ejecución (s)	10,44

Tabla 6.40. *Métricas adicionales (prueba) y tiempo - KNN con ENN (k=1).*

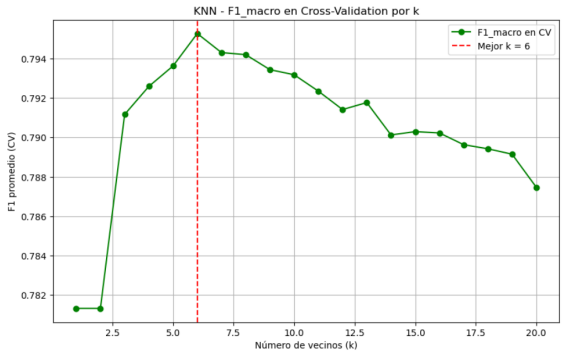


Figura 6.9: F1*Macro* en CV - KNN (*Undersampling*)

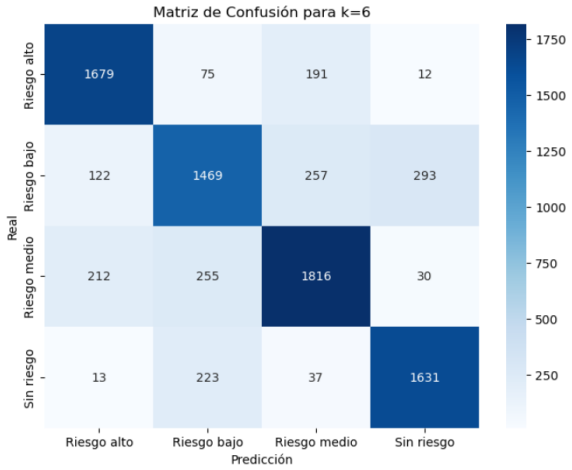


Figura 6.10: Matriz de confusión - KNN (*Undersampling*)

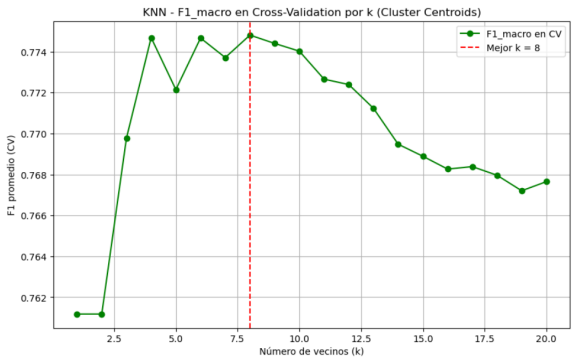


Figura 6.11: F1*Macro* en CV - KNN (CC)

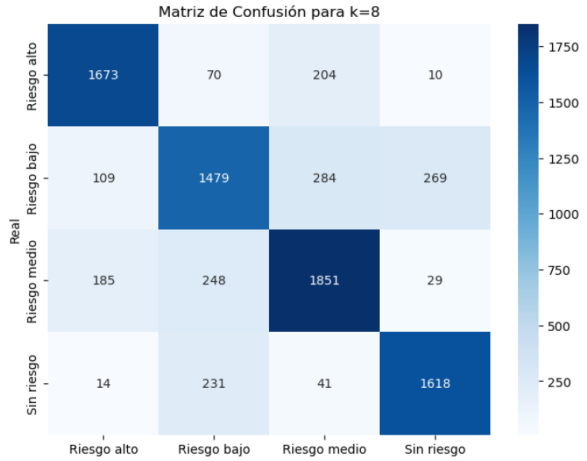


Figura 6.12: Matriz de confusión - KNN (CC)

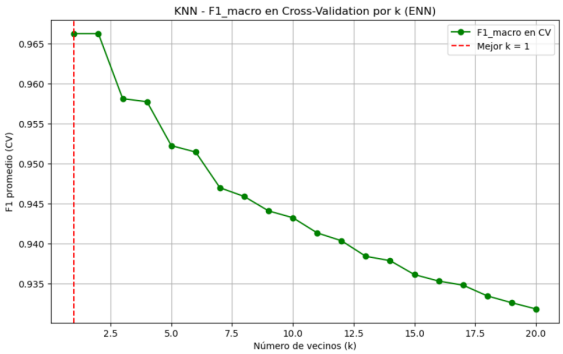


Figura 6.13: F1*Macro* en CV - KNN (ENN)

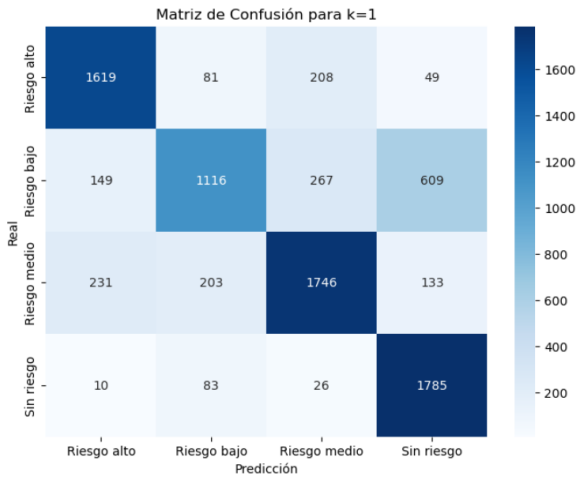


Figura 6.14: Matriz de confusión - KNN (ENN)

Aunque esta configuración alcanza el pico más alto en la validación cruzada, su rendimiento en el conjunto de prueba demuestra una incapacidad para generalizar. La matriz de confusión es contundente: no solo es el peor modelo para la clase "Riesgo bajo" (con solo 1116 verdaderos positivos y una masiva fuga de 609 instancias hacia "Sin riesgo"), sino que también comete una alarmante cantidad de errores críticos, clasificando erróneamente 49 instancias de "Riesgo alto" como "Sin riesgo". En contraste directo, tanto el submuestreo aleatorio (con k=6) como Cluster Centroids (con k=8) ofrecen un rendimiento mucho más equilibrado y fiable. Sus valores de 'k' óptimos, más moderados, ofrecen modelos con fronteras de decisión menos complejas y, por lo tanto, más robustas. Ambas técnicas manejan la clase "Riesgo bajo" de manera comparable y mucho más efectiva que ENN (logrando aproximadamente 1470 verdaderos positivos). De manera crucial, son significativamente más seguras, manteniendo los errores críticos ("Riesgo alto" → "Sin riesgo") en un nivel más bajo y aceptable (12 y 10 casos, respectivamente). Tanto el submuestreo aleatorio como Cluster Centroids muestran patrones de confusión similares y esperables, con la principal dificultad en la separación de clases adyacentes. Considerando que Cluster Centroids mostró métricas globales ligeramente superiores en los reportes previos y comete un número marginalmente menor de errores críticos en la matriz de confusión, podría considerarse como la opción preferente. No obstante, ambas representan una mejora sustancial y son soluciones viables.

6.4. LightGBM

Las tablas siguiente recogen las métricas globales y por clase obtenidas para este modelo, mostrando su excelente capacidad de generalización y, en particular, la notable mejora alcanzada en la clase "Riesgo bajo", tradicionalmente la más difícil de clasificar.

Resultados promedio de *cross-validation* (5 *Folds*) - *LightGBM*				
Clase	Avg *Precision*	Avg *Recall*	Avg *F1-score*	Avg Logloss
Riesgo alto	0,9400	0,9580	0,9500	
Riesgo bajo	0,7780	0,7780	0,7760	
Riesgo medio	0,8720	0,8480	0,8600	
Sin riesgo	0,8680	0,8800	0,8760	
Avg Accuracy			**0,8620**	**0,340419**
Macro Avg	0,8620	0,8640	0,8640	
Weighted Avg	0,8620	0,8620	0,8620	

Tabla 6.41. *Resultados promedio de cross-validation (5 Folds) - LightGBM sin balanceo.*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,9543	0,9734	0,9638	7827
Riesgo bajo	0,8256	0,8210	0,8233	8562
Riesgo medio	0,9021	0,8796	0,8907	9251
Sin riesgo	0,8944	0,9088	0,9015	7617
Accuracy			0,8933	33257
Macro Avg	0,8941	0,8957	0,8948	33257
Weighted Avg	0,8929	0,8933	0,8930	33257

Tabla 6.42. *Reporte de clasificación en el conjunto de entrenamiento - LightGBM sin balanceo.*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,9444	0,9545	0,9494	1957
Riesgo bajo	0,7836	0,7697	0,7766	2141
Riesgo medio	0,8706	0,8642	0,8674	2313
Sin riesgo	0,8695	0,8850	0,8771	1904
Accuracy			0,8659	8315
Macro Avg	0,8670	0,8684	0,8677	8315
Weighted Avg	0,8653	0,8659	0,8656	8315

Tabla 6.43. *Reporte de clasificación en el conjunto de prueba - LightGBM sin balanceo.*

Métrica	Valor
MCC	0,8209
ROC AUC (*Macro*)	0,9754
Tiempo de ejecución (s)	211,59

Tabla 6.44. *Métricas adicionales del conjunto de prueba y tiempo de ejecución - LightGBM sin balanceo.*

El modelo alcanza un nivel de rendimiento sobresaliente en el conjunto de prueba, con una precisión del 86.59 %, un MCC de 0,8209 y un ROC AUC de 0,9754. Se observa una alta consistencia entre las métricas de validación cruzada (Avg. *Accuracy* de 0,8620) y las de prueba, lo que confirma que el modelo generaliza de manera muy eficaz. El sobreajuste es moderado y totalmente aceptable, evidenciado por la pequeña brecha entre el rendimiento de entrenamiento (89.33 %) y el de prueba. Destaca especialmente la mejora sustancial en la clase "Riesgo bajo", que alcanza un *F1-score* de 0,7766.

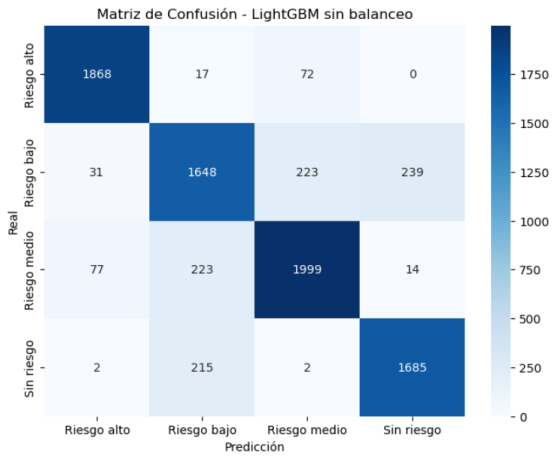


Figura 6.15: Matriz de confusión - *LightGBM*

Se aprecia una reducción clara de la confusión de la clase "Riesgo bajo" con sus adyacentes ("Riesgo medio" y "Sin riesgo"). El modelo demuestra ser extremadamente fiable al mantener en cero el número de instancias de "Riesgo alto" clasificadas erróneamente como "Sin riesgo", una característica fundamental para la confianza en el sistema. Los errores que persisten se concentran lógicamente en estas fronteras adyacentes, evitando saltos ilógicos entre clases distantes y demostrando un comportamiento predictivo más coherente.

6.5. MLP

6.5.1. *Oversampling*

Las tablas siguiente presentan las métricas globales y por clase obtenidas con la red neuronal, evidenciando tanto el fuerte sobreajuste como el elevado coste computacional de esta configuración, así como las dificultades persistentes en la clasificación de la clase "Riesgo bajo".

Parámetro	Valor
Mejor config. capas ocultas (CV KFold=5)	(100, 100)
Técnica de balanceo aplicada	SMOTE

Tabla 6.45. *Configuración del modelo red neuronal (MLP) y preprocesamiento*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,9819	0,9894	0,9856	9251
Riesgo bajo	0,9132	0,9521	0,9323	9251
Riesgo medio	0,9655	0,9337	0,9493	9251
Sin riesgo	0,9750	0,9581	0,9665	9251
Accuracy			0,9583	37004
Macro Avg	0,9589	0,9583	0,9584	37004
Weighted Avg	0,9589	0,9583	0,9584	37004

Tabla 6.46. *Reporte de clasificación en el conjunto de entrenamiento (MLP con SMOTE)*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,9198	0,9198	0,9198	1957
Riesgo bajo	0,7123	0,7585	0,7347	2141
Riesgo medio	0,8204	0,8020	0,8111	2313
Sin riesgo	0,8635	0,8241	0,8433	1904
Accuracy			0,8236	8315
Macro Avg	0,8290	0,8261	0,8272	8315
Weighted Avg	0,8258	0,8236	0,8244	8315

Tabla 6.47. *Reporte de clasificación en el conjunto de prueba (MLP con SMOTE)*

Métrica	Valor
MCC (Test)	0,7645
ROC AUC (*Macro*, Test)	0,9540
Tiempo de ejecución (s)	631,01

Tabla 6.48. *Métricas adicionales del conjunto de prueba y tiempo de ejecución (MLP con SMOTE)*

La arquitectura de red neuronal seleccionada mediante validación cruzada fue de dos capas ocultas, ambas con 100 neuronas, una configuración razonable para muchos problemas de clasificación. Se utilizó SMOTE para balancear las clases en el conjunto de entrenamiento, lo que se evidencia en el ***`Support`*** idéntico (9251) para todas las clases en el reporte de entrenamiento. La característica más definitoria de este modelo de red neuronal es un **marcado sobreajuste**, observable en una brecha de rendimiento de aproximadamente 13.5 puntos entre la precisión casi perfecta en el conjunto de entrenamiento (0,9583) y el resultado en el de prueba (0,8236). Este fenómeno sugiere que la red, potenciada por SMOTE, ha memorizado eficazmente el conjunto de entrenamiento aumentado, pero le cuesta generalizar dichos patrones a datos no vistos con la misma eficacia.

A pesar del sobreajuste, el rendimiento en el conjunto de prueba es robusto, con una precisión del 82.36 % y un MCC de 0,7645. Un punto fuerte destacable es su excelente ROC AUC de 0,9540, indicativo de una gran capacidad para discriminar entre clases. No obstante, el desafío histórico con la clase “Riesgo bajo” persiste de manera

notable: con un *F1-score* de 0,73, sigue siendo la categoría con el rendimiento más bajo. Aunque su *Recall* es aceptable (0,7585), su precisión es más limitada (0,7123), lo que significa que una proporción considerable de sus predicciones para esta clase son incorrectas. Finalmente, un factor práctico determinante es su **elevado coste computacional**, con un tiempo de entrenamiento que supera los 10 minutos (631 segundos).

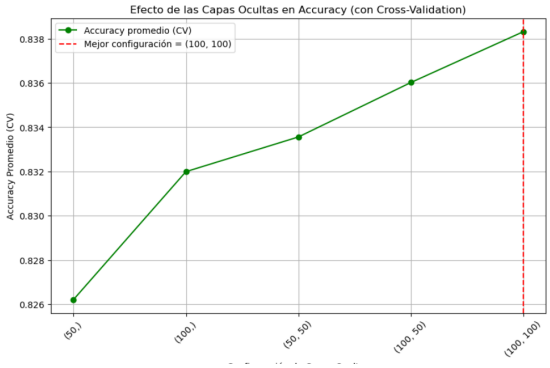


Figura 6.16: Capas ocultas MLP - SMOTE

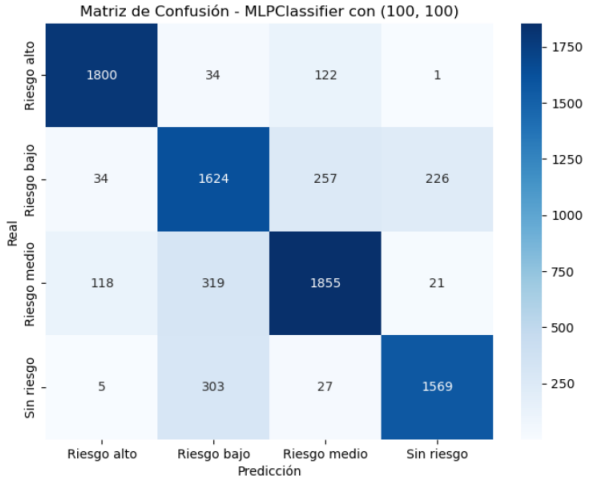


Figura 6.17: Matriz de confusión - MLP (SMOTE)

La primera gráfica, "Efecto de las Capas Ocultas en *Accuracy*" (Figura 6.16), ilustra cómo se seleccionó la arquitectura (100, 100) como la óptima entre las configuraciones probadas, alcanzando el *Accuracy* promediomás alto (≈0,8385) en la validación cruzada.

La segunda gráfica, la "Matriz de confusión - MLPClassifier con (100, 100)" (Figura 6.17), detalla el rendimiento de este modelo seleccionado cuando se enfrenta al conjunto de prueba final (con su desbalance original de clases y un total de 8.315 muestras). Si bien el modelo identifica correctamente un número considerable de instancias en la diagonal (e.g., 1.800 para "Riesgo alto", 1624 para "Riesgo bajo"), también revela las dificultades específicas:

- La clase **“Riesgo bajo”** es la que presenta mayor confusión, siendo erróneamente clasificada como “Riesgo medio” (257 instancias) y “Sin riesgo” (226 instancias) con notable frecuencia.
- También se observan confusiones significativas para **“Riesgo medio”** (principalmente con “Riesgo bajo”, 319 instancias) y para **“Sin riesgo”** (principalmente con “Riesgo bajo”, 303 instancias).
- La clase **“Riesgo alto”** es la más distintiva, aunque presenta cierta confusión con “Riesgo medio” (122 instancias).

6.5.2. *Undersampling*

Las tablas siguientes muestran de forma comparativa los resultados de los tres modelos MLP evaluados con diferentes técnicas de submuestreo. Se incluyen métricas globales y por clase, lo que permite comprobar el mejor desempeño de Cluster Centroids y, al mismo tiempo, analizar las limitaciones de ENN y del submuestreo aleatorio.

Parámetro	Valor
Mejor config. capas ocultas (CV)	(100,)
Técnica de balanceo aplicada	*Undersampling*

Tabla 6.49. *Configuración del modelo red neuronal (MLP) y preprocesamiento (Undersampling)*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,9369	0,9668	0,9516	7617
Riesgo bajo	0,8086	0,8133	0,8110	7617
Riesgo medio	0,8760	0,8509	0,8633	7617
Sin riesgo	0,9086	0,9005	0,9045	7617
Accuracy			0,8829	30468
Macro Avg	0,8825	0,8829	0,8826	30468
Weighted Avg	0,8825	0,8829	0,8826	30468

Tabla 6.50. *Reporte de clasificación en el conjunto de entrenamiento (MLP con Undersampling)*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,8998	0,9453	0,9220	1957
Riesgo bajo	0,7262	0,7146	0,7203	2141
Riesgo medio	0,8213	0,7946	0,8077	2313
Sin riesgo	0,8396	0,8440	0,8418	1904
Accuracy			0,8208	8315
Macro Avg	0,8217	0,8246	0,8230	8315
Weighted Avg	0,8195	0,8208	0,8199	8315

Tabla 6.51. *Reporte de clasificación en el conjunto de prueba (MLP con Undersampling)*

Métrica	Valor
MCC (Test)	0,7609
ROC AUC (*Macro*, Test)	0,9590
Tiempo de ejecución (s)	221,80

Tabla 6.52. *Métricas adicionales del conjunto de prueba y tiempo de ejecución (MLP con Undersampling)*

Parámetro	Valor
Mejor config. capas ocultas (ENN)	(100, 50)
Técnica de balanceo aplicada	ENN (Edited Nearest Neighbours)

Tabla 6.53. *Configuración del modelo red neuronal (MLP) y preprocesamiento (ENN)*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,9981	1	0,9990	5209
Riesgo bajo	0,9609	0,9960	0,9781	3704
Riesgo medio	0,9979	0,9964	0,9972	5830
Sin riesgo	0,9988	0,9810	0,9898	7617
Accuracy			0,9919	22360
Macro Avg	0,9889	0,9933	0,9910	22360
Weighted Avg	0,9921	0,9919	0,9919	22360

Tabla 6.54. *Reporte de clasificación en el conjunto de entrenamiento (MLP con ENN)*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,8842	0,9167	0,9002	1957
Riesgo bajo	0,7667	0,6156	0,6829	2141
Riesgo medio	0,8215	0,7860	0,8034	2313
Sin riesgo	0,7468	0,9233	0,8257	1904
Accuracy			0,8043	8315
Macro Avg	0,8048	0,8104	0,8030	8315
Weighted Avg	0,8050	0,8043	0,8002	8315

Tabla 6.55. *Reporte de clasificación en el conjunto de prueba (MLP con ENN)*

Métrica	Valor
MCC (Test)	0,7421
ROC AUC (*Macro*, Test)	0,9463
Tiempo de ejecución (s)	17,68

Tabla 6.56. *Métricas adicionales del conjunto de prueba y tiempo de ejecución (MLP con ENN)*

Parámetro	Valor
Mejor config. capas ocultas (CC + CV)	(100,)
Técnica de balanceo aplicada	Cluster Centroids

Tabla 6.57. *Configuración del modelo red neuronal (MLP) y preprocesamiento (Cluster Centroids)*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,9449	0,9523	0,9486	7617
Riesgo bajo	0,7708	0,8071	0,7886	7617
Riesgo medio	0,8603	0,8225	0,8410	7617
Sin riesgo	0,8982	0,8883	0,8932	7617
Accuracy			0,8676	30468
Macro Avg	0,8686	0,8676	0,8678	30468
Weighted Avg	0,8686	0,8676	0,8678	30468

Tabla 6.58. *Reporte de clasificación en el conjunto de entrenamiento (MLP con Cluster Centroids)*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,9282	0,9310	0,9296	1957
Riesgo bajo	0,7113	0,7412	0,7260	2141
Riesgo medio	0,8237	0,7998	0,8116	2313
Sin riesgo	0,8491	0,8361	0,8426	1904
Accuracy			0,8239	8315
Macro Avg	0,8281	0,8271	0,8274	8315
Weighted Avg	0,8252	0,8239	0,8244	8315

Tabla 6.59. *Reporte de clasificación en el conjunto de prueba (MLP con Cluster Centroids)*

Métrica	Valor
MCC (Test)	0,7649
ROC AUC (*Macro*, Test)	0,9595
Tiempo de ejecución (s)	40,10

Tabla 6.60. *Métricas adicionales del conjunto de prueba y tiempo de ejecución (MLP con Cluster Centroids)*

De los tres modelos MLP evaluados con diferentes técnicas de submuestreo, el modelo que utiliza **Cluster Centroids** consistentemente muestra el mejor rendimiento general en el conjunto de prueba (*Accuracy*, MCC, ROC AUC y *F1-scores* por clase, especialmente para la difícil clase "Riesgo bajo"). Además, presenta el menor grado de sobreajuste y un tiempo de ejecución muy razonable. En cambio, el modelo con **ENN** sufre de un sobreajuste severo, lo que resulta en el peor rendimiento de generalización, a pesar de ser el más rápido de entrenar. Esto sugiere que ENN, en su configuración actual, podría estar eliminando información crucial o creando un conjunto de entrenamiento demasiado simplificado que no representa la complejidad del problema real. El modelo con ***Undersampling* aleatorio** ofrece un rendimiento intermedio, pero es superado por Cluster Centroids en la mayoría de las métricas clave y muestra un mayor sobreajuste. Para todos los modelos, la clase "Riesgo bajo" sigue siendo la más desafiante, aunque Cluster Centroids parece manejarla relativamente mejor que las otras dos técnicas de submuestreo.

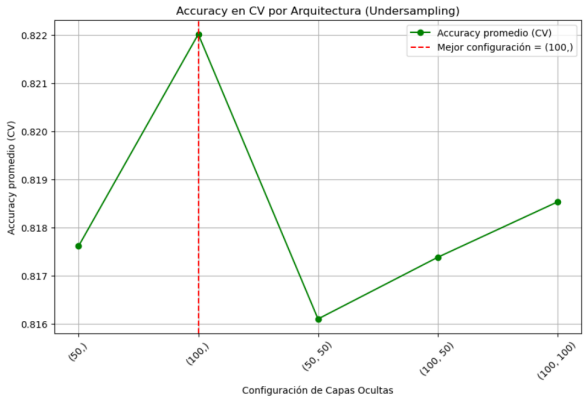


Figura 6.18: Evolución neuronas MLP (*Undersampling*)

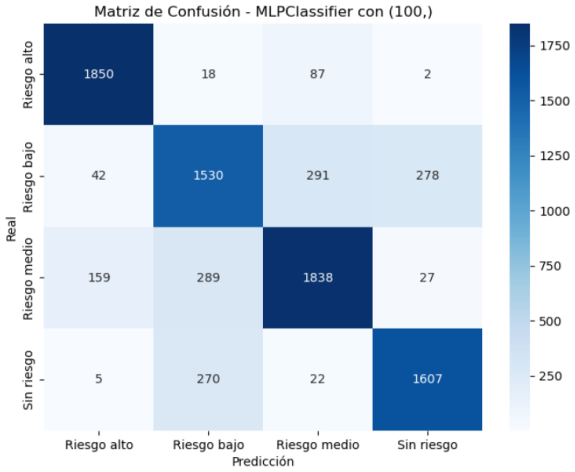


Figura 6.19: Matriz de confusión - MLP (*Undersampling*)

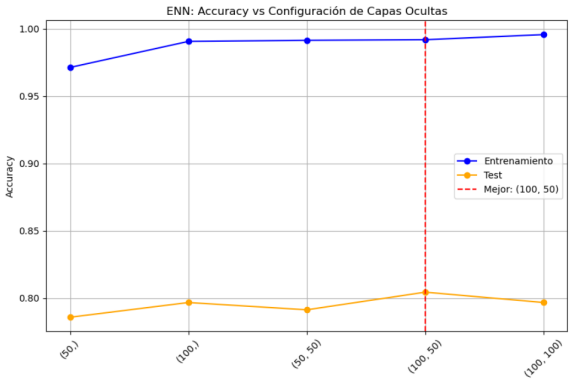


Figura 6.20: Evolución neuronas MLP (ENN)

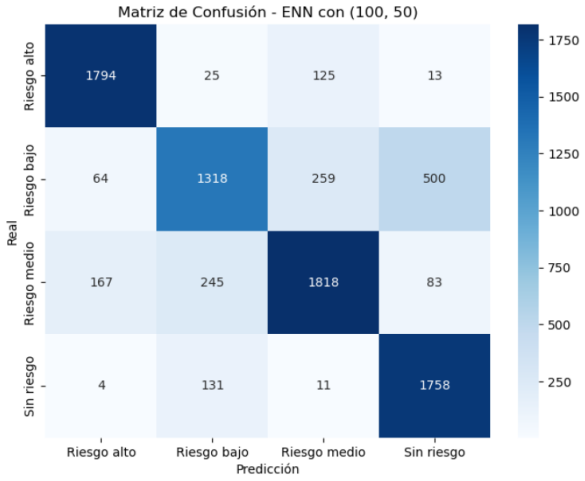


Figura 6.21: Matriz de confusión - MLP (ENN)

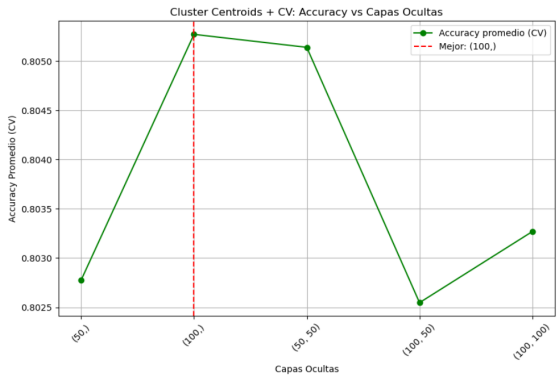


Figura 6.22: Evolución neuronas MLP (CC)

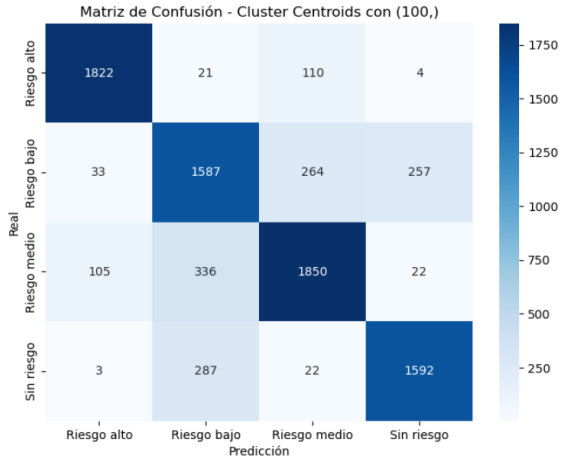


Figura 6.23: Matriz de confusión - MLP (CC)

Las gráficas de CV muestran que la arquitectura óptima del MLP varía según la técnica de submuestreo. Para *Undersampling* aleatorio y Cluster Centroids, una sola capa de 100 neuronas fue óptima, mientras que para ENN se seleccionó una arquitectura de dos capas (100, 50) basada en el rendimiento de prueba/validación (a pesar del severo sobreajuste en entrenamiento). Aunque el modelo aprende el dataset "limpiadoçasi a la perfección, su generalización es la más pobre, y su matriz de confusión revela problemas críticos, especialmente la masiva confusión de "Riesgo bajo" hacia "Sin riesgo". Sin embargo, el modelo MLP con **Cluster Centroids** logra el mayor número de clasificaciones correctas (1587) para "Riesgo bajo" en el conjunto de prueba, y sus confusiones son menos drásticas que con ENN.

6.6. *RandomForest*

6.6.1. Sin balanceo

Las tablas siguientes presentan las métricas globales y por clase de este modelo, evidenciando su gran consistencia entre validación cruzada, entrenamiento y prueba, pero también sus limitaciones en rendimiento global y, en particular, en la clasificación de la clase "Riesgo bajo".

Métrica promedio(CV KFold=5)	Valor
Accuracy	0,7620
Macro Precision	0,7600
Macro Recall	0,7680
Macro F1-score	0,7620

Tabla 6.61. *Resultados promedio de cross-validation (RandomForest)*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,8538	0,9492	0,8990	7827
Riesgo bajo	0,6511	0,5922	0,6202	8562
Riesgo medio	0,7557	0,7425	0,7491	9251
Sin riesgo	0,7953	0,8019	0,7986	7616
Accuracy			0,7660	33256
Macro Avg	0,7640	0,7714	0,7667	33256
Weighted Avg	0,7609	0,7660	0,7625	33256

Tabla 6.62. *Reporte de clasificación en el conjunto de entrenamiento (RandomForest)*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,8575	0,9377	0,8958	1957
Riesgo bajo	0,6457	0,5857	0,6143	2141
Riesgo medio	0,7440	0,7588	0,7513	2313
Sin riesgo	0,7940	0,7815	0,7877	1904
Accuracy			0,7615	8315
Macro Avg	0,7603	0,7659	0,7623	8315
Weighted Avg	0,7568	0,7615	0,7584	8315

Tabla 6.63. *Reporte de clasificación en el conjunto de prueba (RandomForest)*

Métrica	Valor
MCC (Test)	0,6820
ROC AUC (*Macro*, Test)	0,9360
Tiempo de ejecución (s)	11,90

Tabla 6.64. *Métricas adicionales del conjunto de prueba y tiempo de ejecución (RandomForest)*

El punto más fuerte del modelo es su notable estabilidad. La precisión obtenida es prácticamente idéntica en las tres fases de evaluación: validación cruzada (0,7620), entrenamiento (0,7660) y prueba (0,7615). Esta consistencia es una clara indicación de que el modelo generaliza de manera predecible y no sufre de sobreajuste, lo que lo convierte en una opción fiable. Sin embargo, a pesar de esta estabilidad, el rendimiento predictivo del modelo es solo aceptable. Su precisión en prueba (0,7615) y su MCC (0,6820) son significativamente inferiores a algunos de los modelos anteriores. Su capacidad discriminativa, aunque buena (ROC AUC de 0,9360), tampoco alcanza la de las mejores alternativas. La principal limitación del modelo se concentra, una vez más, en la clase "Riesgo bajo". Esta categoría muestra un desempeño muy pobre, con un *F1-score* de apenas 0,6143. El problema es especialmente agudo en el *Recall* (0,5857), lo que significa que el modelo es incapaz de identificar correctamente a más del 40 % de los casos reales pertenecientes a esta clase.

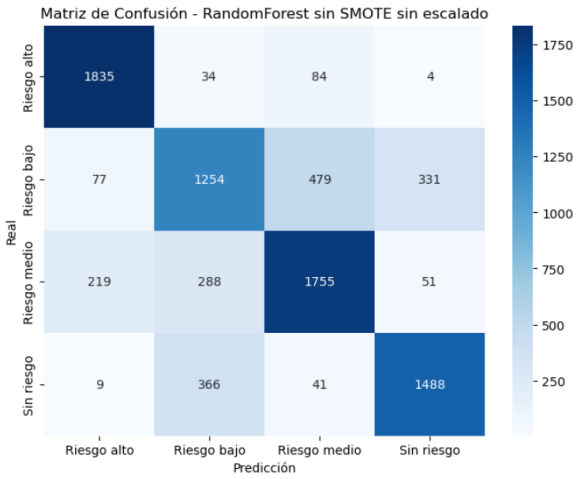


Figura 6.24: Matriz de confusión - *RandomForest*

- La matriz de confusión del modelo *RandomForest* (sin SMOTE, sin escalado) confirma visualmente que, aunque el modelo es estable y generaliza consistentemente (como se vio en el bajo sobreajuste), su capacidad para distinguir la clase "Riesgo bajo" de las clases "Riesgo medio" y "Sin riesgo" es limitada.
- El número absoluto de errores para "Riesgo bajo" (77+479+331 = 887 instancias mal clasificadas de 2141) es muy alto.

6.6.2. *Oversampling*

Las tablas siguientes muestran los resultados obtenidos tras aplicar SMOTE, permitiendo comparar las métricas globales y por clase con las de la versión sin balanceo. Se observa claramente que la técnica no mejora el rendimiento y que la clase "Riesgo bajo" continúa siendo la más problemática.

Métrica promedio(CV KFold=5)	Valor
Accuracy	0,7700
Macro Precision	0,7660
Macro Recall	0,7700
Macro F1-score	0,7660

Tabla 6.65. *Resultados promedio de cross-validation (RandomForest con SMOTE)*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,8743	0,9501	0,9106	9251
Riesgo bajo	0,6437	0,5874	0,6143	9251
Riesgo medio	0,7382	0,7432	0,7407	9251
Sin riesgo	0,8143	0,8094	0,8118	9251
Accuracy			0,7725	37004
Macro Avg	0,7676	0,7725	0,7693	37004
Weighted Avg	0,7676	0,7725	0,7693	37004

Tabla 6.66. *Reporte de clasificación en el conjunto de entrenamiento (RandomForest con SMOTE)*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,8547	0,9382	0,8945	1957
Riesgo bajo	0,6471	0,5806	0,6120	2141
Riesgo medio	0,7439	0,7575	0,7506	2313
Sin riesgo	0,7911	0,7857	0,7884	1904
Accuracy			0,7609	8315
Macro Avg	0,7592	0,7655	0,7614	8315
Weighted Avg	0,7559	0,7609	0,7575	8315

Tabla 6.67. *Reporte de clasificación en el conjunto de prueba (RandomForest con SMOTE)*

Métrica	Valor
MCC (Test)	0,6813
ROC AUC (*Macro*, Test)	0,9358
Tiempo de ejecución (s)	5,77

Tabla 6.68. *Métricas adicionales del conjunto de prueba y tiempo de ejecución (RandomForest con SMOTE)*

Las métricas clave de rendimiento en el conjunto de prueba, como la precisión (0,7609) y el MCC (0,6813), son prácticamente idénticas a las obtenidas por el modelo sin balanceo, lo que demuestra que la técnica no aportó ningún beneficio tangible en el resultado final. El principal objetivo de aplicar SMOTE, que era mejorar el rendimiento en la clase más problemática ("Riesgo bajo"), no se ha cumplido. De hecho, el *F1-score* para esta clase se mantiene en un nivel muy bajo (0,6120) e incluso con retroceso en el *Recall*.

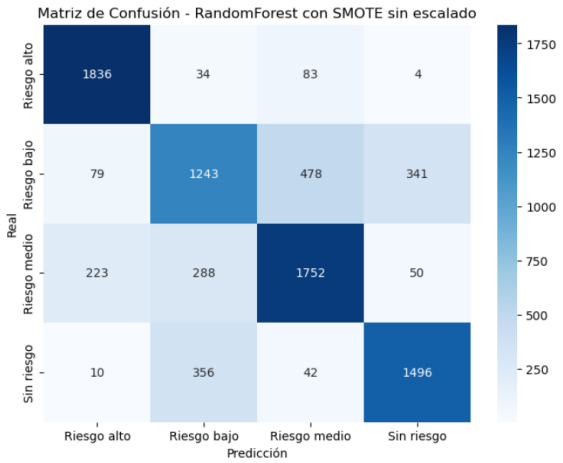


Figura 6.25: Matriz de confusión - *RandomForest* (SMOTE)

La matriz de confusión del *RandomForest* con SMOTE y sin escalado ilustra de manera efectiva que, si bien el modelo tiene un desempeño razonable para la clase "Riesgo alto", sufre significativamente en la clasificación de "Riesgo bajo", confundiéndola frecuentemente con "Riesgo medio" y "Sin riesgo".

6.6.3. *Undersampling*

Las tablas siguientes muestran los resultados obtenidos por los modelos *RandomForest* con distintas técnicas de submuestreo. En ellas se pueden comparar las métricas globales y por clase, confirmando que el *Undersampling* general ofrece el mejor equilibrio, aunque la clase "Riesgo bajo" continúa siendo la más problemática.

Técnica de Submuestreo	Avg Acc (CV)	Avg P (CV)	Avg R (CV)	Avg F1 (CV)
Undersampling General	0,7640	0,7620	0,7640	0,7620
ENN	0,8620	0,8620	0,8120	0,8180
Cluster Centroids	0,7500	0,7460	0,7500	0,7460

Tabla 6.69. *Resultados promedio de Cross-Validation (RandomForest con Submuestreo)*

	entrenamiento				prueba			
Clase	P	R	F1	Sup	P	R	F1	Sup
Riesgo alto	0,871	0,947	0,908	7617	0,856	0,938	0,895	1957
Riesgo bajo	0,644	0,597	0,619	7617	0,652	0,589	0,618	2141
Riesgo medio	0,744	0,747	0,746	7617	0,750	0,763	0,756	2313
Sin riesgo	0,816	0,800	0,808	7617	0,794	0,785	0,790	1904
Accuracy			0,7729	30468			0,7643	8315
Macro Avg	0,769	0,773	0,770		0,763	0,769	0,765	
MCC (Test): 0,6857, ROC AUC (Test): 0,9367, Tiempo: 5,32s								

Tabla 6.70. *Resumen de RandomForest con Undersampling General*

	entrenamiento				prueba			
Clase	P	R	F1	Sup	P	R	F1	Sup
Riesgo alto	0,912	0,981	0,945	5884	0,808	0,940	0,869	1957
Riesgo bajo	0,863	0,431	0,575	3682	0,760	0,269	0,397	2141
Riesgo medio	0,844	0,895	0,869	5966	0,731	0,758	0,744	2313
Sin riesgo	0,845	0,960	0,899	7617	0,624	0,945	0,752	1904
Accuracy			0,8644	23149			0,7177	8315
Macro Avg	0,866	0,817	0,822		0,731	0,728	0,691	
MCC (Test): 0,6435, ROC AUC (Test): 0,9338, Tiempo: 3,28s								

Tabla 6.71. *Resumen de RandomForest con ENN*

	entrenamiento				prueba			
Clase	P	R	F1	Sup	P	R	F1	Sup
Riesgo alto	0,881	0,933	0,906	7617	0,883	0,924	0,903	1957
Riesgo bajo	0,616	0,554	0,583	7617	0,647	0,570	0,606	2141
Riesgo medio	0,705	0,733	0,718	7617	0,725	0,783	0,753	2313
Sin riesgo	0,797	0,797	0,797	7617	0,793	0,785	0,789	1904
Accuracy			0,7542	30468			0,7616	8315
Macro Avg	0,749	0,754	0,751		0,762	0,765	0,763	
MCC (Test): 0,6821, ROC AUC (Test): 0,9355, Tiempo: 3,51s								

Tabla 6.72. *Resumen de RandomForest con Cluster Centroids*

A diferencia de los MLP, donde Cluster Centroids parecía más prometedor, para *RandomForest* el ***Undersampling* general** ha resultado ser la técnica de submuestreo más efectiva, logrando el mejor rendimiento general en prueba y un manejo decente de la clase "Riesgo bajo", aunque esta sigue siendo la más débil. **Cluster Centroids** se presenta como una alternativa muy sólida, con la ventaja de no mostrar prácticamente ningún sobreajuste y un rendimiento en prueba muy cercano al *Undersampling* general y al RF base. Además, alcanzó el mejor *F1-score* para la clase "Riesgo alto". En contraste, **ENN** no es adecuado para *RandomForest* en este escenario: aunque mostró un alto rendimiento en validación cruzada (posiblemente por aprender un dataset muy "limpioz simplificado), no generaliza bien al conjunto de prueba original, especialmente perjudicando a la clase "Riesgo bajo".

La clase "Riesgo bajo" sigue siendo el principal desafío para todos los modelos *RandomForest*, independientemente de la técnica de submuestreo aplicada, con *F1-scores* que rondan el 0,60–0,62 en el mejor de los casos. Comparado con *CatBoost* (Test Acc ≈0,838, MCC ≈0,78, ROC AUC ≈0,96), todos estos modelos *RandomForest*, incluso con submuestreo, siguen teniendo un rendimiento predictivo general inferior.

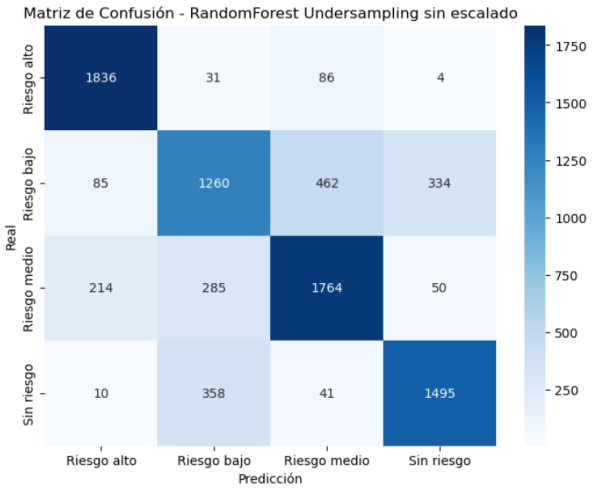


Figura 6.26: Matriz de confusión - *RandomForest* (*Undersampling*)

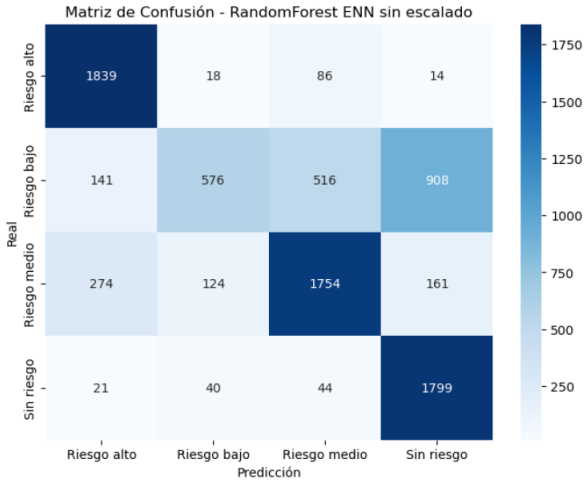


Figura 6.27: Matriz de confusión - *RandomForest* (ENN)

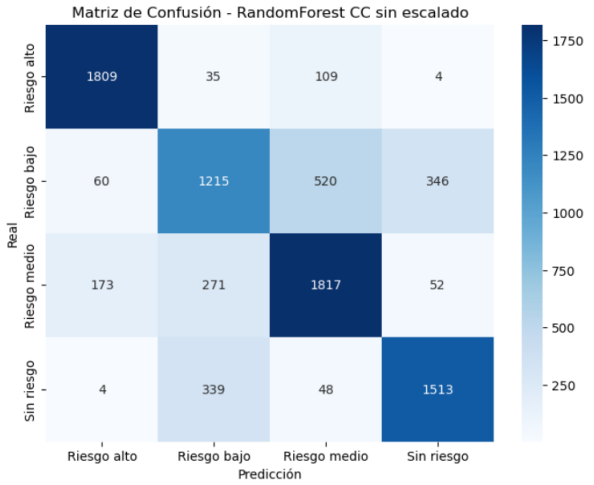


Figura 6.28: Matriz de confusión - *RandomForest* (CC)

Se confirma que el alto rendimiento de ENN en validación cruzada fue engañoso, mientras que el submuestreo aleatorio y Cluster Centroids son, en efecto, las estrategias más equilibradas. La matriz de confusión del modelo entrenado con datos procesados por ENN revela su deficiente capacidad de generalización. El colapso es particularmente severo en la clase "Riesgo bajo", que es masivamente mal clasificada, confirmando que este enfoque, aunque prometedor en CV, no es viable en la práctica.

6.7. SVM

6.7.1. *Oversampling*

Las tablas siguientes recogen las métricas globales y por clase del modelo SVM con SMOTE y C=10, mostrando su gran competitividad frente a otros enfoques y, en particular, la mejora significativa en la detección de la clase "Riesgo bajo".

Parámetro / Métrica de CV	Valor
Valor de C (seleccionado/usado)	10
Accuracy en CV (KFold)	0,8409 ± 0,0022
Tiempo de *Cross-Validation* (s)	2442,44
Técnica de balanceo aplicada	SMOTE

Tabla 6.73. *Configuración del modelo SVM,resultados de Cross-Validation y preprocesamiento*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,9534	0,9720	0,9626	9251
Riesgo bajo	0,8438	0,7893	0,8156	9251
Riesgo medio	0,8791	0,8821	0,8806	9251
Sin riesgo	0,8885	0,9255	0,9067	9251
Accuracy			0,8922	37004
Macro Avg	0,8912	0,8922	0,8914	37004
Weighted Avg	0,8912	0,8922	0,8914	37004

Tabla 6.74. *Reporte de clasificación en el conjunto de entrenamiento (SVM con SMOTE, C=10)*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,9131	0,9341	0,9235	1957
Riesgo bajo	0,7627	0,7099	0,7354	2141
Riesgo medio	0,8151	0,8327	0,8238	2313
Sin riesgo	0,8472	0,8708	0,8588	1904
Accuracy			0,8337	8315
Macro Avg	0,8345	0,8369	0,8354	8315
Weighted Avg	0,8320	0,8337	0,8325	8315

Tabla 6.75. *Reporte de clasificación en el conjunto de prueba (SVM con SMOTE, C=10)*

Métrica	Valor
MCC (Test)	0,7781
ROC AUC (*Macro*, Test)	0,9628
Tiempo de ejecución (s)	789,76

Tabla 6.76. *Métricas adicionales del conjunto de prueba y tiempo de ejecución (SVM con SMOTE, C=10)*

El modelo SVM con SMOTE y C=10 ofrece un rendimiento predictivo general muy bueno y competitivo, con un Test *Accuracy*, MCC y ROC AUC que se encuentran entre los más altos de los modelos probados, superando a los *RandomForest* y a varios de los MLP. Es particularmente notable que el *F1-score* para la clase "Riesgo bajo" (0,7354) es uno de los mejores obtenidos para esta clase difícil, lo que sugiere que la combinación SVM+SMOTE es efectiva para mejorar su identificación.

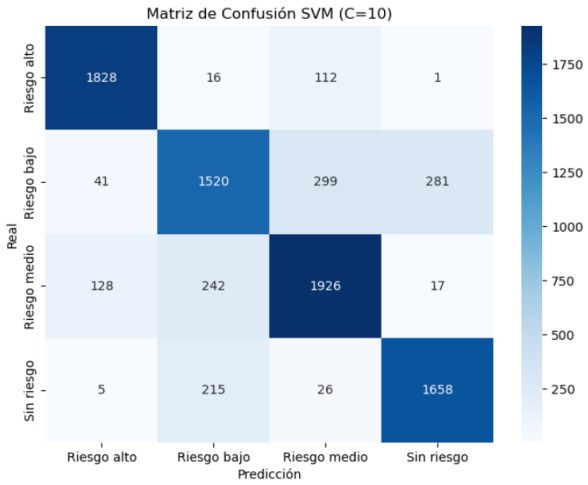


Figura 6.29: Matriz de confusión - SVM (SMOTE)

La matriz de confusión del modelo SVM (C=10, con SMOTE) demuestra un rendimiento de clasificación general robusto y equilibrado en comparación con muchos de los otros modelos, especialmente los basados en *RandomForest* o algunos MLP con submuestreo.

Aunque la clase “Riesgo bajo” sigue siendo la más difícil de clasificar con precisión, este modelo SVM logra un manejo notablemente mejor de ella que la mayoría de las alternativas, reduciendo la magnitud de las confusiones. Las clases “Riesgo alto” y “Sin riesgo” son clasificadas con alta efectividad, y “Riesgo medio” también muestra un buen rendimiento.

Esta visualización, junto con las métricas numéricas (alto MCC de 0,7781 y ROC AUC de 0,9628), posiciona a este SVM como uno de los modelos más prometedores en términos de rendimiento predictivo para este problema, a pesar de su mayor coste computacional.

6.7.2. *Undersampling*

Las tablas siguientes muestran los resultados obtenidos para los modelos SVM con distintas técnicas de submuestreo (Undersampling general, Cluster Centroids y ENN). Se incluyen métricas globales y por clase, lo que permite comparar el equilibrio entre rendimiento, sobreajuste y coste computacional de cada enfoque.

Parámetro / Métrica de CV	Valor
Valor de C (usado)	10
Accuracy en CV (KFold)	0,8290 ± 0,0037
Técnica de balanceo aplicada	*Undersampling* General

Tabla 6.77. *Configuración del modelo SVM y resultados de CV (Undersampling)*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,9497	0,9686	0,9591	7617
Riesgo bajo	0,8403	0,7863	0,8124	7617
Riesgo medio	0,8760	0,8828	0,8794	7617
Sin riesgo	0,8864	0,9189	0,9023	7617
Accuracy			0,8891	30468
Macro Avg	0,8881	0,8891	0,8883	30468
Weighted Avg	0,8881	0,8891	0,8883	30468

Tabla 6.78. *Reporte de clasificación en entrenamiento (SVM con Undersampling, C=10)*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,9082	0,9351	0,9215	1957
Riesgo bajo	0,7631	0,7085	0,7348	2141
Riesgo medio	0,8168	0,8271	0,8219	2313
Sin riesgo	0,8442	0,8734	0,8585	1904
Accuracy			0,8326	8315
Macro Avg	0,8331	0,8360	0,8342	8315
Weighted Avg	0,8307	0,8326	0,8313	8315

Tabla 6.79. *Reporte de clasificación en prueba (SVM con Undersampling, C=10)*

Métrica	Valor
MCC (Test)	0,7767
ROC AUC (*Macro*, Test)	0,9620
Tiempo de ejecución (s)	533,66

Tabla 6.80. *Métricas adicionales y tiempo (SVM con Undersampling, C=10)*

Parámetro / Métrica de CV	Valor
Valor de C (usado)	10
Accuracy en CV (KFold)	0,9506 ± 0,0014
Técnica de balanceo aplicada	ENN

Tabla 6.81. *Configuración del modelo SVM y resultados de CV (ENN)*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,9946	0,9975	0,9961	5209
Riesgo bajo	0,9354	0,9182	0,9267	3704
Riesgo medio	0,9846	0,9864	0,9855	5830
Sin riesgo	0,9679	0,9732	0,9705	7617
Accuracy			0,9732	22360
Macro Avg	0,9706	0,9688	0,9697	22360
Weighted Avg	0,9731	0,9732	0,9731	22360

Tabla 6.82. *Reporte de clasificación en entrenamiento (SVM con ENN, C=10)*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,8727	0,9075	0,8898	1957
Riesgo bajo	0,7756	0,5488	0,6428	2141
Riesgo medio	0,7979	0,8007	0,7993	2313
Sin riesgo	0,7316	0,9391	0,8224	1904
Accuracy			0,7927	8315
Macro Avg	0,7945	0,7990	0,7886	8315
Weighted Avg	0,7946	0,7927	0,7856	8315

Tabla 6.83. *Reporte de clasificación en prueba (SVM con ENN, C=10)*

Métrica	Valor
MCC (Test)	0,7286
ROC AUC (*Macro*, Test)	0,9460
Tiempo de ejecución (s)	132,74

Tabla 6.84. *Métricas adicionales y tiempo (SVM con ENN, C=10)*

Parámetro / Métrica de CV	Valor
Valor de C (usado)	10
Accuracy en CV (KFold)	0,8168 ± 0,0041
Técnica de balanceo aplicada	Cluster Centroids

Tabla 6.85. *Configuración del modelo SVM y resultados de CV (Cluster Centroids)*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,9460	0,9639	0,9549	7617
Riesgo bajo	0,8240	0,7597	0,7906	7617
Riesgo medio	0,8571	0,8687	0,8629	7617
Sin riesgo	0,8759	0,9158	0,8954	7617
Accuracy			0,8771	30468
Macro Avg	0,8758	0,8771	0,8759	30468
Weighted Avg	0,8758	0,8771	0,8759	30468

Tabla 6.86. *Reporte de clasificación en entrenamiento (SVM con Cluster Centroids, C=10)*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,9161	0,9315	0,9237	1957
Riesgo bajo	0,7562	0,6955	0,7246	2141
Riesgo medio	0,8093	0,8383	0,8235	2313
Sin riesgo	0,8408	0,8655	0,8530	1904
Accuracy			0,8297	8315
Macro Avg	0,8306	0,8327	0,8312	8315
Weighted Avg	0,8280	0,8297	0,8284	8315

Tabla 6.87. *Reporte de clasificación en prueba (SVM con Cluster Centroids, C=10)*

Métrica	Valor
MCC (Test)	0,7728
ROC AUC (*Macro*, Test)	0,9626
Tiempo de ejecución (s)	395,31

Tabla 6.88. *Métricas adicionales y tiempo (SVM con Cluster Centroids, C=10)*

El SVM con *Undersampling* General (C=10) emerge como una opción muy competitiva, logrando un rendimiento en prueba (*Accuracy*, MCC, *F1-score* para "Riesgo bajo") a la par o incluso marginalmente superior al SVM con SMOTE, pero con un tiempo de entrenamiento significativamente menor. Su grado de sobreajuste es moderado. El SVM con Cluster Centroids (C=10) también se presenta como una alternativa sólida, con un rendimiento en prueba muy cercano al de *Undersampling* y SMOTE, el menor sobreajuste y un tiempo de entrenamiento intermedio. Además, su ROC AUC es ligeramente el mejor entre los submuestreos. Por otro lado, el SVM con ENN (C=10), a pesar de su excelente rendimiento en validación cruzada y su rapidez, sufre de un sobreajuste severo y su rendimiento en prueba, especialmente para "Riesgo bajo", es el peor, lo que lo descarta como una opción viable a pesar de la eficiencia.

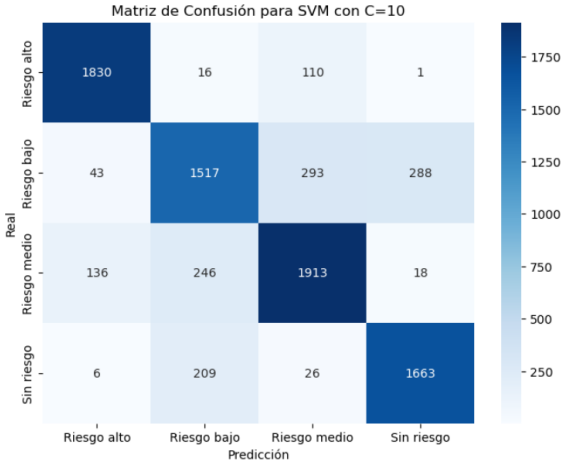


Figura 6.30: Matriz de confusión - SVM (*Undersampling*)

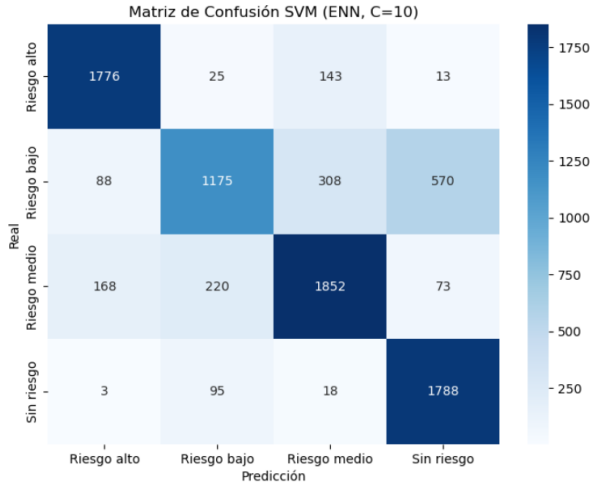


Figura 6.31: Matriz de confusión - SVM (ENN)

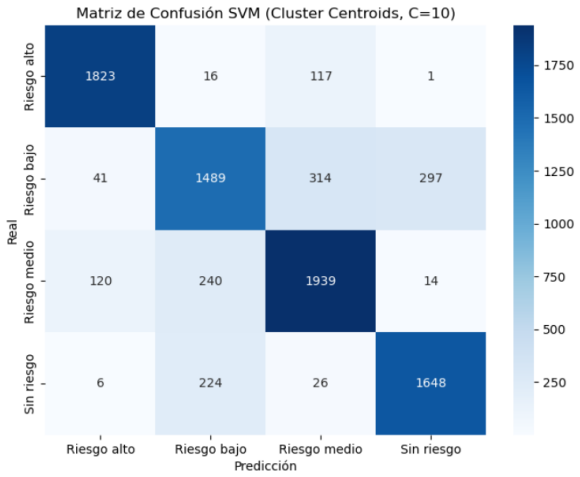


Figura 6.32: Matriz de confusión - SVM (CC)

Las matrices de confusión confirman visualmente las tendencias observadas en los resultados numéricos: el SVM entrenado con datos procesados por **ENN** no generaliza bien, a pesar de un rendimiento en CV muy alto. Su manejo de la clase "Riesgo bajo" es particularmente deficiente. Los SVM con *Undersampling* General y Cluster Centroids muestran un comportamiento de clasificación mucho más robusto y similar entre sí, y también muy parecido al SVM con SMOTE. Visualmente, las matrices de *Undersampling* y Cluster Centroids son muy parecidas. Ambas indican un buen manejo de las clases "Riesgo alto", "Riesgo medio" y "Sin riesgo", y un manejo aceptable (y el mejor entre los submuestreos) de la clase "Riesgo bajo". El SVM con *Undersampling* General parece tener una ligera ventaja visual en la diagonal para "Riesgo bajo" (1517 vs 1489 de CC), lo que se alinea con su *F1-score* ligeramente superior para esa clase en los resultados numéricos (0,7348 vs 0,7246).

6.8. *XGBoost*

6.8.1. Sin balanceo

Las tablas siguientes recogen los resultados obtenidos por el modelo *XGBoost*, incluyendo tanto métricas globales como por clase. Estos resultados permiten apreciar su excelente capacidad de generalización y, en particular, su destacada eficacia en la clasificación de la clase "Riesgo bajo".

Métrica promedio(CV KFold=5)	Valor
Accuracy	0,8476
Macro Precision	0,8493
Macro Recall	0,8507
Macro F1-score	0,8498

Tabla 6.89. *Resultados promedio de Cross-Validation (XGBoost)*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,9454	0,9652	0,9552	7827
Riesgo bajo	0,7935	0,7849	0,7891	8562
Riesgo medio	0,8796	0,8581	0,8687	9251
Sin riesgo	0,8771	0,8950	0,8860	7617
Accuracy			0,8729	33257
Macro Avg	0,8739	0,8758	0,8748	33257
Weighted Avg	0,8723	0,8729	0,8725	33257

Tabla 6.90. *Reporte de clasificación en el conjunto de entrenamiento (XGBoost)*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,9339	0,9453	0,9396	1957
Riesgo bajo	0,7557	0,7543	0,7550	2141
Riesgo medio	0,8527	0,8387	0,8457	2313
Sin riesgo	0,8585	0,8666	0,8625	1904
Accuracy			0,8485	8315
Macro Avg	0,8502	0,8512	0,8507	8315
Weighted Avg	0,8482	0,8485	0,8483	8315

Tabla 6.91. *Reporte de clasificación en el conjunto de prueba (XGBoost)*

Métrica	Valor
MCC (Test)	0,7976
ROC AUC (*Macro*, Test)	0,9710
Tiempo de ejecución (s)	26,98

Tabla 6.92. *Métricas adicionales del conjunto de prueba y tiempo de ejecución (XGBoost)*

Su capacidad para manejar la clase "Riesgo bajo" con un *F1-score* de 0,7550 es un diferenciador importante, ya que esta ha sido la clase más difícil para la mayoría de los otros modelos. La combinación de alto rendimiento predictivo, excelente generalización (bajo sobreajuste) y eficiencia computacional hace de *XGBoost* una opción extremadamente atractiva para este problema.

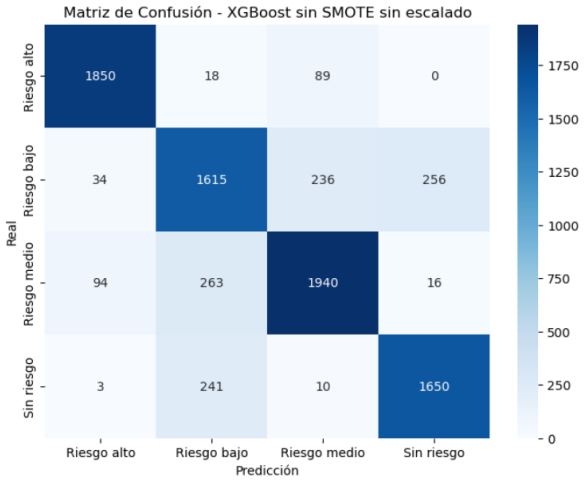


Figura 6.33: Matriz de confusión - *XGBoost*

La matriz de confusión del modelo *XGBoost* (sin SMOTE, sin escalado) ilustra de manera concluyente su rendimiento superior y más equilibrado en comparación con los otros modelos evaluados. Destaca especialmente por:

1. El manejo significativamente mejorado de la clase "Riesgo bajo", tanto en términos de verdaderos positivos como en la reducción de su confusión con otras clases.
2. La clasificación casi perfecta de la clase "Riesgo alto", con una confusión mínima hacia otras categorías.
3. Un patrón general de errores menos pronunciado en todas las clases.

6.8.2. *Oversampling*

Las tablas siguientes presentan los resultados de *XGBoost* con SMOTE, comparados con la versión sin balanceo. Se incluyen métricas globales y por clase, lo que permite comprobar que la técnica no aporta mejoras y que, en la clase "Riesgo bajo", incluso se produce un ligero retroceso.

Métrica promedio(CV KFold=5)	Valor
Accuracy	0,8553
Macro Precision	0,8552
Macro Recall	0,8553
Macro F1-score	0,8552
Técnica de balanceo aplicada	SMOTE

Tabla 6.93. *Resultados promedio de cross-validation (XGBoost con SMOTE)*

Todas las métricas globales, como la precisión (0,8468 vs. 0,8485) y el MCC (0,7954 vs. 0,7976), son marginalmente inferiores a las del modelo *XGBoost* sin balanceo. El objetivo principal de utilizar SMOTE, que era mejorar el rendimiento en

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,9544	0,9683	0,9613	9251
Riesgo bajo	0,7941	0,7909	0,7925	9251
Riesgo medio	0,8731	0,8592	0,8661	9251
Sin riesgo	0,8931	0,8980	0,8955	9251
Accuracy			0,8791	37004
Macro Avg	0,8787	0,8791	0,8789	37004
Weighted Avg	0,8787	0,8791	0,8789	37004

Tabla 6.94. *Reporte de clasificación en el conjunto de entrenamiento (XGBoost con SMOTE)*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,9342	0,9433	0,9387	1957
Riesgo bajo	0,7494	0,7543	0,7519	2141
Riesgo medio	0,8526	0,8379	0,8452	2313
Sin riesgo	0,8592	0,8624	0,8608	1904
Accuracy			0,8468	8315
Macro Avg	0,8489	0,8495	0,8491	8315
Weighted Avg	0,8468	0,8468	0,8467	8315

Tabla 6.95. *Reporte de clasificación en el conjunto de prueba (XGBoost con SMOTE)*

Métrica	Valor
MCC (Test)	0,7954
ROC AUC (*Macro*, Test)	0,9707
Tiempo de ejecución (s)	63,82

Tabla 6.96. *Métricas adicionales del conjunto de prueba y tiempo de ejecución (XGBoost con SMOTE)*

la clase “Riesgo bajo”, no se ha cumplido. De hecho, el *F1-score* para esta categoría disminuye ligeramente (0,7519 vs. 0,7550), lo que confirma que la técnica no aportó ningún beneficio a la principal debilidad del modelo.

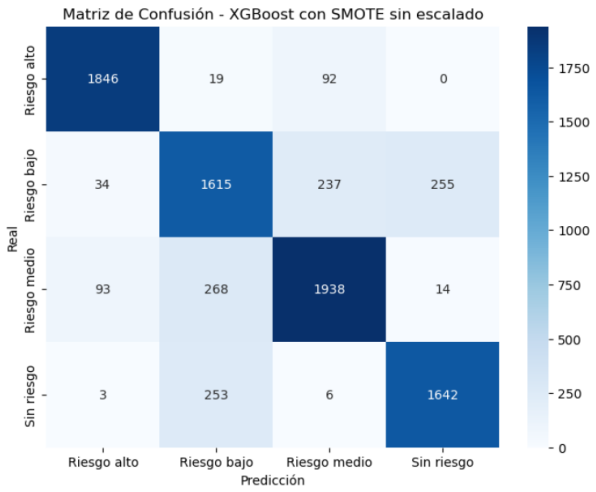


Figura 6.34: Matriz de confusión - *XGBoost* (SMOTE)

Esta matriz de confusión es coherente con los resultados numéricos que indicaron que el *XGBoost* con SMOTE (Test *Accuracy*: 0,8468, MCC: 0,7954, F1 para "Riesgo bajo": 0,7519) tuvo un rendimiento marginalmente inferior al *XGBoost* sin SMOTE (Test *Accuracy*: 0,8485, MCC: 0,7976, F1 para "Riesgo bajo": 0,7550). El hecho de que el número de verdaderos positivos para "Riesgo bajo" sea idéntico (1.615) en ambas matrices, y que los patrones de error sean tan similares, explica por qué el *F1-score* para esta clase fue casi igual (0,7519 con SMOTE vs 0,7550 sin SMOTE).

6.8.3. *Undersampling*

Las tablas siguientes muestran los resultados de *XGBoost* aplicando distintas técnicas de submuestreo. Se presentan métricas globales y por clase, lo que permite comprobar cómo *Undersampling* y Cluster Centroids logran no solo mantener, sino incluso mejorar ligeramente el rendimiento del modelo base, mientras que ENN evidencia un sobreajuste severo.

Técnica de Submuestreo	Avg Acc (CV)	Avg P (CV)	Avg R (CV)	Avg F1 (CV)
Undersampling General	0,8505	0,8501	0,8505	0,8503
ENN	**0,9463**	**0,9397**	**0,9345**	**0,9370**
Cluster Centroids	0,8393	0,8391	0,8393	0,8392

Tabla 6.97. *Resultados promedio de cross-validation (XGBoost con Submuestreo)*

	entrenamiento				prueba			
Clase	P	R	F1	Sup	P	R	F1	Sup
Riesgo alto	0,952	0,965	0,958	7617	0,935	0,948	0,941	1957
Riesgo bajo	0,796	0,791	0,793	7617	0,759	0,752	0,755	2141
Riesgo medio	0,872	0,862	0,867	7617	0,855	0,844	0,849	2313
Sin riesgo	0,891	0,895	0,893	7617	0,859	0,869	0,864	1904
Accuracy			0,8780	30468			0,8504	8315
Macro Avg	0,878	0,878	0,878		0,852	0,853	0,852	
MCC (Test): 0,8002, ROC AUC (Test): 0,9709, Tiempo: 53,52s								

Tabla 6.98. *Resumen de XGBoost con Undersampling General*

	entrenamiento				prueba			
Clase	P	R	F1	Sup	P	R	F1	Sup
Riesgo alto	0,995	0,995	0,995	5884	0,915	0,944	0,929	1957
Riesgo bajo	0,927	0,878	0,902	3682	0,796	0,574	0,667	2141
Riesgo medio	0,977	0,982	0,979	5966	0,830	0,823	0,826	2313
Sin riesgo	0,954	0,974	0,964	7617	0,740	0,955	0,834	1904
Accuracy			0,9660	23149			0,8176	8315
Macro Avg	0,963	0,957	0,960		0,820	0,824	0,814	
MCC (Test): 0,7618, ROC AUC (Test): 0,9635, Tiempo: 72,22s								

Tabla 6.99. *Resumen de XGBoost con ENN*

	entrenamiento				prueba			
Clase	P	R	F1	Sup	P	R	F1	Sup
Riesgo alto	0,950	0,960	0,955	7617	0,940	0,943	0,942	1957
Riesgo bajo	0,779	0,771	0,775	7617	0,759	0,751	0,755	2141
Riesgo medio	0,854	0,850	0,852	7617	0,846	0,854	0,850	2313
Sin riesgo	0,886	0,889	0,888	7617	0,867	0,864	0,865	1904
Accuracy			0,8676	30468			0,8506	8315
Macro Avg	0,867	0,868	0,867		0,853	0,853	0,853	
MCC (Test): 0,8004, ROC AUC (Test): 0,9709, Tiempo: 7,85s								

Tabla 6.100. *Resumen de XGBoost con Cluster Centroids*

Tanto *XGBoost* con *Undersampling* General como *XGBoost* con Cluster Centroids demuestran ser estrategias de preprocesamiento **exitosas**, ya que no solo mantienen el altísimo rendimiento del *XGBoost* base, sino que lo superan marginalmente en varias métricas clave de prueba, incluyendo *Accuracy* y MCC. *XGBoost* con Cluster Centroids se destaca particularmente por lograr el mejor MCC (0,8004) y Test Accuracy (0,8506) junto con *Undersampling*, el mejor *Macro* F1, un excelente control del sobreajuste y un tiempo de ejecución sorprendentemente bajo (7,85s). Esto lo convierte en una opción extremadamente atractiva. Por su parte, *XGBoost* con *Undersampling* General también es una excelente opción, con un rendimiento casi idéntico al de Cluster Centroids, aunque con un tiempo de ejecución mayor. En contraste, *XGBoost* con ENN no es recomendable: a pesar de su rendimiento estelar en CV, el modelo sobreajusta severamente y no generaliza bien al conjunto de prueba, especialmente perjudicando a la clase “Riesgo bajo”.

El *F1-score* para "Riesgo bajo" se mantiene alrededor de 0,755 con *Undersampling* y Cluster Centroids, lo cual representa el mejor resultado obtenido de forma consistente para esta clase entre todos los modelos y técnicas.

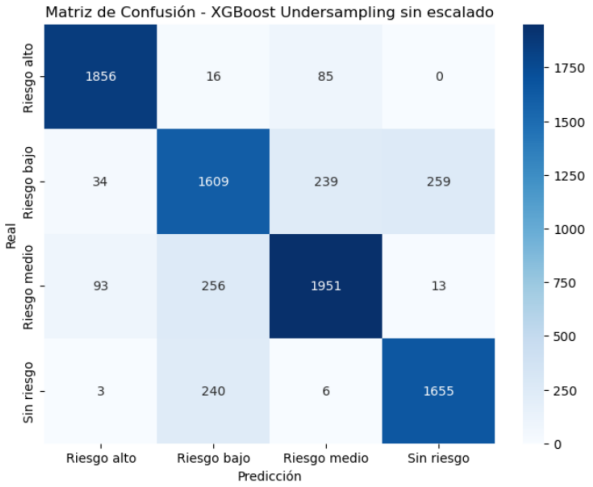


Figura 6.35: Matriz de confusión - *XGBoost* (*Undersampling*)

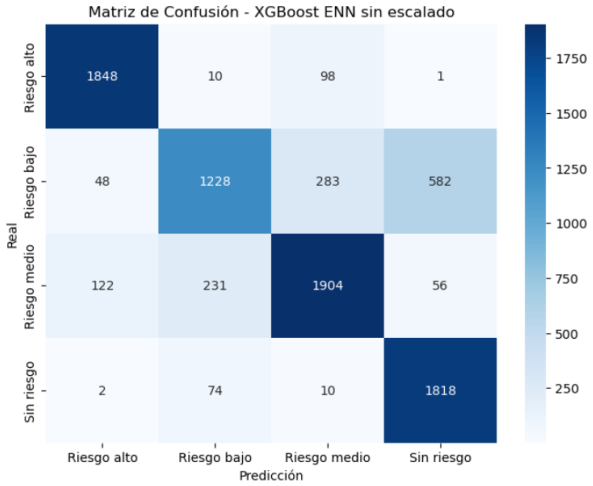


Figura 6.36: Matriz de confusión - *XGBoost* (ENN)

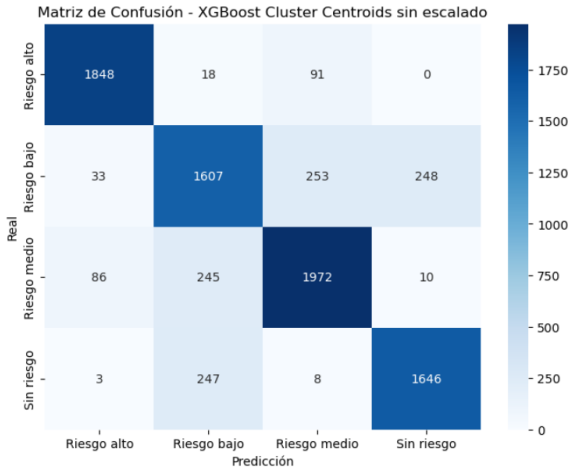


Figura 6.37: Matriz de confusión - *XGBoost* (CC)

Las matrices de confusión confirman visualmente los hallazgos de los resultados numéricos, donde *XGBoost* con *Undersampling* General y con Cluster Centroids superaron ligeramente al *XGBoost* base y significativamente al *XGBoost* con ENN. El modelo *XGBoost* entrenado con **ENN** (Figura 6.36) no generaliza bien, y sufre especialmente en la clasificación de "Riesgo bajo". Los modelos con ***Undersampling* General** (Figura 6.35) y **Cluster Centroids** (Figura 6.37) presentan matrices de confusión muy robustas y similares entre sí. Ambas muestran un excelente manejo de todas las clases, incluyendo la difícil "Riesgo bajo". Comparando *Undersampling* y Cluster Centroids con el *XGBoost* base (que tenía 1615 VP para "Riesgo bajo"), ambos modelos con submuestreo mantienen o mejoran ligeramente este número (*Undersampling*: 1.609, CC: 1.607). Visualmente, las tres matrices (Base, *Undersampling*, CC) son muy parecidas en cuanto a su calidad, lo que es un resultado excelente. La principal diferencia entre *Undersampling* y Cluster Centroids es sutil. Cluster Centroids parece tener una ligera ventaja en los verdaderos positivos para "Riesgo medio" (1.972 vs 1.951), mientras que *Undersampling* lo hace para "Riesgo alto" (1.856 vs 1.848). Ambas son excelentes. La capacidad de estos dos modelos para mantener un alto número de verdaderos positivos para "Riesgo bajo" y, al mismo tiempo, clasificar correctamente las otras clases, los posiciona como las mejores opciones visualizadas hasta ahora.

6.9. Red Neuronal

6.9.1. *Oversampling*

Las tablas siguientes presentan los resultados de las dos arquitecturas de red DNN con SMOTE, mostrando cómo la configuración (64, 128) logra un rendimiento global ligeramente superior y, en particular, una mejora en la clasificación de la clase "Riesgo bajo".

Métrica	Valor
Arquitectura DNN (Capas Ocultas)	(32, 64)
Técnica de balanceo aplicada	SMOTE
Train Accuracy	0,8673
Test Accuracy	0,8130
MCC (Test)	0,7508
ROC AUC (*Macro*, Test)	0,9567

Tabla 6.101. *Resumen de rendimiento del modelo DNN con SMOTE (32-64)*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,89	0,94	0,92	1957
Riesgo bajo	0,72	0,66	0,69	2141
Riesgo medio	0,81	0,80	0,81	2313
Sin riesgo	0,82	0,87	0,84	1904
Accuracy			0,81	8315
Macro Avg	0,81	0,82	0,81	8315
Weighted Avg	0,81	0,81	0,81	8315

Tabla 6.102. *Reporte de clasificación en prueba (DNN con SMOTE 32-64)*

Métrica	Valor
Tiempo de ejecución (s)	31.14

Tabla 6.103. *Tiempo de ejecución (DNN con SMOTE 32-64)*

Métrica	Valor
Arquitectura DNN (Capas Ocultas)	(64, 128)
Técnica de balanceo aplicada	SMOTE
Test *Accuracy*	0,8182
MCC (Test)	0,7572
ROC AUC (*Macro*, Test)	0,9573

Tabla 6.104. *Resumen de rendimiento del modelo DNN con SMOTE (64-128)*

Clase	*Precision*	*Recall*	*F1-score*	*Support*
Riesgo alto	0,93	0,90	0,92	1957
Riesgo bajo	0,72	0,69	0,71	2141
Riesgo medio	0,80	0,82	0,81	2313
Sin riesgo	0,83	0,87	0,85	1904
Accuracy			0,82	8315
Macro Avg	0,82	0,82	0,82	8315
Weighted Avg	0,82	0,82	0,82	8315

Tabla 6.105. *Reporte de clasificación en prueba (DNN con SMOTE 64-128)*

Métrica	Valor
Tiempo de ejecución (s)	31.18

Tabla 6.106. *Tiempo de ejecución (DNN con SMOTE 64-128)*

El modelo DNN con la arquitectura de capas ocultas **(64, 128)** y SMOTE ofrece un rendimiento general ligeramente superior al de la arquitectura (32, 64) con SMOTE. La mejora es más notable en el *F1-score* de la clase "Riesgo bajo", que es consistentemente la más desafiante.

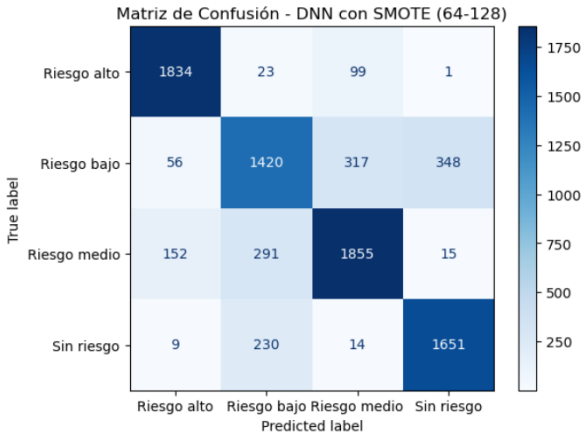


Figura 6.38: Matriz de confusión (SMOTE 32-64)

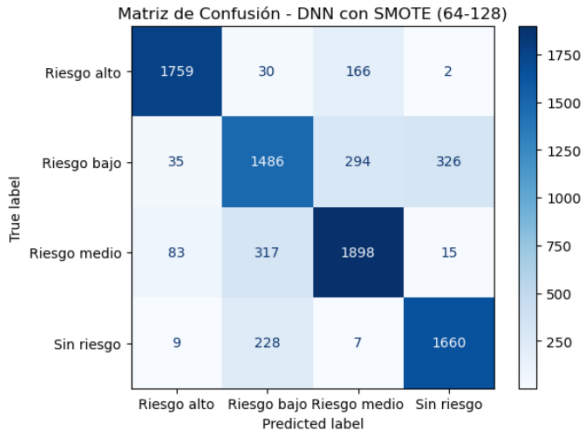


Figura 6.39: Matriz de confusión (SMOTE 64-128)

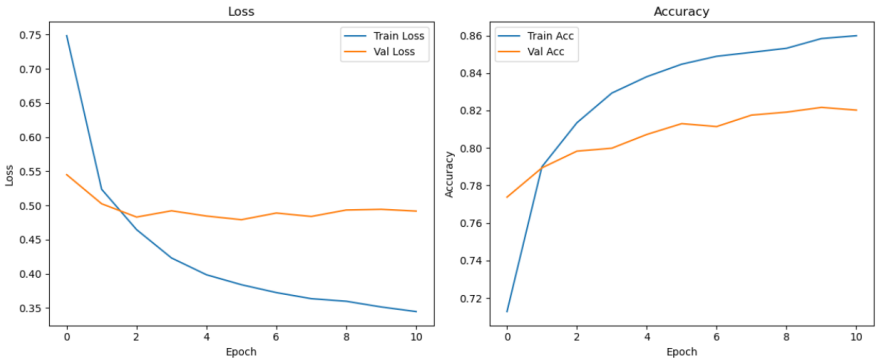


Figura 6.40: Gráficos LossAccuracy - DNN SMOTE(32-64)

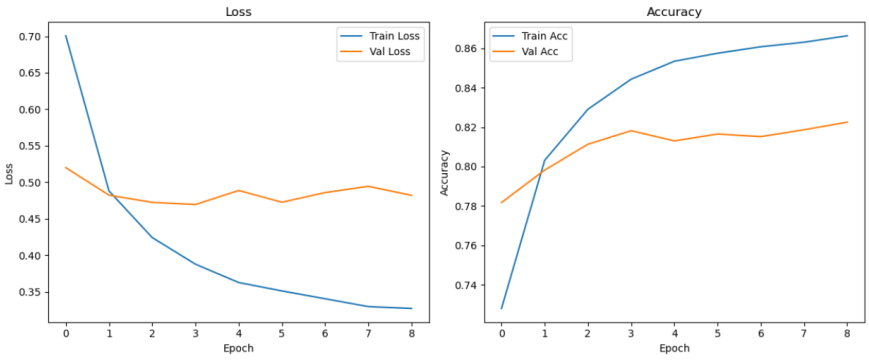


Figura 6.41: Gráficos LossAccuracy - DNN SMOTE (64-128)

Ambas arquitecturas DNN muestran un comportamiento de aprendizaje típico, con evidencia de sobreajuste a medida que avanza el entrenamiento. Esto es normal y se maneja con técnicas como Early Stopping o regularización. El modelo **DNN (64-128)** parece alcanzar una precisión de validación ligeramente superior y la mantiene de forma más estable en las últimas épocas visualizadas, lo que se alinea con su Test *Accuracy* numérico ligeramente mejor (0,8182 vs 0,8130 para el 32-64). La brecha de sobreajuste entre las curvas de entrenamiento y validación es similar para ambas arquitecturas, lo que sugiere que la mayor complejidad del modelo (64-128) no ha llevado a un sobreajuste descontrolado dentro de este número de épocas y podría estar permitiendo capturar patrones más sutiles que benefician la generalización. Estas curvas son fundamentales para diagnosticar problemas de entrenamiento y para guiar la selección de hiperparámetros como el número de épocas o la configuración de Early Stopping. Si el entrenamiento hubiera continuado por muchas más épocas sin Early Stopping, es probable que la brecha de sobreajuste se hubiera acentuado.

6.9.2. *Undersampling*

Las tablas siguientes recogen los resultados de las distintas configuraciones de DNN con submuestreo, comparando arquitecturas y técnicas aplicadas. Estos resultados permiten comprobar que el uso de *Undersampling* con la arquitectura (64-128) ofrece el mejor rendimiento global, mientras que ENN se muestra inadecuado para este problema.

Métrica	Valor
Arquitectura DNN	(32, 64)
Técnica de balanceo	*Undersampling*
Test *Accuracy*	0,8063
MCC (Test)	0,7421
ROC AUC (*Macro*, Test)	0,9533
Tiempo ejecución (s)	77,61

Tabla 6.107. *Resumen del modelo DNN con Undersampling (32-64)*

Clase	*Precision*	*Recall*	*F1-score*
Riesgo alto	0,8803	0,9280	0,9035
Riesgo bajo	0,7131	0,6628	0,6870
Riesgo medio	0,8175	0,7748	0,7956
Sin riesgo	0,8101	0,8808	0,8440
Accuracy			0,8063
Macro Avg	0,8053	0,8116	0,8075
Weighted Avg	0,8037	0,8063	0,8041

Tabla 6.108. *Reporte de clasificación en prueba (DNN con Undersampling 32-64)*

Métrica	Valor
Arquitectura DNN	(64, 128)
Técnica de balanceo	*Undersampling*
Test *Accuracy*	0,8290
MCC (Test)	0,7722
ROC AUC (*Macro*, Test)	0,9611
Tiempo ejecución (s)	116.73

Tabla 6.109. *Resumen del modelo DNN con Undersampling (64-128)*

Clase	*Precision*	*Recall*	*F1-score*
Riesgo alto	0,9065	0,9361	0,9211
Riesgo bajo	0,7238	0,7440	0,7338
Riesgo medio	0,8548	0,7838	0,8178
Sin riesgo	0,8392	0,8692	0,8540
Accuracy			0,8290
Macro Avg	0,8311	0,8333	0,8316
Weighted Avg	0,8297	0,8290	0,8287

Tabla 6.110. *Reporte de clasificación en prueba (DNN con Undersampling 64-128)*

Métrica	Valor
Arquitectura DNN	(32, 64)
Técnica de balanceo	ENN
Test *Accuracy*	0,7634
MCC (Test)	0,6891
ROC AUC (*Macro*, Test)	0,9343
Tiempo ejecución (s)	36,20

Tabla 6.111. *Resumen del modelo DNN con ENN (32-64)*

Clase	*Precision*	*Recall*	*F1-score*
Riesgo alto	0,8669	0,8585	0,8626
Riesgo bajo	0,7197	0,5180	0,6024
Riesgo medio	0,7591	0,7821	0,7704
Sin riesgo	0,7134	0,9191	0,8033
Accuracy			0,7634
Macro Avg	0,7648	0,7694	0,7597
Weighted Avg	0,7639	0,7634	0,7564

Tabla 6.112. *Reporte de clasificación en prueba (DNN con ENN 32-64)*

Métrica	Valor
Arquitectura DNN	(64, 128)
Técnica de balanceo	ENN
Test *Accuracy*	0,7525
MCC (Test)	0,6762
ROC AUC (*Macro*, Test)	0,9338
Tiempo ejecución (s)	36,42

Tabla 6.113. *Resumen del modelo DNN con ENN (64-128)*

Clase	*Precision*	*Recall*	*F1-score*
Riesgo alto	0,8563	0,8493	0,8527
Riesgo bajo	0,7136	0,4876	0,5794
Riesgo medio	0,7592	0,7687	0,7639
Sin riesgo	0,6902	0,9312	0,7928
Accuracy			0,7525
Macro Avg	0,7548	0,7592	0,7472
Weighted Avg	0,7545	0,7525	0,7439

Tabla 6.114. *Reporte de clasificación en prueba (DNN con ENN 64-128)*

Métrica	Valor
Arquitectura DNN	(32, 64)
Técnica de balanceo	Cluster Centroids
Test *Accuracy*	0,8174
MCC (Test)	0,7564
ROC AUC (*Macro*, Test)	0,9573
Tiempo ejecución (s)	87,59

Tabla 6.115. *Resumen del modelo DNN con Cluster Centroids (32-64)*

Clase	*Precision*	*Recall*	*F1-score*
Riesgo alto	0,9200	0,9228	0,9214
Riesgo bajo	0,7251	0,6740	0,6986
Riesgo medio	0,8011	0,8167	0,8088
Sin riesgo	0,8278	0,8713	0,8490
Accuracy			0,8174
Macro Avg	0,8185	0,8212	0,8195
Weighted Avg	0,8157	0,8174	0,8162

Tabla 6.116. *Reporte de clasificación en prueba (DNN con Cluster Centroids 32-64)*

Métrica	Valor
Arquitectura DNN	(64, 128)
Técnica de balanceo	Cluster Centroids
Test *Accuracy*	0,8174
MCC (Test)	0,7565
ROC AUC (*Macro*, Test)	0,9564
Tiempo ejecución (s)	92,98

Tabla 6.117. *Resumen del modelo DNN con Cluster Centroids (64-128)*

Clase	*Precision*	*Recall*	*F1-score*
Riesgo alto	0,9084	0,9274	0,9178
Riesgo bajo	0,7351	0,6702	0,7012
Riesgo medio	0,7982	0,8206	0,8092
Sin riesgo	0,8299	0,8661	0,8476
Accuracy			0,8174
Macro Avg	0,8179	0,8211	0,8190
Weighted Avg	0,8151	0,8174	0,8158

Tabla 6.118. *Reporte de clasificación en prueba (DNN con Cluster Centroids 64-128)*

El modelo **DNN con *Undersampling* y arquitectura (64-128)** se destaca como la mejor combinación entre las opciones de DNN con submuestreo evaluadas. Ofrece el mejor rendimiento general en prueba (*Accuracy*, MCC, ROC AUC) y, crucialmente, el mejor *F1-score* para la clase "Riesgo bajo". Los modelos con **Cluster Centroids** (ambas arquitecturas) representan una segunda opción sólida y consistente, con un buen manejo general de las clases y un rendimiento muy cercano, aunque ligeramente inferior al obtenido con *Undersampling* (64-128) en la categoría "Riesgo bajo". En cambio, los modelos con **ENN** no son recomendables para este problema con DNNs, ya que, aunque pueden ser rápidos de entrenar y mostrar buen rendimiento en datos "limpiados", no generalizan bien al conjunto de prueba original, perjudicando severamente la identificación de la clase "Riesgo bajo".

Finalmente, aumentar la complejidad de la arquitectura DNN de (32-64) a (64-128) parece ser beneficioso cuando se combina con *Undersampling* general, pero no ofrece una mejora clara o incluso puede resultar perjudicial con ENN o Cluster Centroids, donde el rendimiento fue similar o ligeramente peor para (64-128) en algunas métricas.

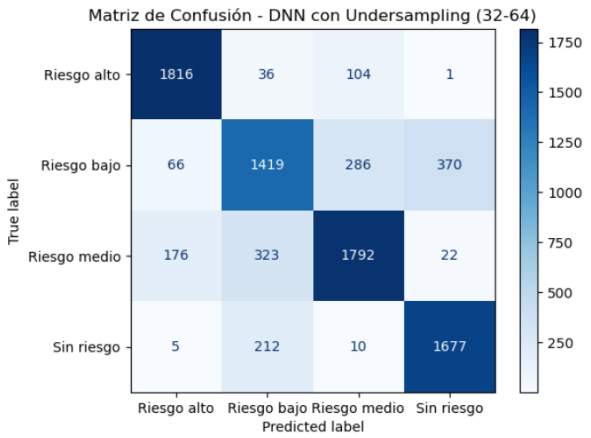


Figura 6.42: Matriz de confusión - DNN (*Undersampling* 32-64)

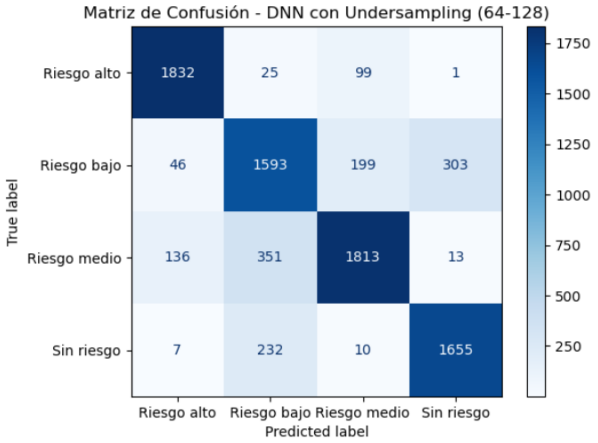


Figura 6.43: Matriz de confusión - DNN (*Undersampling* 64-128)

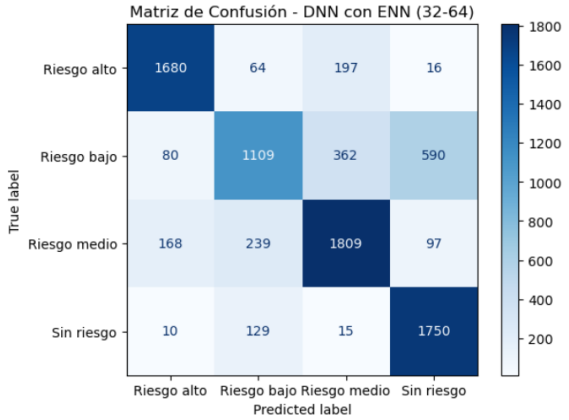


Figura 6.44: Matriz de confusión - DNN (ENN 32-64

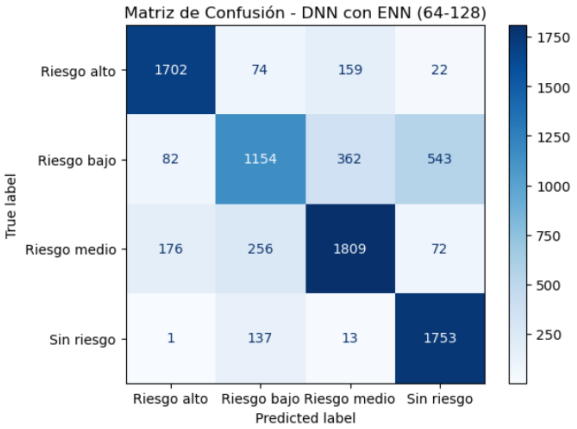


Figura 6.45: Matriz de confusión - DNN (ENN 64-128)

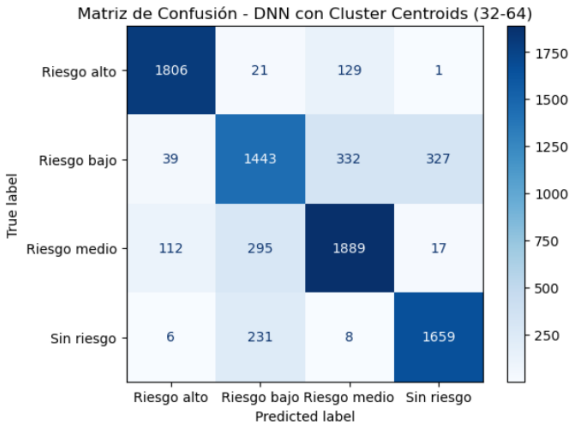


Figura 6.46: Matriz de confusión - DNN (CC 32-64)

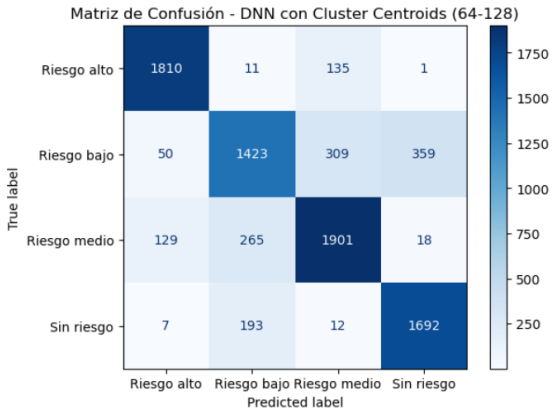


Figura 6.47: Matriz de confusión - DNN (CC 64-128)

El **DNN con *Undersampling* y arquitectura (64-128)** (Figura 6.43) sigue destacándose como la combinación con la matriz de confusión visualmente más robusta, principalmente por su superioridad en la clasificación correcta de la clase "Riesgo bajo" y la reducción general de errores para esta clase. Los modelos DNN con **ENN** (Figuras 6.44 y 6.45) son consistentemente los peores visualmente. La masiva confusión de "Riesgo bajo" hacia "Sin riesgo" es un problema crítico que ninguna de las dos arquitecturas resuelve. Los modelos DNN con **Cluster Centroids** (Figuras 6.46 y 6.47) ofrecen un rendimiento visual intermedio y estable. La arquitectura más simple (32-64) parece ser suficiente e incluso ligeramente preferible para "Riesgo bajo" en términos de verdaderos positivos visualizados, aunque las métricas F1 numéricas eran muy parejas. Estas matrices de confusión corregidas refuerzan que para los modelos DNN: **ENN es perjudicial** para la generalización. ***Undersampling* General**, especialmente con una arquitectura más compleja (64-128), es la estrategia de submuestreo más efectiva para mejorar el rendimiento general y el de la clase "Riesgo bajo". **Cluster Centroids** es una alternativa viable y consistente, pero no supera al mejor *Undersampling*.

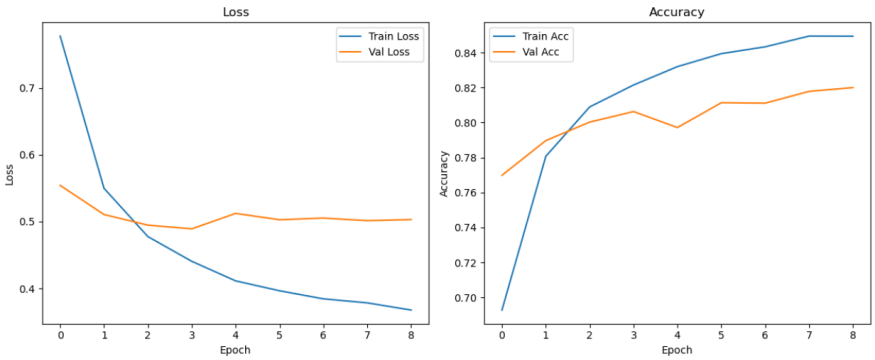


Figura 6.48: Gráficos LossAccuracy - DNN *Undersampling* (32-64)

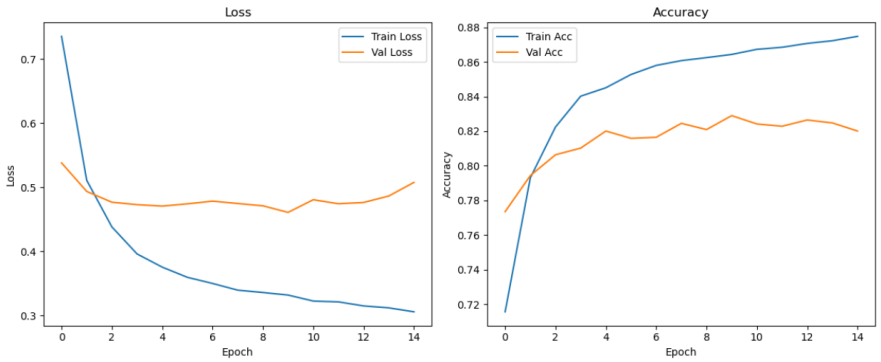


Figura 6.49: Gráficos LossAccuracy - DNN *Undersampling* (64-128)

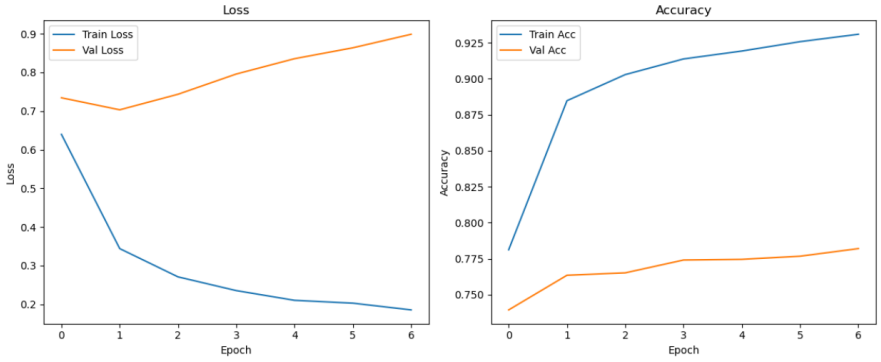


Figura 6.50: Gráficos LossAccuracy - DNN ENN (32-64)

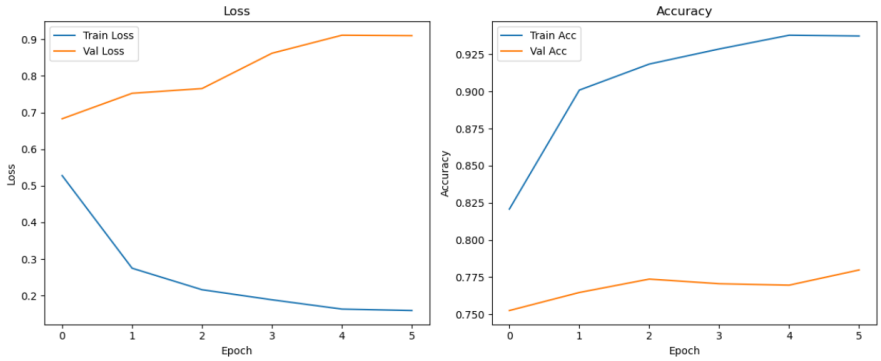


Figura 6.51: Gráficos LossAccuracy - DNN ENN (64-128)

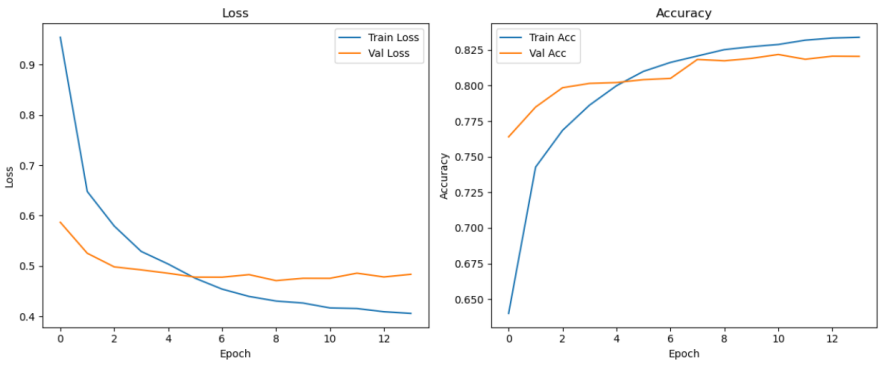


Figura 6.52: Gráficos LossAccuracy - DNN CC (32-64)

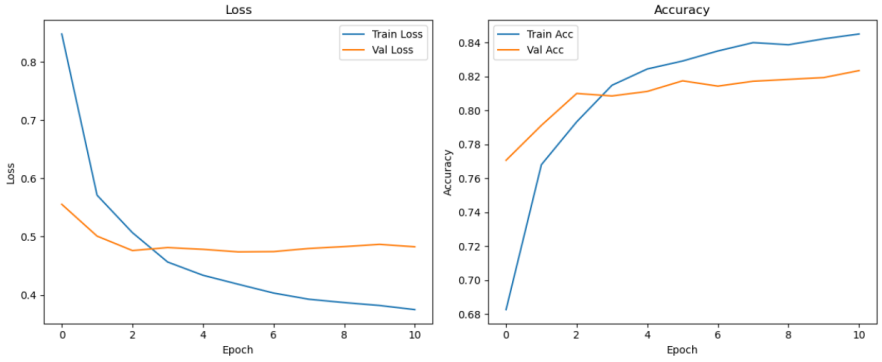


Figura 6.53: Gráficos LossAccuracy - DNN CC (64-128)

Las técnicas de ***Undersampling*** **General** y **Cluster Centroids** producen curvas de aprendizaje mucho más saludables para los modelos DNN en comparación con **ENN**. Con *Undersampling* y CC, la pérdida de validación se estabiliza y la precisión de validación alcanza una meseta razonable, con una brecha de sobreajuste moderada. Con **ENN**, las curvas muestran un **sobreajuste severo e inmediato**. La pérdida de validación aumenta casi desde el principio, y la precisión de validación se estanca en un nivel bajo mientras la precisión de entrenamiento se dispara. Esto indica que ENN no es una técnica de preprocesamiento adecuada para estos modelos DNN en este problema, ya que los modelos simplemente memorizan el dataset resultante sin aprender características generalizables. Para ***Undersampling*** **y Cluster Centroids**, la arquitectura más compleja **(64-128) consistentemente muestra una capacidad para alcanzar una precisión de validación ligeramente superior** cuando se entrena durante un número adecuado de épocas (más épocas que la arquitectura 32-64). Estas curvas de aprendizaje son consistentes con los resultados numéricos finales en el conjunto de prueba, donde los modelos DNN con *Undersampling* (64-128) y Cluster Centroids (ambas arquitecturas) tuvieron un rendimiento significativamente mejor que los modelos con ENN.

6.10. Resultados y Discusión

El análisis de los múltiples modelos y técnicas de preprocesamiento muestra que los algoritmos de *Gradient Boosting* (*LightGBM*, *XGBoost*, *CatBoost*) presentan una clara superioridad frente al resto, tanto en métricas globales como en la clasificación de las clases más complejas. Dentro de este grupo, *LightGBM* se posiciona como el modelo más eficaz y equilibrado: no solo alcanza la mayor precisión global y el mejor coeficiente de correlación de Matthews (MCC), sino que además logra el mayor *F1-score* en la clase más difícil de diferenciar, demostrando una capacidad notable de generalización con un sobreajuste mínimo. Muy cerca se sitúa *XGBoost*, que con un submuestreo adecuado ofrece métricas muy competitivas y una gran eficiencia computacional, mientras que *CatBoost* destaca por su estabilidad y facilidad de uso, aunque con un rendimiento ligeramente inferior. En un segundo nivel se encuentran SVC y las redes neuronales (MLP), que alcanzan buenos resultados cuando se aplican técnicas de muestreo y logran métricas respetables, aunque con limitaciones importantes: SVC muestra dificultades consistentes para diferenciar correctamente la clase minoritaria, mientras que MLP presenta una tendencia acusada al sobreajuste y un elevado coste computacional. Los modelos más clásicos como KNN, *Random Forest* y *Decision Tree* se sitúan claramente en un escalón inferior. KNN obtiene métricas aceptables en validación, pero sufre de sobreajuste extremo, lo que compromete su fiabilidad. *Random Forest*, aunque muy estable, muestra un desempeño predictivo insuficiente, especialmente en la clase más compleja, mientras que *Decision Tree* confirma las expectativas iniciales al ser el peor de los modelos evaluados, incluso tras aplicar técnicas de balanceo. En conjunto, los resultados establecen una jerarquía clara: los modelos de *Gradient Boosting* son la mejor opción, siendo *LightGBM* el algoritmo más prometedor para problemas de clasificación multiclase desbalanceados y con fronteras de decisión complejas, seguido de cerca por *XGBoost* y *CatBoost*; en contraste, los modelos tradicionales quedan descartados como alternativas viables en este escenario.

El estudio también permite reflexionar sobre la aplicabilidad práctica de los modelos. Un aspecto clave observado es que las técnicas de balanceo de clases no siempre resultan beneficiosas: en modelos complejos como las redes neuronales, el uso de *SMOTE* tiende a generar sobreajuste, mientras que en algoritmos de *Gradient Boosting* la intervención de técnicas de muestreo apenas aporta mejoras, e incluso puede ser contraproducente. En cambio, las estrategias de submuestreo resultaron más estables, especialmente en modelos como SVC, lo que confirma que el preprocesamiento debe evaluarse de manera conjunta con el algoritmo. Otro factor relevante es el coste computacional: mientras que *LightGBM* y *CatBoost* destacan por su rapidez, el entrenamiento de MLP requiere más tiempo, lo que limita su uso. Finalmente, cabe destacar la importancia de la interpretabilidad. Aunque los modelos de *Gradient Boosting* lideran en rendimiento, su complejidad interna plantea la necesidad de técnicas de inteligencia artificial explicable (*XAI*) para entender qué variables guían sus decisiones, lo cual resulta esencial si se busca un despliegue en entornos críticos como la ciberseguridad.

Parte III

Epílogo

7. Conclusiones

En este Trabajo de Fin de Máster, se ha llevado a cabo un estudio riguroso y exhaustivo sobre la clasificación de niveles de riesgo, abordando el problema desde una perspectiva comparativa de múltiples algoritmos de *Machine Learning* y estrategias de preprocesamiento de datos. El objetivo principal ha sido no solo aplicar técnicas, sino también realizar una evaluación sistemática para un problema de clasificación multiclase con un desbalance significativo y fronteras de decisión complejas.

A lo largo del estudio, se ha puesto bastante enfásis en la capacidad de los modelos para diferenciar correctamente entre clases relacionadas, particularmente la clase "Riesgo bajo", que ha demostrado ser la más complicada de diferenciar. Este desafío es el objetivo de muchos problemas del mundo real, donde los errores en clases minoritarias o de difícil separación son bastante comunes. Se ha explorado un amplio rango de algoritmos, desde modelos clásicos como árboles de decisión, hasta ensamblados avanzados como *RandomForest* y *Gradient Boosting* (*CatBoost*, *XGBoost*, *LightGBM*), pasando por modelos basados en distancia (KNN), máquinas de soporte vectorial (SVC) y redes neuronales profundas (MLP).

Otro de los aspectos contra el que se ha luchado ha sido el desbalanceo de las clases que se ha podido solucionar incluyendo sobremuestreo (*SMOTE*), submuestreo (genérico y Cluster Centroids y ENN). La elección de la técnica de balanceo demostró ser un factor crítico. *SMOTE* tendió a generar sobreajuste en modelos complejos como las redes neuronales, mientras que para los potentes algoritmos de *boosting* resultó ser una intervención innecesaria o incluso perjudicial. Por el contrario, las estrategias de submuestreo ofrecieron un camino más robusto y estable en los distintos escenarios.

Los algoritmos de *Gradient Boosting*, y en particular ***LightGBM***, demuestran una capacidad excepcional para gestionar la complejidad del problema. El modelo *LightGBM* no solo alcanzó las mejores métricas globales (**MCC de 0,8209** y ***Accuracy* de 0,8659**), sino que, de forma decisiva, resolvió con mayor eficacia el punto más débil del problema, obteniendo el *F1-score* más alto para la clase "Riesgo bajo" (*0,*7766). Los modelos SVC y MLP, con el preprocesamiento adecuado, se posicionaron como alternativas competentes, aunque con desventajas en términos de coste computacional y propensión al sobreajuste. Finalmente, los modelos KNN, *RandomForest* y *DecisionTree* mostraron limitaciones fundamentales, especialmente en su capacidad para clasificar correctamente la clase "Riesgo bajo", lo que los convierte en opciones menos viables para este caso de uso.

Trabajo Futuro

- **Ingeniería de características avanzada:** explorar la creación de nuevas variables a partir de las existentes que puedan ayudar a los modelos a diferenciar mejor entre las clases, especialmente entre las categorías de riesgo adyacentes.

- **Ensamblado de modelos (*Stacking*):** investigar la combinación de las predicciones de los mejores modelos individuales (p. ej., *LightGBM*, *XGBoost* y SVC) mediante un meta-modelo.
- **Extensión a la clasificación de familias de *malware*:** una vez que un fichero es identificado como de "Riesgo alto", se podría entrenar un nuevo modelo multiclase para determinar su familia (p.ej., *Ransomware*, *Spyware*, *Troyano*, *Adware*).
- **Inteligencia artificial explicable (XAI):** aplicar técnicas como SHAP (*SHapley Additive exPlanations*) o LIME (*Local Interpretable Model-agnostic Explanations*) sobre el modelo ganador (LightGBM) para entender qué características son las más influyentes en sus decisiones.
- **Desarrollo de una herramienta de análisis en tiempo real:** crear una aplicación web o un servicio API que materialice los resultados de este trabajo. Dicha herramienta permitiría a un usuario enviar el *hash* de un fichero (p.ej., SHA-256) y, a través de las peticiones necesarias para obtener sus características, devolvería una evaluación de riesgo basada en las probabilidades generadas por el modelo final, ofreciendo un servicio de análisis práctico e inmediato.

Bibliografía

abuse.ch (n.d.). Malwarebazaar. https://bazaar.abuse.ch/.

Amazon Web Services (2024). ¿qué es una red neuronal? https://aws.amazon.com/es/what-is/neural-network/.

Arlot, S. y Celisse, A. (2010). A survey of cross-validation procedures for model selection. *Statistics Surveys*, 4:40–79.

Baghirov, E. (2025). A comprehensive investigation into robust malware detection with explainable ai. *Cyber Security and Applications*, 3:100072.

Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.

Breiman, L. (2001). Random forests. *Machine Learning*, 45(1):5–32.

Breiman, L., Friedman, J. H., Olshen, R. A., y Stone, C. J. (1984). *Classification and Regression Trees*. Wadsworth International Group.

Center, C. C. (1999). Cih virus: The first bios killer. *Security Bulletin*.

Chawla, N. V., Bowyer, K. W., Hall, L. O., y Kegelmeyer, W. P. (2002). Smote: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357.

Chen, T. y Guestrin, C. (2016). Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794. ACM.

Cortes, C. y Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3):273–297.

DataCamp (2023). Multilayer perceptrons in machine learning. https://www.datacamp.com/tutorial/multilayer-perceptrons-in-machine-learning.

DIDA.do (2023). What is random forest? https://dida.do/what-is-random-forest.

Dorogush, A. V., Ershov, V., y Gulin, A. (2018). Catboost: unbiased boosting with categorical features. *Advances in Neural Information Processing Systems*, 31.

Egele, M., Scholte, T., Kirda, E., y Kruegel, C. (2012). A survey on automated dynamic malware analysis techniques and tools. *ACM Computing Surveys*, 44(2):1–42.

Encord (2023). Classification metrics: Accuracy, precision, recall, explained. https://encord.com/blog/classification-metrics-accuracy-precision-recall/.

Fernández, A. (2020). Code a decision tree in python from scratch. https://anderfernandez.com/en/blog/code-decision-tree-python-from-scratch/.

Fernández, A., García, S., Galar, M., Prati, R. C., Krawczyk, B., y Herrera, F. (2018). *Learning from Imbalanced Data Sets*. Springer.

Foundation, P. S. (2024). The python tutorial — python 3 documentation. https://docs.python.org/3/tutorial/index.html.

Ghosh, S. (2020). Bank data & smote. https://medium.com/analytics-vidhya/bank-data-smote-b5cb01a5e0a2.

Glorot, X., Bordes, A., y Bengio, Y. (2011). Deep sparse rectifier neural networks. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, pages 315–323.

Goodfellow, I., Bengio, Y., y Courville, A. (2016). *Deep Learning*. MIT Press.

Group, C. (2024). Cyberthreat defense report 2024.

Group, T. I. (2017). Notpetya: Timeline of a destructive cyberattack. *Cisco Talos*.

Gupta, A. (2023). Classification model - svm classifier python example. https://vitalflux.com/classification-model-svm-classifier-python-example/.

Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems*. O'Reilly Media, 2nd edition.

Hastie, T., Tibshirani, R., y Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer.

Hoglund, G. y Butler, J. (2005). *Rootkits: Subverting the Windows Kernel*. Addison-Wesley.

Hornik, K., Stinchcombe, M., y White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366.

Huang, H., Huang, W., Zhou, Y., Luo, W., y Wang, Y. (2025). Fedroid: A lightweight and interpretable machine learning-based android malware detection system. *Cluster Computing*, 28(4).

IABAC (2024). The branches of artificial intelligence. *IABAC Blog*.

IBM (2023). Ejemplos, aplicaciones y casos de uso de machine learning.

IBM (2024). ¿qué es la inteligencia artificial explicable?

IBM (n.d.). What is feature engineering? https://www.ibm.com/think/topics/feature-engineering.

J2Logo (2024). Python requests: cómo hacer peticiones http en python. https://j2logo.com/python/python-requests-peticiones-http/.

Japkowicz, N. y Stephen, S. (2002). Class imbalance problem: A systematic study. In *Proceedings of the International Conference on Intelligent Data Analysis*, pages 111–117. Springer.

Kareem, M. (2020). Understanding lightgbm — a guide to its innovations and usage. https://towardsdatascience.com/understanding-lightgbm-a-guide-to-its-innovations-and-usage-9265f1447b56.

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., y Liu, T.-Y. (2017). Lightgbm: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30.

Kushwaha, R. (2021). Smart aspects of catboost algorithm. https://python.plainenglish.io/smart-aspects-of-catboost-algorithm-2720a6de4da6.

Lab, K. (2017). Wannacry ransomware attack analysis. *Kaspersky Reports*.

learn Developers, I. (2024a). imblearn.over_sampling.smote. https://imbalanced-learn.org/stable/references/generated/imblearn.over_sampling.SMOTE.html.

learn Developers, I. (2024b). imblearn.under_sampling.clustercentroids. https://imbalanced-learn.org/stable/references/generated/imblearn.under_sampling.ClusterCentroids.html.

learn Developers, I. (2024c). imblearn.under_sampling.editednearestneighbours. https://imbalanced-learn.org/stable/references/generated/imblearn.under_sampling.EditedNearestNeighbours.html.

learn Developers, I. (2024d). imblearn.under_sampling.randomundersampler. https://imbalanced-learn.org/stable/references/generated/imblearn.under_sampling.RandomUnderSampler.html.

learn Developers, S. (2024e). Cross-validation: evaluating estimator performance. https://scikit-learn.org/stable/modules/cross_validation.html.

Mandot, P. (2021). What is lightgbm? how to implement it? how to fine-tune the parameters? https://medium.com/@pushkarmandot/https-medium-com-pushkarmandot-what-is-lightgbm-how-to-implement-it-how-to-fine-t

Manikanta Prasad, J., Abhijna, C. D., Aishwarya, A. S., Sai Pranam, B. R., y Raghuramegowda, S. M. (2024). Malware detection using machine learning. In *2024 Second International Conference on Advances in Information Technology (ICAIT)*, pages 1–5. IEEE.

Micro, T. (n.d.). Tlsh - trend micro locality sensitive hash. https://github.com/trendmicro/tlsh. Accedido el 20 de mayo de 2025.

Microsoft (2024). Lightgbm documentation. https://lightgbm.readthedocs.io/.

Microsoft (2025). ¿qué es la ciberseguridad?

Mitchell, T. (2018). The paradigm: A concise explanation of learning algorithms. https://www.kdnuggets.com/2018/10/mitchell-paradigm-concise-explanation-learning-algorithms.html.

Mitchell, T. M. (1997). *Machine Learning*. McGraw-Hill Education.

Nigam, A. (2021). Understanding how xgboost works. https://medium.com/analytics-vidhya/understanding-how-xgboost-works-7caa4d1a5b8c.

Omarzai, F. (2021). Xgboost regression in-depth. https://medium.com/@fraidoonomarzai99/xgboost-regression-in-depth-cb2b3f623281.

Omopintemi, A. H., Ghafir, I., Eltanani, S., Kabir, S., y Lefoane, M. (2023). Machine learning for malware detection in network traffic. In *Proceedings of the 7th International Conference on Future Networks and Distributed Systems*, pages 605–610.

Patel, D. (2022). How to avoid overfitting in machine learning. https://www.kdnuggets.com/2022/08/avoid-overfitting.html.

Powers, D. M. W. (2011). Evaluation: From precision, recall and f-measure to roc, informedness, markedness and correlation. *Journal of Machine Learning Technologies*, 2(1):37–63.

Project Jupyter (2024). *Jupyter Notebook Documentation*.

Quinlan, J. R. (1993). *C4.5: Programs for Machine Learning*. Morgan Kaufmann.

ResearchGate (2020a). Illustration of random undersampling technique [figura]. https://www.researchgate.net/figure/llustration-of-random-undersampling-technique_fig3_343326638.

ResearchGate (2020b). Training a balanced k-means clustering model (k=3) [figura]. https://www.researchgate.net/figure/Training-a-balanced-k-means-clustering-model-k3documentclass12ptminimal_fig3_351486637.

ResearchGate (2023). Edited nearest neighbors [figura]. https://www.researchgate.net/figure/Edited-Nearest-Neighbors_fig6_371936273.

Response, S. S. (2016). Locky ransomware spreads via email campaign. *Symantec Blog*.

Ring, M., Wunderlich, S., Scheuring, D., Landes, D., y Hotho, A. (2019). A survey of network-based intrusion detection data sets. *Computers & Security*, 86:147–167.

scikit-learn developers (2023a). sklearn.preprocessing.labelencoder — scikit-learn 1.3.2 documentation.

scikit-learn developers (2023b). sklearn.preprocessing.standardscaler — scikit-learn 1.3.2 documentation.

Scikit-learn Developers (n.d.). Getting started. https://scikit-learn.org/stable/getting_started.html.

Security, G. D. (2020). Types of malware and how to recognize them. *G DATA Whitepaper*.

Shabtai, A., Moskovitch, R., Elovici, Y., y Glezer, S. (2012). Detection of malicious code by applying machine learning classifiers on static features: A state-of-the-art survey. *Information Security Technical Report*, 14(1):16–29.

Sharma, A., Babbar, H., y Vats, A. K. (2024). Securing the internet of things: Using machine learning for malware detection with cic-malmem dataset. In *2024 4th International Conference on Innovative Practices in Technology and Management (ICIPTM)*, pages 1–5.

Sharma, R. (2020). A gentle introduction to catboost for machine learning. https://towardsdatascience.com/a-gentle-introduction-to-catboost-for-machine-learning-4cfa896defb2.

Shelke, M. V., Deshmukh, J. Y., Ajalkar, D. A., y Dhumal, R. B. (2024). A robust ensemble learning approach for malware detection and classification. *Journal of Advanced Research in Applied Sciences and Engineering Technology*, 48(1):152–167.

Shukla, A. (2020). Xgboost algorithm: Long may it reign! https://towardsdatascience.com/xgboost-algorithm-long-may-it-reign-edd174425f34.

Singh, A. y De, T. (2016). Botnet detection based on system behavior. *International Journal of Computer Applications*, 133(3):1–6.

Skoudis, E. y Zeltser, L. (2003). *Malware: Fighting Malicious Code*. Prentice Hall.

Sommer, R. y Paxson, V. (2010). Outside the closed world: On using machine learning for network intrusion detection. In *2010 IEEE Symposium on Security and Privacy*, pages 305–316. IEEE.

Soni, S. (2019). K-nearest neighbours - introduction to machine learning algorithms. https://medium.com/@sachinsoni600517/k-nearest-neighbours-introduction-to-machine-learning-algorithms-9dbc9d9fb3b2.

Stallings, W. (2020). *Computer Security: Principles and Practice*. Pearson.

Symantec (2019). Intrusion detection systems and the role of machine learning. https://symantec-enterprise-blogs.security.com/blogs/feature-stories/intrusion-detection-machine-learning.

TensorFlow Developers (2025). *Keras: The high-level API for TensorFlow*.

The imbalanced-learn Developers (2025). *imbalanced-learn User Guide, Version 0.13.0*.

The Matplotlib Development Team (2025). *Matplotlib User Guide, Release 3.10.3*.

The NumPy Community (2025). *NumPy User Guide, Release 2.2.0*.

The pandas Development Team (2025). *pandas User Guide, Release 2.2.3*.

Tridgell, A. (2009). ssdeep – context triggered piecewise hashes. https://ssdeep-project.github.io/.

Vapnik, V. N. (1998). *Statistical Learning Theory*. Wiley.

VirusTotal (2024). How it works.

Waskom, M. y the Seaborn Development Team (2025). Introduction to seaborn. https://seaborn.pydata.org/tutorial/introduction.html.

Wilson, D. L. (1972). Asymptotic properties of nearest neighbor rules using edited data. *IEEE Transactions on Systems, Man, and Cybernetics*, 2(3):408–421.

Wueest, C. (2013). The risks of automated malware analysis. https://www.symantec.com. Symantec Security Response report.

XGBoost Developers (2024). Xgboost documentation (release 3.0.0). https://xgboost.readthedocs.io/en/release_3.0.0/.

Yandex (2024). Catboost: Fast, scalable, high-performance gradient boosting. https://catboost.ai/.

You, I. y Yim, K. (2010). Malware obfuscation techniques: A brief survey. In *2010 International Conference on Broadband, Wireless Computing, Communication and Applications (BWCCA)*, pages 297–300. IEEE.